

SD403 Trailcam IV : Group 6

Source Code Reference

Cole Zenk

April 24, 2026

Contents

esp/esp32_devkitv1/main/inc/cam_spi.h	2
esp/esp32_devkitv1/main/inc/config.h	3
esp/esp32_devkitv1/main/inc/fpga_spi.h	8
esp/esp32_devkitv1/main/inc/image_processor.h	9
esp/esp32_devkitv1/main/inc/isr_signals.h	10
esp/esp32_devkitv1/main/inc/lora.h	11
esp/esp32_devkitv1/main/inc/peripherals/aht20.h	12
esp/esp32_devkitv1/main/inc/peripherals/fpga_gpio.h	13
esp/esp32_devkitv1/main/inc/peripherals/i2c_bus.h	14
esp/esp32_devkitv1/main/inc/peripherals/motor.h	15
esp/esp32_devkitv1/main/inc/peripherals/pir.h	16
esp/esp32_devkitv1/main/inc/peripherals/sensors_temp_humidity.h	17
esp/esp32_devkitv1/main/inc/utils.h	18
esp/esp32_devkitv1/main/src/interprocessor/cam_spi.c	20
esp/esp32_devkitv1/main/src/interprocessor/fpga_spi.c	24
esp/esp32_devkitv1/main/src/interprocessor/i2c_bus.c	29
esp/esp32_devkitv1/main/src/interprocessor/lora.c	34
esp/esp32_devkitv1/main/src/main.c	39
esp/esp32_devkitv1/main/src/peripherals/aht20.c	41
esp/esp32_devkitv1/main/src/peripherals/fpga_gpio.c	43
esp/esp32_devkitv1/main/src/peripherals/motor.c	46
esp/esp32_devkitv1/main/src/peripherals/pir.c	48
esp/esp32_devkitv1/main/src/peripherals/sensors_temp_humidity.c	49
esp/esp32_devkitv1/main/src/utilities/image_processor.c	53
esp/esp32_devkitv1/main/src/utilities/isr.c	55
esp/esp32_devkitv1/main/src/utilities/utils.c	56
esp/esp32cam/main/inc/cam_ctrl.h	58

esp/esp32cam/main/inc/dma_spi.h	59
esp/esp32cam/main/inc/img_buf.h	60
esp/esp32cam/main/inc/img_data.h	62
esp/esp32cam/main/inc/ov2640_cfg.h	63
esp/esp32cam/main/inc/sd_stg.h	64
esp/esp32cam/main/src/cam_ctrl.c	66
esp/esp32cam/main/src/dma_spi.c	71
esp/esp32cam/main/src/img_buf.c	75
esp/esp32cam/main/src/main.c	80
esp/esp32cam/main/src/ov2640_cfg.c	82
esp/esp32cam/main/src/sd_stg.c	84
esp/esp32s3_wroom/main/inc/config.h	87
esp/esp32s3_wroom/main/inc/image_reconstruct.h	89
esp/esp32s3_wroom/main/inc/lora.h	90
esp/esp32s3_wroom/main/inc/wifi_ap.h	91
esp/esp32s3_wroom/main/inc/ws_server.h	92
esp/esp32s3_wroom/main/src/image_reconstruct.c	93
esp/esp32s3_wroom/main/src/lora.c	96
esp/esp32s3_wroom/main/src/main.c	100
esp/esp32s3_wroom/main/src/wifi_ap.c	102
esp/esp32s3_wroom/main/src/ws_server.c	104
fpga/src/rtl/blk_schd.v	108
fpga/src/rtl/bram_image_buffer.v	114
fpga/src/rtl/coeff.v	115
fpga/src/rtl/esp_interface.v	117
fpga/src/rtl/gowin_rpll.v	120
fpga/src/rtl/i2c_gpio_expander.v	122
fpga/src/rtl/lcd_controller.v	128
fpga/src/rtl/psram-tang-nano-9k/src/gowin_rpll/gowin_rpll.v	132
fpga/src/rtl/psram-tang-nano-9k/src/gowin_rpll/gowin_rpll_tmp.v	134
fpga/src/rtl/psram-tang-nano-9k/src/print.v	135
fpga/src/rtl/psram-tang-nano-9k/src/psram_controller.v	137
fpga/src/rtl/psram-tang-nano-9k/src/psram_test_top.v	141
fpga/src/rtl/psram-tang-nano-9k/src/uart_tx.v	145
fpga/src/rtl/psram_controller.v	147

fpga/src/rtl/seq_msk.v	151
fpga/src/rtl/top.v	152
fpga/src/rtl/wht2d.v	159
fpga/src/rtl/wht8.v	160

esp/esp32_devkitv1/main/inc/cam_spi.h

```
1  #ifndef CAM_SPI_H
2  #define CAM_SPI_H
3
4  #include "esp_err.h"
5  #include "freertos/FreeRTOS.h"
6  #include "freertos/queue.h"
7  #include "config.h"
8
9  /*****
10   * Camera SPI Slave Interface
11   *
12   * Receives image data from ESP32-CAM via SPI slave DMA
13   * Hot path optimized for minimal latency
14   *****/
15
16 /**
17  * Initialize camera SPI slave interface
18  *
19  * @return ESP_OK on success, error code otherwise
20  */
21 esp_err_t cam_spi_init(void);
22
23 /**
24  * Deinitialize camera SPI slave interface
25  */
26 void cam_spi_deinit(void);
27
28 /**
29  * Get queue handle for received images
30  *
31  * @return Queue handle or NULL if not initialized
32  */
33 QueueHandle_t cam_spi_get_queue(void);
34
35 /**
36  * SPI receive task - should be run as FreeRTOS task
37  * Marked as hot function for optimization
38  *
39  * @param pvParameters Task parameters (unused)
40  */
41 HOT_FUNCTION void cam_spi_receive_task(void *pvParameters);
42
43 #endif // CAM_SPI_H
```

esp/esp32_devkitv1/main/inc/config.h

```
1  #ifndef CONFIG_H
2  #define CONFIG_H
3
4  #include "esp_err.h"
5  #include "driver/spi_common.h"
6  #include "driver/gpio.h"
7  #include "freertos/FreeRTOS.h"
8
9  /*****
10   * Build Configuration & Test Modes
11   *****/
12
13  /**
14   * Test mode configuration - similar to WarblingWire pattern
15   * Uncomment ONE of these to enable specific test modes
16   */
17  // #define TEST_MODE_FPGA_PATTERNS
18  /* #define TEST_MODE_LORA_LOOPBACK */
19  /* #define TEST_MODE_CAMERA_INJECT */
20  /* #define TEST_MODE_FPGA_GPIO */
21  // #define TEST_MODE_LORA_BENCH
22  #define DEBUG_PIR_ALWAYS_ON
23
24  // Verify only one test mode is active
25  #if defined(TEST_MODE_FPGA_PATTERNS) && defined(TEST_MODE_LORA_LOOPBACK)
26      #error "Multiple test modes enabled - choose only one"
27  #endif
28  #if defined(TEST_MODE_LORA_BENCH) && (defined(TEST_MODE_FPGA_PATTERNS) || defined(
29      TEST_MODE_LORA_LOOPBACK))
30      #error "LORA_BENCH is standalone - disable other test modes"
31  #endif
32
33  // Performance optimization level
34  #define OPTIMIZE_FOR_SPEED 1 // Set to 0 for size optimization
35
36  /*****
37   * System Configuration - Computed Constants
38   *****/
39
40  // Memory allocation
41  #define IMAGE_MAX_SIZE_BYTES (1024 * 1024)
42  #define IMAGE_QUEUE_LENGTH 3
43  #define SPI_BUFFER_SIZE 4096
44  #define UART_BUFFER_SIZE 2048
45
46  // Computed timeout values (compile-time calculations)
47  #define MS_TO_TICKS(ms) pdMS_TO_TICKS(ms)
48  #define SECONDS_TO_TICKS(s) pdMS_TO_TICKS((s) * 1000)
49
50  // Task priorities
51  #define TASK_PRIORITY_HIGH 5
52  #define TASK_PRIORITY_MEDIUM 4
53  #define TASK_PRIORITY_LOW 3
54
55  #define STACK_SIZE_BIT_WIDTH_S 11
56  #define STACK_SIZE_BIT_WIDTH_M 12
57  #define STACK_SIZE_BIT_WIDTH_L 13
58
59  // Task stack sizes
60  #define STACK_SIZE_SMALL (1 << STACK_SIZE_BIT_WIDTH_S) // 2048
61  #define STACK_SIZE_MEDIUM (1 << STACK_SIZE_BIT_WIDTH_M) // 4096
62  #define STACK_SIZE_LARGE (1 << STACK_SIZE_BIT_WIDTH_L) // 8192
63
64  /*****
65   * Camera SPI Slave Configuration (receiving from ESP32-CAM)
66   *****/
```

```

67 #define CAM_SPI_HOST          SPI2_HOST
68 #define CAM_SPI_DMA_CHAN      SPI_DMA_CH_AUTO
69
70 #define CAM_PIN_MOSI          GPIO_NUM_23
71 #define CAM_PIN_MISO          GPIO_NUM_19
72 #define CAM_PIN_SCLK          GPIO_NUM_18
73 #define CAM_PIN_CS            GPIO_NUM_5
74
75 #define CAM_SPI_MODE           0
76 #define CAM_SPI_QUEUE_SIZE     3
77 #define CAM_TRANSFER_TIMEOUT_MS 5000
78 #define CAM_TRANSFER_TIMEOUT_TICKS MS_TO_TICKS(CAM_TRANSFER_TIMEOUT_MS)
79
80 // Camera reception strategy
81 #define CAM_USE_ZERO_COPY      1 // DMA directly to final buffer when possible
82
83 /*****
84  * FPGA SPI Master Configuration (sending to Tang Nano 9K)
85  *****/
86
87 #define FPGA_SPI_HOST          SPI3_HOST
88 #define FPGA_SPI_DMA_CHAN      SPI_DMA_CH_AUTO
89
90 #define FPGA_PIN_MOSI          GPIO_NUM_32
91 #define FPGA_PIN_MISO          GPIO_NUM_33
92 #define FPGA_PIN_SCLK          GPIO_NUM_25
93 #define FPGA_PIN_CS            GPIO_NUM_26
94
95 #define FPGA_SPI_CLOCK_MHZ     9
96 #define FPGA_SPI_CLOCK_HZ      (FPGA_SPI_CLOCK_MHZ * 1000000)
97 #define FPGA_SPI_MODE           0
98 #define FPGA_SPI_QUEUE_SIZE     3
99 #define FPGA_MAX_TRANSFER_SIZE 76800 /* full BRAM - single CS transaction, no mid-frame
    resets */
100
101 /*****
102  * FPGA I2C GPIO Expander Configuration (PCA9534-compatible at 0x27)
103  *****/
104
105 #define FPGA_I2C_PORT          I2C_NUM_0
106 #define FPGA_I2C_SDA_PIN       GPIO_NUM_21
107 #define FPGA_I2C_SCL_PIN       GPIO_NUM_22
108 #define FPGA_I2C_CLK_HZ        100000 /* 100 kHz - safe for LVCMOS18 */
109 #define FPGA_I2C_ADDR          0x27
110 #define FPGA_I2C_TIMEOUT_MS    100
111
112 /* Register map (PCA9534-compatible) */
113 #define FPGA_GPIO_REG_OUT      0x00
114 #define FPGA_GPIO_REG_IN       0x01
115 #define FPGA_GPIO_REG_DIR      0x02
116 #define FPGA_GPIO_REG_INV      0x03
117
118 /* Boot config: bits[7:4]=inputs(unused), bits[3:0]=outputs(stepers) */
119 #define FPGA_GPIO_DIR_BOOT     0xF0
120 /* Invert bits[2:0] so button reads 1=pressed (active-low with pull-up) */
121 #define FPGA_GPIO_INV_BOOT     0x07
122
123 /* Button bit positions in INPUT_PORT after invert applied */
124 #define FPGA_BTN_L_BIT         0 /* gpio_in[0] = button_L, pin 84 */
125 #define FPGA_BTN_R_BIT         1 /* gpio_in[1] = button_R, pin 83 */
126 #define FPGA_BTN_S_BIT         2 /* gpio_in[2] = button_S, pin 85 */
127
128 /*****
129  * LoRa UART Configuration
130  *****/
131
132 #define LORA_UART_NUM           UART_NUM_2
133 #define LORA_PIN_TX             GPIO_NUM_17
134 #define LORA_PIN_RX             GPIO_NUM_16
135 #define LORA_BAUD_RATE          115200

```

```

136
137 #define LORA_ADDRESS_RECEIVER      2
138 #define LORA_ADDRESS_SENDER        2
139 #define LORA_NETWORK_ID            6
140
141 #define LORA_AT_TIMEOUT_MS          500
142 #define LORA_AT_TIMEOUT_TICKS      MS_TO_TICKS(LORA_AT_TIMEOUT_MS)
143 #define LORA_INIT_DELAY_MS          300
144 #define LORA_INIT_DELAY_TICKS      MS_TO_TICKS(LORA_INIT_DELAY_MS)
145
146 // LoRa polling configuration
147 #define LORA_POLL_INTERVAL_MS       100
148 #define LORA_POLL_INTERVAL_TICKS   MS_TO_TICKS(LORA_POLL_INTERVAL_MS)
149 #define LORA_HEARTBEAT_COUNT        100 // Log every N polls
150
151 // Trivial constants
152 #define NUM_BITS_RXTX_BUF           11
153
154 /*****
155  * LoRa Bench Test Configuration (TEST_MODE_LORA_BENCH)
156  *
157  * MUST match the values in ESP_Ecosystem/esp32s3_wroom/main/inc/config.h.
158  *
159  * RYLR998 AT+PARAMETER: AT+PARAMETER=<SF>,<BW>,<CR>,<Preamble>
160  *   BW: 7=125 kHz  8=250 kHz  9=500 kHz
161  *   SF: 7-12  (lower = faster, less range)
162  *
163  * Start with SF=9/BW=7 (defaults) to confirm link, then switch to SF=7/BW=9
164  * to test 6-FPS throughput target. Both boards must be reflashed each time.
165  *****/
166
167 /*****
168  * LoRa RF Parameters (must match S3 config.h)
169  *****/
170 #define LORA_SF          7          /* spreading factor (7-12) */
171 #define LORA_BW          9          /* 7=125kHz 8=250kHz 9=500kHz */
172 #define LORA_CR          1          /* coding rate: 1=4/5 */
173 #define LORA_PREAMBLE    12
174 #define LORA_DEST_ADDR   1          /* S3 receiver address */
175 #define LORA_MAX_PAYLOAD 240        /* RYLR998 max */
176
177 #ifdef TEST_MODE_LORA_BENCH
178     #define BENCH_SF      LORA_SF
179     #define BENCH_BW      LORA_BW
180     #define BENCH_CR      LORA_CR
181     #define BENCH_PREAMBLE LORA_PREAMBLE
182     #define BENCH_DEST_ADDR LORA_DEST_ADDR
183     #define BENCH_PKT_SIZE 16
184     #define BENCH_N        50
185     #define BENCH_TIMEOUT_MS 10000
186     #define BENCH_INTER_PACKET_MS 10
187     #define BENCH_BURST_SECS 10
188     #define BENCH_BURST_TIMEOUT_MS 500
189     #warning "LoRa bench mode -- DevKitV1 is TX master, S3 must be echo slave"
190 #endif
191
192 /*****
193  * Test/Debug Configuration
194  *****/
195
196 #define TEST_BUTTON_PIN    GPIO_NUM_0
197 #define TEST_LED_PIN       GPIO_NUM_2
198 #define TEST_IMAGE_SIZE    32768 // FPGA BRAM size (Tang Nano 9K)
199
200 // Test pattern configuration
201 #ifdef TEST_MODE_FPGA_PATTERNS
202     #define TEST_PATTERN_COUNT 4
203     #define TEST_PATTERN_HOLD_MS 2000
204     #warning "FPGA test pattern mode enabled"
205 #endif

```

```

206
207 #ifdef TEST_MODE_LORA_LOOPBACK
208     #warning "LoRa loopback test mode enabled"
209 #endif
210
211 #ifdef TEST_MODE_CAMERA_INJECT
212     #define TEST_INJECT_INTERVAL_MS 5000
213     #warning "Camera injection test mode enabled"
214 #endif
215
216 /*****
217  * Performance Attributes - Compiler Hints
218  *****/
219
220 // Hot path optimization (functions called frequently)
221 #if OPTIMIZE_FOR_SPEED
222     #define HOT_FUNCTION      __attribute__((hot))
223     #define COLD_FUNCTION     __attribute__((cold))
224     #define ALWAYS_INLINE     __attribute__((always_inline)) static inline
225     #define LIKELY(x)         __builtin_expect(!!(x), 1)
226     #define UNLIKELY(x)       __builtin_expect(!!(x), 0)
227 #else
228     #define HOT_FUNCTION
229     #define COLD_FUNCTION
230     #define ALWAYS_INLINE     static inline
231     #define LIKELY(x)         (x)
232     #define UNLIKELY(x)       (x)
233 #endif
234
235 // ISR functions (critical timing)
236 #define ISR_FUNCTION          __attribute__((interrupt))
237
238 // Alignment for DMA buffers
239 #define DMA_ALIGNED           __attribute__((aligned(4)))
240
241 /*****
242  * Protocol Definitions
243  *****/
244
245 #define IMAGE_HEADER_MAGIC    0xCAFEFEEF
246 #define IMAGE_HEADER_SIZE    28
247
248 // Image header structure (packed for wire protocol)
249 typedef struct {
250     uint32_t magic;
251     uint32_t size;
252     uint32_t checksum;
253     uint32_t timestamp;
254     uint16_t width;
255     uint16_t height;
256     uint8_t format;
257     uint8_t reserved[7];
258 } __attribute__((packed)) image_header_t;
259
260 _Static_assert(sizeof(image_header_t) == IMAGE_HEADER_SIZE,
261     "Image header must be 24 bytes");
262
263 // Image data container
264 typedef struct {
265     uint8_t *buffer;
266     size_t size;
267     image_header_t header;
268 } image_data_t;
269
270 /*****
271  * Validation Macros - Compile-Time Checks
272  *****/
273
274 // Ensure buffer sizes are reasonable
275 _Static_assert(SPI_BUFFER_SIZE >= 1024, "SPI buffer too small");

```



```
276 _Static_assert(SPI_BUFFER_SIZE <= 8192, "SPI buffer too large for DMA");
277 _Static_assert((SPI_BUFFER_SIZE % 4) == 0, "SPI buffer must be 4-byte aligned");
278
279 // Ensure queue depth is reasonable
280 _Static_assert(IMAGE_QUEUE_LENGTH >= 1, "Need at least 1 queue slot");
281 _Static_assert(IMAGE_QUEUE_LENGTH <= 10, "Queue too deep - memory waste");
282
283 #endif // CONFIG_H
```

esp/esp32_devkitv1/main/inc/fpga_spi.h

```
1  #ifndef FPGA_SPI_H
2  #define FPGA_SPI_H
3
4  #include "esp_err.h"
5  #include "config.h"
6
7  /*****
8   * FPGA SPI Master Interface
9   *
10  * Sends image data to Tang Nano 9K FPGA via SPI master DMA
11  * Optimized for throughput
12  *****/
13
14 /**
15  * Initialize FPGA SPI master interface
16  *
17  * @return ESP_OK on success, error code otherwise
18  */
19 esp_err_t fpga_spi_init(void);
20
21 /**
22  * Deinitialize FPGA SPI master interface
23  */
24 void fpga_spi_deinit(void);
25
26 /**
27  * Transmit image data to FPGA
28  * Hot function - optimized for throughput
29  *
30  * @param data      Pointer to image data
31  * @param size      Size of data in bytes
32  * @return ESP_OK on success, error code otherwise
33  */
34 HOT_FUNCTION esp_err_t fpga_spi_transmit(const uint8_t *data, size_t size);
35
36 /**
37  * Receive data from FPGA
38  * Hot function - optimized for throughput
39  *
40  * @param data      Pointer to image data
41  * @param size      Size of data in bytes
42  * @return ESP_OK on success, error code otherwise
43  */
44 HOT_FUNCTION esp_err_t fpga_spi_receive(uint8_t *data, size_t size);
45
46 #ifdef TEST_MODE_FPGA_PATTERNS
47 /**
48  * Test task for FPGA interface
49  * Generates test patterns on button press (like WarblingWire outputTest)
50  *
51  * @param pvParameters Task parameters (unused)
52  */
53 void fpga_test_task(void *pvParameters);
54 #endif
55
56 #endif // FPGA_SPI_H
```

esp/esp32_devkitv1/main/inc/image_processor.h

```
1  /**
2   * @file image_processor.h
3   * @brief Image Processor - Camera SPI to FPGA Pipeline
4   * @author Cole Zenk
5   */
6
7  #ifndef IMAGE_PROCESSOR_H
8  #define IMAGE_PROCESSOR_H
9
10 #include "esp_err.h"
11 #include "freertos/FreeRTOS.h"
12
13 esp_err_t image_processor_init(void);
14 void image_processor_task(void *pvParameters);
15
16 #endif /* IMAGE_PROCESSOR_H */
```

esp/esp32_devkitv1/main/inc/isr_signals.h

```
1  /**
2   * @file isr_signals.h
3   * @brief ISR Shared Signals - Include in any file that needs ISR access
4   * @author Cole Zenk
5   */
6
7  #ifndef ISR_SIGNALS_H
8  #define ISR_SIGNALS_H
9
10 #include "freertos/FreeRTOS.h"
11 #include "freertos/semphr.h"
12 #include "driver/spi_master.h"
13 #include "driver/gptimer.h"
14 #include "esp_err.h"
15
16 /*****
17  SHARED SEMAPHORES
18  *****/
19 extern SemaphoreHandle_t g_trigger_sem;
20 extern SemaphoreHandle_t g_button_sem;
21 extern SemaphoreHandle_t g_spi_complete_sem;
22 extern SemaphoreHandle_t g_timer_capture_sem;
23
24 /*****
25  HANDLERS
26  *****/
27 esp_err_t isr_init(void);
28
29 void    ISR_OnLoRaTrigger(void);
30 void    ISR_OnButtonPress(void *arg);
31 void    ISR_OnSpiTransferComplete(spi_transaction_t *trans);
32 bool    ISR_OnTimerAlarm(gptimer_handle_t timer,
33                          const gptimer_alarm_event_data_t *event_data,
34                          void *user_ctx);
35
36 #endif /* ISR_SIGNALS_H */
```

esp/esp32_devkitv1/main/inc/lora.h

```
1 #pragma once
2
3 #include <stddef.h>
4 #include <stdint.h>
5 #include "config.h"
6 #include "esp_err.h"
7
8 esp_err_t lora_init(void);
9 esp_err_t lora_send_packet(const uint8_t *data, size_t len);
10
11 #ifdef TEST_MODE_LORA_BENCH
12 void lora_bench_task(void *arg);
13 #endif
```

esp/esp32_devkitv1/main/inc/peripherals/aht20.h

```
1  /**
2   * aht20.h - AHT20 temperature/humidity driver
3   *
4   * Depends on i2c_bus module. Call i2c_bus_init() before aht20_init().
5   */
6
7  #pragma once
8
9  #include "esp_err.h"
10 #include <stdint.h>
11
12 #define AHT20_I2C_ADDR_DEFAULT  0x38
13
14 typedef struct {
15     uint8_t addr;
16     int      timeout_ms;
17 } aht20_t;
18
19 esp_err_t aht20_init(aht20_t *dev, uint8_t dev_addr);
20
21 esp_err_t aht20_read(aht20_t *dev,
22                     float *temp_c,
23                     float *rh_percent);
```

esp/esp32_devkitv1/main/inc/peripherals/fpga_gpio.h

```
1  /**
2   * fpga_gpio.h - FPGA I2C GPIO Expander Driver
3   *
4   * PCA9534-compatible slave at address 0x27 on the Tang Nano 9K.
5   *
6   * Register map (set in config.h):
7   *   0x00  OUTPUT_PORT - drive stepper coils on bits[3:0]
8   *   0x01  INPUT_PORT - read buttons on bits[2:0] (after invert: 1=pressed)
9   *   0x02  DIR_REG - 1=input, 0=output (boot: 0xF0)
10  *   0x03  INVERT_REG - polarity invert (boot: 0x07)
11  *
12  * Wiring:
13  *   ESP32 GPIO_NUM_21 (SDA) 4.7k 3.3V FPGA pin 82
14  *   ESP32 GPIO_NUM_22 (SCL) 4.7k 3.3V FPGA pin 81
15  *   Common GND
16  *
17  * Requires i2c_bus to be initialised before calling fpga_gpio_init().
18  */
19
20 #pragma once
21
22 #include "esp_err.h"
23 #include <stdint.h>
24 #include <stdbool.h>
25
26 /**
27  * Initialise the GPIO expander.
28  * Writes DIR_REG and INVERT_REG with boot defaults from config.h.
29  *
30  * @return ESP_OK on success
31  */
32 esp_err_t fpga_gpio_init(void);
33
34 /**
35  * Drive the stepper coil outputs (bits[3:0] of OUTPUT_PORT).
36  *
37  * @param pattern Lower 4 bits written to OUTPUT_PORT (bit0=step_1 ... bit3=step_4)
38  * @return ESP_OK on success
39  */
40 esp_err_t fpga_gpio_set_steppers(uint8_t pattern);
41
42 /**
43  * Read button state from INPUT_PORT.
44  * After hardware invert (INVERT_REG=0x07): bit set = button pressed.
45  *
46  * @param[out] buttons Bits[2:0]: bit0=L, bit1=R, bit2=S
47  * @return ESP_OK on success
48  */
49 esp_err_t fpga_gpio_read_buttons(uint8_t *buttons);
50
51 #ifdef TEST_MODE_FPGA_GPIO
52 /**
53  * Test task: polls buttons every 50 ms, logs state changes,
54  * and mirrors button presses to stepper outputs as a simple pattern.
55  *
56  * Button_L pressed → step_1 only
57  * Button_R pressed → step_2 only
58  * Button_S pressed → all steppers on
59  * No button → all steppers off
60  *
61  * @param pvParameters Unused
62  */
63 void fpga_gpio_test_task(void *pvParameters);
64 #endif
```

esp/esp32_devkitv1/main/inc/peripherals/i2c_bus.h

```
1  /**
2   * i2c_bus.h - Reusable I2C bus manager for ESP-IDF v5.1 (legacy driver)
3   */
4
5  #pragma once
6
7  #include "driver/i2c.h"
8  #include "esp_err.h"
9
10 typedef struct {
11     i2c_port_t port;
12     gpio_num_t sda_gpio;
13     gpio_num_t scl_gpio;
14     uint32_t clk_speed_hz;          /**< 0 → default 100 kHz */
15     bool enable_internal_pullup;
16 } i2c_bus_config_t;
17
18 esp_err_t i2c_bus_init(const i2c_bus_config_t *cfg);
19 i2c_port_t i2c_bus_get_port(void);
20 esp_err_t i2c_bus_lock(uint32_t timeout_ms);
21 void i2c_bus_unlock(void);
22
23 esp_err_t i2c_bus_write(uint8_t dev_addr, const uint8_t *data,
24                         size_t len, uint32_t timeout_ms);
25
26 esp_err_t i2c_bus_read(uint8_t dev_addr, uint8_t *data,
27                        size_t len, uint32_t timeout_ms);
28
29 esp_err_t i2c_bus_write_read(uint8_t dev_addr,
30                              const uint8_t *write_data, size_t write_len,
31                              uint8_t *read_data, size_t read_len,
32                              uint32_t timeout_ms);
33
34 esp_err_t i2c_bus_deinit(void);
35 bool i2c_bus_is_init(void);
```


esp/esp32_devkitv1/main/inc/peripherals/motor.h

```
1 // main/motor.h
2 #pragma once
3
4 #include "esp_err.h"
5
6 #ifdef __cplusplus
7 extern "C" {
8 #endif
9
10 typedef struct {
11     int in1_gpio;
12     int in2_gpio;
13     int in3_gpio;
14     int in4_gpio;
15
16     // Coil order remap (fixes "squeak/vibrate no motion" without rewiring)
17     // Default {0,1,2,3}. Common fix: {1,0,3,2}.
18     int wire_map[4];
19
20     // Current phase (0..3), managed internally
21     int phase;
22 } motor_stepper_t;
23
24 esp_err_t motor_stepper_init(motor_stepper_t *m);
25
26 // Force a specific phase (0..3). Useful for known starting position.
27 void motor_stepper_set_phase(motor_stepper_t *m, int phase);
28
29 // De-energise all coils. Call before deep sleep to avoid wasting current.
30 void motor_stepper_release(motor_stepper_t *m);
31
32 // Low-level ramped move. direction = +1 (CW) or -1 (CCW).
33 void motor_stepper_move_ramped(motor_stepper_t *m,
34                                int steps,
35                                int direction,
36                                int start_delay_ms,
37                                int min_delay_ms,
38                                int ramp_steps);
39
40 // -----
41 // High-level helpers (wrap the ramped move with sane defaults)
42 // -----
43
44 // Rotate 'steps' in 'direction', hold for 'hold_ms', rotate back, then
45 // release the coils. Returns after the full sequence is complete.
46 void motor_stepper_swing_and_release(motor_stepper_t *m,
47                                      int steps,
48                                      int direction,
49                                      int hold_ms);
50
51 // Convenience: swing CW then back
52 void motor_stepper_swing_cw(motor_stepper_t *m, int steps, int hold_ms);
53
54 // Convenience: swing CCW then back
55 void motor_stepper_swing_ccw(motor_stepper_t *m, int steps, int hold_ms);
56
57 #ifdef __cplusplus
58 }
59 #endif
```

esp/esp32_devkitv1/main/inc/peripherals/pir.h

```
1 #pragma once
2
3 #include <stdint.h>
4 #include <stdbool.h>
5 #include "esp_err.h"
6
7 typedef enum {
8     PIR1 = 1,
9     PIR2 = 2,
10    PIR3 = 3
11 } pir_id_t;
12
13 esp_err_t pir_init(int pir1_gpio, int pir2_gpio, int pir3_gpio);
14
15 /**
16  * Returns 0 or 1 for the given PIR input.
17  */
18 bool pir_read(pir_id_t pir);
```

esp/esp32_devkitv1/main/inc/peripherals/sensors_temp_humidity.h

```
1 /**
2  * @file sensors_temp_humidity.h
3  * @brief Sensor subsystem - AHT20 + PIR + stepper motor
4  */
5
6 #pragma once
7
8 #include "esp_err.h"
9
10 esp_err_t sensors_init(void);
11 void sensors_task(void *pvParameters);
```

```

1  #ifndef UTILS_H
2  #define UTILS_H
3
4  #include <stdint.h>
5  #include <stddef.h>
6  #include <stdbool.h>
7  #include "esp_err.h"
8  #include "config.h"
9
10 /*****
11  * Checksum Utilities - Inlined for Performance
12  *****/
13
14 /**
15  * Calculate simple XOR checksum
16  * Inlined for hot paths (image verification)
17  */
18 ALWAYS_INLINE uint32_t checksum_calculate_xor(const uint8_t *data, size_t size)
19 {
20     if (UNLIKELY(!data || size == 0)) {
21         return 0;
22     }
23
24     uint32_t checksum = 0;
25     for (size_t i = 0; i < size; i++) {
26         checksum ^= data[i];
27     }
28     return checksum;
29 }
30
31 /**
32  * Verify XOR checksum
33  * Inlined for hot paths
34  */
35 ALWAYS_INLINE bool checksum_verify_xor_fast(const uint8_t *data, size_t size,
36                                             uint32_t expected)
37 {
38     return (checksum_calculate_xor(data, size) == expected);
39 }
40
41 /**
42  * Verify checksum with logging (not inlined - for debugging)
43  */
44 bool checksum_verify_xor(const uint8_t *data, size_t size, uint32_t expected);
45
46 /*****
47  * Memory Utilities
48  *****/
49
50 /**
51  * Allocate DMA-capable memory with error checking
52  */
53 void* dma_malloc(size_t size);
54
55 /**
56  * Safe free (handles NULL)
57  */
58 ALWAYS_INLINE void safe_free(void **ptr)
59 {
60     if (ptr && *ptr) {
61         free(*ptr);
62         *ptr = NULL;
63     }
64 }
65
66 /*****
67  * Validation Utilities - Inlined for Performance

```

```

68  *****/
69
70 /**
71  * Validate image header (fast inline version)
72  */
73 ALWAYS_INLINE bool validate_image_header_fast(const image_header_t *header)
74 {
75     if (UNLIKELY(!header)) {
76         return false;
77     }
78
79     // Fast path - common checks
80     if (header->magic != IMAGE_HEADER_MAGIC) {
81         return false;
82     }
83
84     if (header->size == 0 || header->size > IMAGE_MAX_SIZE_BYTES) {
85         return false;
86     }
87
88     return true;
89 }
90
91 /**
92  * Validate image header (with logging - not inlined)
93  */
94 bool validate_image_header(const image_header_t *header);
95
96 /**
97  * Check if size is within acceptable bounds
98  */
99 ALWAYS_INLINE bool validate_size_fast(size_t size, size_t max_size)
100 {
101     return (size > 0 && size <= max_size);
102 }
103
104 /**
105  * Validate size with logging
106  */
107 bool validate_size(size_t size, size_t max_size);
108
109 *****/
110 * Fast Math Utilities
111 *****/
112
113 /**
114  * Fast min/max (inlined)
115  */
116 ALWAYS_INLINE size_t min_size(size_t a, size_t b)
117 {
118     return (a < b) ? a : b;
119 }
120
121 ALWAYS_INLINE size_t max_size(size_t a, size_t b)
122 {
123     return (a > b) ? a : b;
124 }
125
126 /**
127  * Align size to 4-byte boundary (for DMA)
128  */
129 ALWAYS_INLINE size_t align_4(size_t size)
130 {
131     return (size + 3) & ~3;
132 }
133
134 #endif // UTILS_H

```

esp/esp32_devkitv1/main/src/interprocessor/cam_spi.c

```
1  /**
2   * cam_spi.c - Camera SPI slave interface
3   *
4   * Receives chunked image data from ESP32-CAM over SPI DMA.
5   * State machine: IDLE → header → data chunks → checksum → queue.
6   */
7
8  #include "cam_spi.h"
9  #include "driver/gpio.h"
10 #include "driver/spi_slave.h"
11 #include "esp_log.h"
12 #include "utils.h"
13 #include <inttypes.h>
14 #include <string.h>
15
16 static const char *TAG = "CAM_SPI";
17
18 /*****
19  * State
20  *****/
21
22 typedef enum { RX_IDLE, RX_DATA, RX_COMPLETE } rx_state_t;
23
24 typedef struct {
25     rx_state_t state;
26     image_header_t header;
27     uint8_t *buffer;
28     size_t bytes_received;
29     QueueHandle_t queue;
30 } cam_spi_ctx_t;
31
32 static cam_spi_ctx_t ctx;
33
34 static DMA_ALIGNED uint8_t rx_buf[SPI_BUFFER_SIZE];
35 static DMA_ALIGNED uint8_t tx_buf[SPI_BUFFER_SIZE];
36
37 /*****
38  * Receive State Machine
39  *****/
40
41 static void rx_reset(void)
42 {
43     safe_free((void **)&ctx.buffer);
44     ctx.bytes_received = 0;
45     ctx.state = RX_IDLE;
46     memset(&ctx.header, 0, sizeof(ctx.header));
47 }
48
49 static bool rx_process_header(const uint8_t *data, size_t len)
50 {
51     if (len < sizeof(image_header_t)) return false;
52
53     memcpy(&ctx.header, data, sizeof(image_header_t));
54
55     if (!validate_image_header_fast(&ctx.header)) return false;
56
57     ctx.buffer = dma_malloc(ctx.header.size);
58     if (!ctx.buffer) {
59         ESP_LOGE(TAG, "alloc failed: %" PRIu32 " bytes",
60                  ctx.header.size);
61         return false;
62     }
63
64     ctx.state = RX_DATA;
65     ctx.bytes_received = 0;
66
67     /* Copy trailing data that arrived with the header */
```

```

68     size_t extra = len - sizeof(image_header_t);
69     if (extra > 0) {
70         size_t n = min_size(extra, ctx.header.size);
71         memcpy(ctx.buffer, data + sizeof(image_header_t), n);
72         ctx.bytes_received = n;
73     }
74
75     ESP_LOGI(TAG, "header: %" PRIu32 " bytes incoming", ctx.header.size);
76     return true;
77 }
78
79 static void rx_process_data(const uint8_t *data, size_t len)
80 {
81     size_t remaining = ctx.header.size - ctx.bytes_received;
82     size_t n = min_size(len, remaining);
83
84     memcpy(ctx.buffer + ctx.bytes_received, data, n);
85     ctx.bytes_received += n;
86
87     if (ctx.bytes_received >= ctx.header.size) { ctx.state = RX_COMPLETE; }
88 }
89
90 static void rx_finalize(void)
91 {
92     ESP_LOGI(TAG, "received: %zu bytes", ctx.bytes_received);
93
94     if (!checksum_verify_xor_fast(ctx.buffer, ctx.header.size,
95                                   ctx.header.checksum)) {
96         ESP_LOGE(TAG, "checksum failed - dropping");
97         rx_reset();
98         return;
99     }
100
101     image_data_t img = {
102         .buffer = ctx.buffer,
103         .size = ctx.header.size,
104         .header = ctx.header,
105     };
106
107     if (xQueueSend(ctx.queue, &img, 0) == pdTRUE) {
108         ctx.buffer = NULL; /* queue owns it now */
109     } else {
110         ESP_LOGW(TAG, "queue full - dropping");
111         safe_free((void **)&ctx.buffer);
112     }
113
114     ctx.state = RX_IDLE;
115 }
116
117 static void rx_handle_chunk(const uint8_t *data, size_t len)
118 {
119     switch (ctx.state) {
120     case RX_IDLE: rx_process_header(data, len); break;
121     case RX_DATA:
122         rx_process_data(data, len);
123         if (ctx.state == RX_COMPLETE) rx_finalize();
124         break;
125     default: rx_reset(); break;
126     }
127 }
128
129 /*****
130  * SPI DMA
131  *****/
132
133 static esp_err_t spi_receive_chunk(size_t *out_len, uint32_t timeout_ticks)
134 {
135     spi_slave_transaction_t trans = {
136         .length = SPI_BUFFER_SIZE * 8,
137         .rx_buffer = rx_buf,

```

```

138         .tx_buffer = tx_buf,
139     };
140
141     esp_err_t ret =
142         spi_slave_transmit(CAM_SPI_HOST, &trans, timeout_ticks);
143
144     if (ret == ESP_OK) { *out_len = trans.trans_len / 8; }
145
146     return ret;
147 }
148
149 void cam_spi_send_cmd(uint32_t cmd, uint32_t epoch) {}
150 /*****
151  * Public Interface
152  *****/
153
154 esp_err_t cam_spi_init(void)
155 {
156     ESP_LOGI(TAG, "Initializing camera SPI slave");
157
158     ctx.queue = xQueueCreate(IMAGE_QUEUE_LENGTH, sizeof(image_data_t));
159     if (!ctx.queue) return ESP_FAIL;
160
161     spi_bus_config_t bus = {
162         .mosi_io_num = CAM_PIN_MOSI,
163         .miso_io_num = CAM_PIN_MISO,
164         .sclk_io_num = CAM_PIN_SCLK,
165         .quadwp_io_num = -1,
166         .quadhd_io_num = -1,
167         .max_transfer_sz = SPI_BUFFER_SIZE,
168     };
169
170     spi_slave_interface_config_t slave = {
171         .mode = CAM_SPI_MODE,
172         .spics_io_num = CAM_PIN_CS,
173         .queue_size = CAM_SPI_QUEUE_SIZE,
174         .flags = 0,
175     };
176
177     esp_err_t ret =
178         spi_slave_initialize(CAM_SPI_HOST, &bus, &slave, CAM_SPI_DMA_CHAN);
179     if (ret != ESP_OK) {
180         vQueueDelete(ctx.queue);
181         ctx.queue = NULL;
182         return ret;
183     }
184
185     gpio_set_pull_mode(CAM_PIN_CS, GPIO_PULLUP_ONLY);
186
187     ESP_LOGI(TAG, "ready - MOSI=%d MISO=%d CLK=%d CS=%d", CAM_PIN_MOSI,
188             CAM_PIN_MISO, CAM_PIN_SCLK, CAM_PIN_CS);
189     return ESP_OK;
190 }
191
192 void cam_spi_deinit(void)
193 {
194     rx_reset();
195     spi_slave_free(CAM_SPI_HOST);
196     if (ctx.queue) {
197         vQueueDelete(ctx.queue);
198         ctx.queue = NULL;
199     }
200 }
201
202 QueueHandle_t cam_spi_get_queue(void) { return ctx.queue; }
203
204 /*****
205  * Receive Task
206  *****/
207

```



```

208 void cam_spi_receive_task(void *pvParameters)
209 {
210     ESP_LOGI(TAG, "receive task started");
211
212     for (;;) {
213         size_t chunk_len = 0;
214         esp_err_t ret =
215             spi_receive_chunk(&chunk_len, CAM_TRANSFER_TIMEOUT_TICKS);
216
217         if (ret == ESP_ERR_TIMEOUT) {
218             if (ctx.state != RX_IDLE) {
219                 ESP_LOGW(TAG, "timeout - resetting");
220                 rx_reset();
221             }
222             continue;
223         }
224
225         if (ret != ESP_OK) continue;
226
227         rx_handle_chunk(rx_buf, chunk_len);
228     }
229 }

```

esp/esp32_devkitv1/main/src/interprocessor/fpga_spi.c

```
1 /**
2  * fpga_spi.c - FPGA SPI master interface (Tang Nano 9K)
3  *
4  * Async queue/get pattern: feeds DMA pipeline continuously,
5  * no idle gaps between chunks.
6  */
7
8 #include "fpga_spi.h"
9 #include "driver/gpio.h"
10 #include "driver/spi_master.h"
11 #include "esp_log.h"
12 #include "freertos/FreeRTOS.h"
13 #include "freertos/semphr.h"
14 #include "freertos/task.h"
15 #include "isr_signals.h"
16 #include "utils.h"
17 #include <string.h>
18
19 static const char *TAG = "FPGA_SPI";
20
21 /*****
22  * State
23  *****/
24
25 static struct {
26     spi_device_handle_t device;
27     SemaphoreHandle_t mutex;
28     bool initialized;
29 } ctx;
30
31 static spi_transaction_t trans_pool[FPGA_SPI_QUEUE_SIZE];
32
33 /*****
34  * Test Patterns
35  *****/
36
37 #ifdef TEST_MODE_FPGA_PATTERNS
38
39 typedef enum {
40     PAT_WHITE,
41     PAT_BLACK,
42     PAT_GRADIENT,
43     PAT_CHECKER,
44     PAT_COUNT
45 } test_pattern_t;
46
47 static void generate_pattern(uint8_t *buf, size_t size, test_pattern_t pat)
48 {
49     static const int W = 480; /* LCD active width */
50
51     switch (pat) {
52     case PAT_WHITE: memset(buf, 0xFF, size); break;
53     case PAT_BLACK: memset(buf, 0x00, size); break;
54     case PAT_GRADIENT:
55         /* Horizontal gradient: left=black,
56          * right=white,
57          * same every row */
58         for (size_t i = 0; i < size; i++)
59             buf[i] = (uint8_t)((int)(i % W) * 255 / (W - 1));
60         break;
61     case PAT_CHECKER:
62         /* 32x32 pixel checkerboard blocks */
63         for (size_t i = 0; i < size; i++) {
64             int col = i % W;
65             int row = i / W;
66             buf[i] = (((col / 32) + (row / 32)) & 1) ? 0xFF : 0x00;
67         }
68     }
```

```

68         break;
69     default: memset(buf, 0xFF, size); break;
70 }
71 }
72
73 static esp_err_t setup_test_button(void)
74 {
75     gpio_config_t cfg = {
76         .pin_bit_mask = (1ULL << TEST_BUTTON_PIN),
77         .mode = GPIO_MODE_INPUT,
78         .pull_up_en = GPIO_PULLUP_ENABLE,
79         .pull_down_en = GPIO_PULLDOWN_DISABLE,
80         .intr_type = GPIO_INTR_NEGEDGE,
81     };
82
83     esp_err_t ret = gpio_config(&cfg);
84     if (ret != ESP_OK) return ret;
85
86     ret = gpio_install_isr_service(0);
87     if (ret != ESP_OK && ret != ESP_ERR_INVALID_STATE) return ret;
88
89     return gpio_isr_handler_add(TEST_BUTTON_PIN, ISR_OnButtonPress, NULL);
90 }
91
92 #endif
93
94 /*****
95  * Transmit - async queue/get keeps DMA fed
96  *****/
97
98 esp_err_t fpga_spi_transmit(const uint8_t *data, size_t size)
99 {
100     if (!ctx.initialized) return ESP_ERR_INVALID_STATE;
101
102     if (!data || size == 0) return ESP_ERR_INVALID_ARG;
103
104     if (xSemaphoreTake(ctx.mutex, portMAX_DELAY) != pdTRUE)
105         return ESP_ERR_TIMEOUT;
106
107     size_t offset = 0;
108     size_t in_flight = 0;
109     size_t pool_idx = 0;
110     esp_err_t ret = ESP_OK;
111
112     while (offset < size || in_flight > 0) {
113
114         /* Fill queue slots */
115         while (offset < size && in_flight < FPGA_SPI_QUEUE_SIZE) {
116
117             size_t chunk =
118                 min_size(size - offset, FPGA_MAX_TRANSFER_SIZE);
119
120             spi_transaction_t *t =
121                 &trans_pool[pool_idx % FPGA_SPI_QUEUE_SIZE];
122             t->length = chunk * 8;
123             t->rxtlength =
124                 0; /* reset: driver may set this on previous use */
125             t->tx_buffer = data + offset;
126             t->rx_buffer = NULL;
127             t->flags = 0;
128
129             ret = spi_device_queue_trans(ctx.device, t,
130                                         portMAX_DELAY);
131             if (ret != ESP_OK) goto drain;
132
133             offset += chunk;
134             in_flight++;
135             pool_idx++;
136         }
137     }

```

```

138         /* Collect one completed transfer */
139         spi_transaction_t *result;
140         ret = spi_device_get_trans_result(ctx.device, &result,
141                                         portMAX_DELAY);
142         if (ret != ESP_OK) goto drain;
143         in_flight--;
144     }
145     goto done;
146
147 drain:
148     /* Drain any in-flight transactions
149        leave the SPI queue clean */
150
151     while (in_flight > 0) {
152         spi_transaction_t *result;
153         spi_device_get_trans_result(ctx.device, &result,
154                                     portMAX_DELAY);
155         in_flight--;
156     }
157
158 done:
159     xSemaphoreGive(ctx.mutex);
160     return ret;
161 }
162
163 /*****
164  * Receive logic
165  *****/
166
167 esp_err_t fpga_spi_receive(uint8_t *data, size_t size)
168 {
169     if (!ctx.initialized) return ESP_ERR_INVALID_STATE;
170     if (!data || size == 0) return ESP_ERR_INVALID_ARG;
171     if (xSemaphoreTake(ctx.mutex, portMAX_DELAY) != pdTRUE)
172         return ESP_ERR_TIMEOUT;
173
174     spi_transaction_t *t = &trans_pool[0];
175
176     t->length = size * 8;
177     t->rxlength = 0;
178     t->tx_buffer = NULL;
179     t->rx_buffer = data;
180     t->flags = 0;
181
182     esp_err_t ret = ESP_OK;
183
184     ret = spi_device_queue_trans(ctx.device, t, portMAX_DELAY);
185
186     /* Collect one completed transfer */
187     spi_transaction_t *result;
188     if (ret == ESP_OK)
189         ret = spi_device_get_trans_result(ctx.device,
190                                         &result, portMAX_DELAY);
191
192     xSemaphoreGive(ctx.mutex);
193     return ret;
194 }
195
196
197 /*****
198  * Init / Deinit
199  *****/
200
201 esp_err_t fpga_spi_init(void)
202 {
203     if (ctx.initialized) return ESP_OK;
204
205     ctx.mutex = xSemaphoreCreateMutex();
206     if (!ctx.mutex) return ESP_ERR_NO_MEM;
207

```

```

208     spi_bus_config_t bus = {
209         .mosi_io_num = FPGA_PIN_MOSI,
210         .miso_io_num = FPGA_PIN_MISO,
211         .sclk_io_num = FPGA_PIN_SCLK,
212         .quadwp_io_num = -1,
213         .quadhd_io_num = -1,
214         .max_transfer_sz = FPGA_MAX_TRANSFER_SIZE,
215         .flags = SPICOMMON_BUSFLAG_MASTER,
216     };
217
218     esp_err_t ret =
219         spi_bus_initialize(FPGA_SPI_HOST, &bus, FPGA_SPI_DMA_CHAN);
220     if (ret != ESP_OK) return ret;
221
222     spi_device_interface_config_t dev = {
223         .clock_speed_hz = FPGA_SPI_CLOCK_HZ,
224         .mode = FPGA_SPI_MODE,
225         .spics_io_num = FPGA_PIN_CS,
226         .queue_size = FPGA_SPI_QUEUE_SIZE,
227         .flags = 0,
228     };
229
230     ret = spi_bus_add_device(FPGA_SPI_HOST, &dev, &ctx.device);
231     if (ret != ESP_OK) {
232         spi_bus_free(FPGA_SPI_HOST);
233         return ret;
234     }
235
236     ctx.initialized = true;
237     ESP_LOGI(TAG, "ready - %.1f MHz, queue depth %d",
238         (double)FPGA_SPI_CLOCK_MHZ, FPGA_SPI_QUEUE_SIZE);
239
240 #ifndef TEST_MODE_FPGA_PATTERNS
241     setup_test_button();
242 #endif
243
244     return ESP_OK;
245 }
246
247 void fpga_spi_deinit(void)
248 {
249     if (!ctx.initialized) return;
250
251 #ifndef TEST_MODE_FPGA_PATTERNS
252     gpio_isr_handler_remove(TEST_BUTTON_PIN);
253 #endif
254
255     spi_bus_remove_device(ctx.device);
256     spi_bus_free(FPGA_SPI_HOST);
257     vSemaphoreDelete(ctx.mutex);
258     ctx.device = NULL;
259     ctx.mutex = NULL;
260     ctx.initialized = false;
261 }
262
263 /*****
264  * Test Task
265  *****/
266
267 #ifndef TEST_MODE_FPGA_PATTERNS
268
269 void fpga_test_task(void *pvParameters)
270 {
271     if (!g_button_sem) {
272         vTaskDelete(NULL);
273         return;
274     }
275
276     uint8_t *buf = dma_malloc(TEST_IMAGE_SIZE);
277     if (!buf) {

```

```
278         vTaskDelete(NULL);
279         return;
280     }
281
282     test_pattern_t pat = PAT_WHITE;
283
284     for (;;) {
285         if (xSemaphoreTake(g_button_sem, portMAX_DELAY) == pdTRUE) {
286             generate_pattern(buf, TEST_IMAGE_SIZE, pat);
287             fpga_spi_transmit(buf, TEST_IMAGE_SIZE);
288             pat = (pat + 1) % PAT_COUNT;
289         }
290     }
291 }
292
293 #endif
```

esp/esp32_devkitv1/main/src/interprocessor/i2c_bus.c

```
1  /**
2   * i2c_bus.c - Shared I2C bus manager
3   *
4   * Mutex-protected bus access. All I2C peripherals go through here.
5   *
6   * Includes:
7   * - init/deinit
8   * - write/read/write_read
9   * - probe (address ACK check)
10  * - scan (log all ACKing addresses)
11  *
12  * NOTE: Uses legacy ESP-IDF I2C driver (driver/i2c.h). Works fine on IDF v5.2.
13  */
14
15 #include "peripherals/i2c_bus.h"
16
17 #include <string.h>
18
19 #include "freertos/FreeRTOS.h"
20 #include "freertos/semphr.h"
21
22 #include "esp_err.h"
23 #include "esp_log.h"
24 #include "esp_rom_sys.h" /* esp_rom_delay_us() - tick-independent µs delay */
25
26 #include "driver/i2c.h"
27
28 static const char *TAG = "I2C";
29
30 #define DEFAULT_CLK 100000
31
32 static struct {
33     i2c_port_t port;
34     i2c_config_t conf; /* stored for bus recovery after timeout */
35     SemaphoreHandle_t mutex;
36     bool initialized;
37 } bus;
38
39 /* Recover a stuck I2C bus.
40  * Toggles SCL up to 9 times (advances any slave state machine), then
41  * generates a STOP condition so the bus is clean before reinstalling
42  * the driver. Must be called with bus.mutex already held. */
43 static void bus_recover(void)
44 {
45     ESP_LOGW(TAG, "I2C timeout - recovering bus (port %d)", bus.port);
46     i2c_driver_delete(bus.port);
47
48     /* After i2c_driver_delete the GPIO matrix still routes the I2C
49     * peripheral output to the pins (possibly frozen low).
50     * gpio_reset_pin() reconnects the software GPIO register to the
51     * physical pin before we pulse SCL. */
52     gpio_reset_pin(bus.conf.scl_io_num);
53     gpio_reset_pin(bus.conf.sda_io_num);
54
55     /* Clock recovery: release SCL, then pulse until SDA goes high.
56     * esp_rom_delay_us gives real µs delays independent of tick rate. */
57     gpio_set_direction(bus.conf.scl_io_num, GPIO_MODE_OUTPUT_OD);
58     gpio_set_direction(bus.conf.sda_io_num, GPIO_MODE_INPUT_OUTPUT_OD);
59     gpio_set_level(bus.conf.scl_io_num, 1);
60     esp_rom_delay_us(5);
61
62     ESP_LOGW(TAG, "SDA before recovery: %d",
63              gpio_get_level(bus.conf.sda_io_num));
64
65     int pulses = 0;
66     for (; pulses < 9 && !gpio_get_level(bus.conf.sda_io_num); pulses++) {
67         gpio_set_level(bus.conf.scl_io_num, 0);
```

```

68         esp_rom_delay_us(5);
69         gpio_set_level(bus.conf.scl_io_num, 1);
70         esp_rom_delay_us(5);
71     }
72     ESP_LOGW(TAG, "SCL pulses: %d, SDA after: %d", pulses,
73         gpio_get_level(bus.conf.sda_io_num));
74
75     /* STOP: SDA low → SCL high → SDA high */
76     gpio_set_level(bus.conf.sda_io_num, 0);
77     esp_rom_delay_us(5);
78     gpio_set_level(bus.conf.scl_io_num, 1);
79     esp_rom_delay_us(5);
80     gpio_set_level(bus.conf.sda_io_num, 1);
81     esp_rom_delay_us(5);
82
83     i2c_param_config(bus.port, &bus.conf);
84     esp_err_t err = i2c_driver_install(bus.port, I2C_MODE_MASTER, 0, 0, 0);
85     if (err != ESP_OK)
86         ESP_LOGE(TAG, "bus recovery failed: %s", esp_err_to_name(err));
87     else ESP_LOGI(TAG, "bus recovered");
88 }
89
90 /* =====
91  * Public: init / state
92  * ===== */
93
94 esp_err_t i2c_bus_init(const i2c_bus_config_t *cfg)
95 {
96     if (!cfg) return ESP_ERR_INVALID_ARG;
97     if (bus.initialized) return ESP_ERR_INVALID_STATE;
98
99     i2c_config_t conf = {
100         .mode = I2C_MODE_MASTER,
101         .sda_io_num = cfg->sda_gpio,
102         .scl_io_num = cfg->scl_gpio,
103         .sda_pullup_en = cfg->enable_internal_pullup ? GPIO_PULLUP_ENABLE
104             : GPIO_PULLUP_DISABLE,
105         .scl_pullup_en = cfg->enable_internal_pullup ? GPIO_PULLUP_ENABLE
106             : GPIO_PULLUP_DISABLE,
107         .master.clk_speed =
108             cfg->clk_speed_hz ? cfg->clk_speed_hz : DEFAULT_CLK,
109     };
110
111     esp_err_t err;
112
113     err = i2c_param_config(cfg->port, &conf);
114     if (err != ESP_OK) return err;
115
116     err = i2c_driver_install(cfg->port, I2C_MODE_MASTER, 0, 0, 0);
117     if (err != ESP_OK) return err;
118
119     bus.mutex = xSemaphoreCreateMutex();
120     if (!bus.mutex) {
121         i2c_driver_delete(cfg->port);
122         return ESP_ERR_NO_MEM;
123     }
124
125     bus.port = cfg->port;
126     bus.conf = conf;
127     bus.initialized = true;
128
129     ESP_LOGI(TAG, "ready - port %d, SDA=%d SCL=%d, %lu Hz", cfg->port,
130         cfg->sda_gpio, cfg->scl_gpio,
131         cfg->clk_speed_hz ? cfg->clk_speed_hz
132             : (uint32_t)DEFAULT_CLK);
133
134     return ESP_OK;
135 }
136
137 i2c_port_t i2c_bus_get_port(void) { return bus.port; }

```



```

138 bool i2c_bus_is_init(void) { return bus.initialized; }
139
140 /* =====
141  * Public: lock / unlock
142  * ===== */
143
144 esp_err_t i2c_bus_lock(uint32_t timeout_ms)
145 {
146     if (!bus.initialized) return ESP_ERR_INVALID_STATE;
147     return (xSemaphoreTake(bus.mutex, pdMS_TO_TICKS(timeout_ms)) == pdTRUE)
148         ? ESP_OK
149         : ESP_ERR_TIMEOUT;
150 }
151
152 void i2c_bus_unlock(void)
153 {
154     if (bus.mutex) xSemaphoreGive(bus.mutex);
155 }
156
157 /* =====
158  * Public: probe / scan
159  * ===== */
160
161 esp_err_t i2c_bus_probe(uint8_t addr, uint32_t timeout_ms)
162 {
163     if (!bus.initialized) return ESP_ERR_INVALID_STATE;
164
165     esp_err_t err = i2c_bus_lock(timeout_ms);
166     if (err != ESP_OK) return err;
167
168     i2c_cmd_handle_t cmd = i2c_cmd_link_create();
169     if (!cmd) {
170         i2c_bus_unlock();
171         return ESP_ERR_NO_MEM;
172     }
173
174     i2c_master_start(cmd);
175     i2c_master_write_byte(cmd, (addr << 1) | I2C_MASTER_WRITE, true);
176     i2c_master_stop(cmd);
177
178     err = i2c_master_cmd_begin(bus.port, cmd, pdMS_TO_TICKS(timeout_ms));
179     i2c_cmd_link_delete(cmd);
180
181     i2c_bus_unlock();
182     return err;
183 }
184
185 void i2c_bus_scan(uint32_t timeout_ms)
186 {
187     if (!bus.initialized) {
188         ESP_LOGW(TAG, "scan requested but bus not initialized");
189         return;
190     }
191
192     ESP_LOGI(TAG, "scan start (port %d)...", (int)bus.port);
193
194     int found = 0;
195     for (uint8_t addr = 0x08; addr < 0x78; addr++) {
196         esp_err_t err = i2c_bus_probe(addr, timeout_ms);
197         if (err == ESP_OK) {
198             ESP_LOGI(TAG, "ACK at 0x%02X", addr);
199             found++;
200         }
201     }
202
203     if (!found) ESP_LOGW(TAG, "no I2C devices ACKed");
204     ESP_LOGI(TAG, "scan done");
205 }
206
207 /* =====

```

```

208 * Public: write / read / write_read
209 * ===== */
210
211 esp_err_t i2c_bus_write(uint8_t addr, const uint8_t *data, size_t len,
212                          uint32_t timeout_ms)
213 {
214     if (!bus.initialized) return ESP_ERR_INVALID_STATE;
215     if (len > 0 && !data) return ESP_ERR_INVALID_ARG;
216
217     esp_err_t err = i2c_bus_lock(timeout_ms);
218     if (err != ESP_OK) return err;
219
220     i2c_cmd_handle_t cmd = i2c_cmd_link_create();
221     if (!cmd) {
222         i2c_bus_unlock();
223         return ESP_ERR_NO_MEM;
224     }
225
226     i2c_master_start(cmd);
227     i2c_master_write_byte(cmd, (addr << 1) | I2C_MASTER_WRITE, true);
228     if (len > 0) i2c_master_write(cmd, data, len, true);
229     i2c_master_stop(cmd);
230
231     err = i2c_master_cmd_begin(bus.port, cmd, pdMS_TO_TICKS(timeout_ms));
232     i2c_cmd_link_delete(cmd);
233
234     if (err == ESP_ERR_TIMEOUT) bus_recover();
235
236     i2c_bus_unlock();
237     return err;
238 }
239
240 esp_err_t i2c_bus_read(uint8_t addr, uint8_t *data, size_t len,
241                        uint32_t timeout_ms)
242 {
243     if (!bus.initialized) return ESP_ERR_INVALID_STATE;
244     if (!data || len == 0) return ESP_ERR_INVALID_ARG;
245
246     esp_err_t err = i2c_bus_lock(timeout_ms);
247     if (err != ESP_OK) return err;
248
249     i2c_cmd_handle_t cmd = i2c_cmd_link_create();
250     if (!cmd) {
251         i2c_bus_unlock();
252         return ESP_ERR_NO_MEM;
253     }
254
255     i2c_master_start(cmd);
256     i2c_master_write_byte(cmd, (addr << 1) | I2C_MASTER_READ, true);
257
258     if (len > 1) i2c_master_read(cmd, data, len - 1, I2C_MASTER_ACK);
259     i2c_master_read_byte(cmd, &data[len - 1], I2C_MASTER_NACK);
260
261     i2c_master_stop(cmd);
262
263     err = i2c_master_cmd_begin(bus.port, cmd, pdMS_TO_TICKS(timeout_ms));
264     i2c_cmd_link_delete(cmd);
265
266     if (err == ESP_ERR_TIMEOUT) bus_recover();
267
268     i2c_bus_unlock();
269     return err;
270 }
271
272 esp_err_t i2c_bus_write_read(uint8_t addr, const uint8_t *wr, size_t wr_len,
273                              uint8_t *rd, size_t rd_len, uint32_t timeout_ms)
274 {
275     if (!bus.initialized) return ESP_ERR_INVALID_STATE;
276     if (!wr || wr_len == 0) return ESP_ERR_INVALID_ARG;
277     if (!rd || rd_len == 0) return ESP_ERR_INVALID_ARG;

```

```

278     esp_err_t err = i2c_bus_lock(timeout_ms);
279     if (err != ESP_OK) return err;
280
281     i2c_cmd_handle_t cmd = i2c_cmd_link_create();
282     if (!cmd) {
283         i2c_bus_unlock();
284         return ESP_ERR_NO_MEM;
285     }
286
287     i2c_master_start(cmd);
288     i2c_master_write_byte(cmd, (addr << 1) | I2C_MASTER_WRITE, true);
289     i2c_master_write(cmd, wr, wr_len, true);
290
291     i2c_master_start(cmd);
292     i2c_master_write_byte(cmd, (addr << 1) | I2C_MASTER_READ, true);
293
294     if (rd_len > 1) i2c_master_read(cmd, rd, rd_len - 1, I2C_MASTER_ACK);
295     i2c_master_read_byte(cmd, &rd[rd_len - 1], I2C_MASTER_NACK);
296
297     i2c_master_stop(cmd);
298
299     err = i2c_master_cmd_begin(bus.port, cmd, pdMS_TO_TICKS(timeout_ms));
300     i2c_cmd_link_delete(cmd);
301
302     if (err == ESP_ERR_TIMEOUT) bus_recover();
303
304     i2c_bus_unlock();
305     return err;
306 }
307
308 /* =====
309  * Public: deinit
310  * ===== */
311
312 esp_err_t i2c_bus_deinit(void)
313 {
314     if (!bus.initialized) return ESP_OK;
315
316     esp_err_t err = i2c_driver_delete(bus.port);
317     if (err != ESP_OK) return err;
318
319     if (bus.mutex) {
320         vSemaphoreDelete(bus.mutex);
321         bus.mutex = NULL;
322     }
323
324     bus.initialized = false;
325     return ESP_OK;
326 }
327

```

```

1  /**
2   * lora.c -- DevKitV1 LoRa transmitter
3   *
4   * Sends compressed diff packets to S3 receiver via RYLR998.
5   */
6  #include "config.h"
7
8  #include "driver/uart.h"
9  #include "esp_log.h"
10 #include "esp_timer.h"
11 #include "freertos/FreeRTOS.h"
12 #include "freertos/task.h"
13 #include <stdio.h>
14 #include <stdlib.h>
15 #include <string.h>
16
17 #include "lora.h"
18
19 static const char *TAG = "LORA";
20
21 static const uint16_t LORA_BUF_SIZE = (1 << 9);
22
23 /* ----- */
24 static void uart_init_lora(void)
25 {
26     uart_config_t cfg = {
27         .baud_rate = LORA_BAUD_RATE,
28         .data_bits = UART_DATA_8_BITS,
29         .parity = UART_PARITY_DISABLE,
30         .stop_bits = UART_STOP_BITS_1,
31         .flow_ctrl = UART_HW_FLOWCTRL_DISABLE,
32         .source_clk = UART_SCLK_DEFAULT,
33     };
34     uart_driver_install(LORA_UART_NUM, LORA_BUF_SIZE, LORA_BUF_SIZE, 0,
35         NULL, 0);
36     uart_param_config(LORA_UART_NUM, &cfg);
37     uart_set_pin(LORA_UART_NUM, LORA_PIN_TX, LORA_PIN_RX, -1, -1);
38     vTaskDelay(pdMS_TO_TICKS(LORA_INIT_DELAY_MS));
39 }
40
41 static bool readline(char *buf, int lenmax, int timeout_ms)
42 {
43     int pos = 0;
44     int64_t deadline = esp_timer_get_time() + (int64_t)timeout_ms * 1000;
45     while (esp_timer_get_time() < deadline) {
46         uint8_t b;
47         if (uart_read_bytes(LORA_UART_NUM, &b, 1, pdMS_TO_TICKS(10)) <=
48             0)
49             continue;
50         if (b == '\n') {
51             if (pos > 0 && buf[pos - 1] == '\r') pos--;
52             buf[pos] = '\0';
53             return pos > 0;
54         }
55         if (pos < lenmax - 1) buf[pos++] = (char)b;
56     }
57     buf[pos] = '\0';
58     return false;
59 }
60
61 static bool at_cmd(const char *cmd, const char *expect, int timeout_ms)
62 {
63     static const int POLL_MS = 50;
64     uart_flush_input(LORA_UART_NUM);
65     uart_write_bytes(LORA_UART_NUM, cmd, strlen(cmd));
66     char line[INT8_MAX + 1];
67     int elapsed = 0;

```

```

68     while (elapsed < timeout_ms) {
69         if (readline(line, sizeof(line), POLL_MS)) {
70             ESP_LOGI(TAG, " %s", line);
71             if (expect && strstr(line, expect)) return true;
72         }
73         elapsed += POLL_MS;
74     }
75     return false;
76 }
77
78 /* ----- */
79 static void module_init(void)
80 {
81     at_cmd("AT+RESET\r\n", "+RESET", 1000);
82     vTaskDelay(pdMS_TO_TICKS(1500));
83
84     char cmd[64];
85     snprintf(cmd, sizeof(cmd), "AT+ADDRESS=%d\r\n", LORA_ADDRESS_SENDER);
86     at_cmd(cmd, "+OK", LORA_AT_TIMEOUT_MS);
87
88     snprintf(cmd, sizeof(cmd), "AT+NETWORKID=%d\r\n", LORA_NETWORK_ID);
89     at_cmd(cmd, "+OK", LORA_AT_TIMEOUT_MS);
90
91     snprintf(cmd, sizeof(cmd), "AT+PARAMETER=%d,%d,%d,%d\r\n", LORA_SF,
92         LORA_BW, LORA_CR, LORA_PREAMBLE);
93     at_cmd(cmd, "+OK", LORA_AT_TIMEOUT_MS);
94
95     at_cmd("AT+CRFOP=7\r\n", "+OK", LORA_AT_TIMEOUT_MS);
96
97     at_cmd("AT+ADDRESS?\r\n", "+ADDRESS=", LORA_AT_TIMEOUT_MS);
98     at_cmd("AT+NETWORKID?\r\n", "+NETWORKID=", LORA_AT_TIMEOUT_MS);
99     at_cmd("AT+PARAMETER?\r\n", "+PARAMETER=", LORA_AT_TIMEOUT_MS);
100    at_cmd("AT+BAND?\r\n", "+BAND=", LORA_AT_TIMEOUT_MS);
101    at_cmd("AT+CRFOP?\r\n", "+CRFOP=", LORA_AT_TIMEOUT_MS);
102 }
103
104 /* ----- */
105 __attribute__((unused))
106 static bool parse_rcv(char *line, int *src, char **data, int *len)
107 {
108     if (strncmp(line, "+RCV=", 5) != 0) return false;
109
110     char *p = line + 5;
111
112     char *c1 = strchr(p, ',');
113     if (!c1) return false;
114     *c1 = '\0';
115     *src = atoi(p);
116
117     char *p_len = c1 + 1;
118     char *c2 = strchr(p_len, ',');
119     if (!c2) return false;
120     *c2 = '\0';
121
122     *len = atoi(p_len);
123     if (*len <= 0 || *len > LORA_MAX_PAYLOAD) return false;
124
125     *data = c2 + 1;
126
127     char *end = *data + *len;
128     if (*end == ',') {
129         *end = '\0';
130     } else if (*end == '\0') {
131         /* firmware omitted rssi/snr */
132     } else {
133         return false;
134     }
135
136     return true;
137 }

```

```

138
139 /* ----- */
140 esp_err_t lora_init(void)
141 {
142     uart_init_lora();
143     vTaskDelay(pdMS_TO_TICKS(500));
144     module_init();
145     vTaskDelay(pdMS_TO_TICKS(500));
146
147     ESP_LOGI(TAG, "LoRa transmitter ready addr=%d dest=%d net=%d SF=%d BW=%d",
148             LORA_ADDRESS_SENDER, LORA_DEST_ADDR, LORA_NETWORK_ID,
149             LORA_SF, LORA_BW);
150     return ESP_OK;
151 }
152
153 esp_err_t lora_send_packet(const uint8_t *data, size_t len)
154 {
155     if (!data || len == 0 || len > LORA_MAX_PAYLOAD)
156         return ESP_ERR_INVALID_ARG;
157
158     char hdr[32];
159     int hdr_len = snprintf(hdr, sizeof(hdr), "AT+SEND=%d,%d,",
160                             LORA_DEST_ADDR, (int)len);
161
162     uart_write_bytes(LORA_UART_NUM, hdr, hdr_len);
163     uart_write_bytes(LORA_UART_NUM, (const char *)data, len);
164     uart_write_bytes(LORA_UART_NUM, "\r\n", 2);
165
166     return ESP_OK;
167 }
168
169 /* ----- */
170 #ifdef TEST_MODE_LORA_BENCH
171
172 static void send_msg(uint32_t seq, int size)
173 {
174     static const int IDEAL_PAYLOAD = 0xF0;
175     static const int MAX_HDR = 0x20;
176     static const int num_hold = 9;
177     uint8_t payload[IDEAL_PAYLOAD + 1];
178     int mask = -(size > IDEAL_PAYLOAD);
179     size = size ^ (mask & (IDEAL_PAYLOAD ^ size));
180     memset(payload, 'A', size);
181     char seq_str[num_hold];
182     snprintf(seq_str, num_hold, "%08lx", (unsigned long)seq);
183     memcpy(payload, seq_str, num_hold - 1);
184     char hdr[MAX_HDR];
185     int hdr_len = snprintf(hdr, sizeof(hdr), "AT+SEND=%d,%d,",
186                             BENCH_DEST_ADDR, size);
187     uart_write_bytes(LORA_UART_NUM, hdr, hdr_len);
188     uart_write_bytes(LORA_UART_NUM, (char *)payload, size);
189     uart_write_bytes(LORA_UART_NUM, "\r\n", 2);
190 }
191
192 static int64_t wait_echo(int timeout_ms, int *rssi_out, int *snr_out)
193 {
194     int64_t t0 = esp_timer_get_time() >> 10;
195     char line[300];
196     bool ok_received = false;
197
198     while ((esp_timer_get_time() >> 10) - t0 < timeout_ms) {
199         if (!readline(line, sizeof(line), 100))
200             continue;
201
202         if (strstr(line, "+OK")) {
203             ESP_LOGI(TAG, "AT+SEND ack: %s", line);
204             ok_received = true;
205         } else if (strncmp(line, "+RCV=", 5) == 0) {
206             ESP_LOGI(TAG, "rx: %s", line);
207             if (!ok_received)

```

```

208         ESP_LOGW(TAG, "echo before +OK");
209         int64_t rtt = (esp_timer_get_time() >> 10) - t0;
210         *rssi_out = 0;
211         *snr_out = 0;
212         char *p = line + 5;
213         char *c1 = strchr(p, ',');
214         if (c1) {
215             char *c2 = strchr(c1 + 1, ',');
216             if (c2) {
217                 char *after = c2 + 1 + BENCH_PKT_SIZE;
218                 if (*after == ',')
219                     sscanf(after + 1, "%d,%d",
220                             rssi_out, snr_out);
221             }
222         }
223         return rtt;
224     }
225 }
226
227 ESP_LOGW(TAG, "wait_echo timed out");
228 return -1;
229 }
230
231 static bool wait_for_ready(char *line, int line_size)
232 {
233     int src, len;
234     char *data;
235
236     bool valid_line = readline(line, line_size, 500);
237     bool valid_rcvp = valid_line && parse_rcv(line, &src, &data, &len);
238     bool valid_ascii = valid_rcvp && (len == 5) && !memcmp(data, "READY", 5);
239
240     if (valid_line)
241         ESP_LOGI(TAG, "handshake rx: %s (match=%d)", line, valid_ascii);
242
243     return !valid_ascii;
244 }
245
246 void lora_bench_task(void *arg)
247 {
248     (void)arg;
249
250     ESP_LOGI(TAG, "=== DevKit TX bench ===");
251     ESP_LOGI(TAG, "addr=%d dest=%d net=%d SF=%d BW=%d",
252             LORA_ADDRESS_SENDER, BENCH_DEST_ADDR, LORA_NETWORK_ID,
253             BENCH_SF, BENCH_BW);
254
255     uart_init_lora();
256     vTaskDelay(pdMS_TO_TICKS(500));
257     module_init();
258     vTaskDelay(pdMS_TO_TICKS(500));
259
260     ESP_LOGI(TAG, "waiting for handshake...");
261
262     char line[300];
263     while (wait_for_ready(line, sizeof(line)));
264     const char *cmd = "AT+SEND=1,5,START\r\n";
265     uart_write_bytes(LORA_UART_NUM, cmd, strlen(cmd));
266     readline(line, sizeof(line), 500);
267     ESP_LOGI(TAG, "START sent");
268
269     vTaskDelay(pdMS_TO_TICKS(100));
270
271     int delivered = 0;
272     int64_t rtt_sum = 0;
273
274     ESP_LOGI(TAG, "echo trial: %d packets, %d bytes", BENCH_N,
275             BENCH_PKT_SIZE);
276
277     for (int i = 0; i < BENCH_N; i++) {

```

```

278     int rssi, snr;
279     send_msg(i, BENCH_PKT_SIZE);
280     int64_t rtt = wait_echo(BENCH_TIMEOUT_MS, &rssi, &snr);
281
282     if (rtt >= 0) {
283         delivered++;
284         rtt_sum += rtt;
285         if (delivered % 10 == 0)
286             ESP_LOGI(TAG,
287                      " [%d] RTT=%lldms RSSI=%d SNR=%d", i,
288                      rtt, rssi, snr);
289     } else {
290         ESP_LOGW(TAG, " [%d] timeout", i);
291     }
292     vTaskDelay(pdMS_TO_TICKS(BENCH_INTER_PACKET_MS));
293 }
294
295 ESP_LOGI(TAG, "");
296 ESP_LOGI(TAG, "=== results ===");
297 ESP_LOGI(TAG, "delivered: %d/%d (%.1f%%)", delivered, BENCH_N,
298           100.0f * delivered / BENCH_N);
299 if (delivered > 0)
300     ESP_LOGI(TAG, "avg RTT: %lld ms", rtt_sum / delivered);
301
302 vTaskDelete(NULL);
303 }
304 #endif

```


esp/esp32_devkitv1/main/src/main.c

```
1  /**
2   * main.c - ESP32 DevKitV1 entry point
3   *
4   * Initializes all subsystems, creates tasks, monitors heap.
5   */
6
7  #include "esp_log.h"
8  #include "esp_system.h"
9  #include "freertos/FreeRTOS.h"
10 #include "freertos/task.h"
11 #include <stdio.h>
12
13 #include "cam_spi.h"
14 #include "config.h"
15 #include "fpga_spi.h"
16 #include "image_processor.h"
17 #include "isr_signals.h"
18 #include "lora.h"
19 #include "peripherals/fpga_gpio.h"
20 #include "peripherals/i2c_bus.h"
21 #include "peripherals/sensors_temp_humidity.h"
22
23 static const char *TAG = "MAIN";
24
25 static esp_err_t init_system(void)
26 {
27     ESP_LOGI(TAG, "=== Trail Camera IV - ESP32 DevKitV1 ===");
28     ESP_LOGI(TAG, "IDF %s | heap %lu bytes", esp_get_idf_version(),
29             esp_get_free_heap_size());
30
31     esp_err_t ret;
32
33     /* ISR semaphores must exist before any tasks */
34     ret = isr_init();
35     if (ret != ESP_OK) return ret;
36     ret = cam_spi_init();
37     if (ret != ESP_OK) return ret;
38     ret = fpga_spi_init();
39     if (ret != ESP_OK) return ret;
40     ret = lora_init();
41     if (ret != ESP_OK) return ret;
42     ret = image_processor_init();
43     if (ret != ESP_OK) return ret;
44
45     /* I2C bus must be up before any I2C peripheral (AHT20, FPGA GPIO) */
46     static const i2c_bus_config_t i2c_cfg = {
47         .port = FPGA_I2C_PORT,
48         .sda_gpio = FPGA_I2C_SDA_PIN,
49         .scl_gpio = FPGA_I2C_SCL_PIN,
50         .clk_speed_hz = FPGA_I2C_CLK_HZ,
51         .enable_internal_pullup =
52             false, /* external 4.7k pull-ups fitted */
53     };
54     ret = i2c_bus_init(&i2c_cfg);
55     if (ret != ESP_OK) return ret;
56
57     ret = sensors_init();
58     if (ret != ESP_OK) return ret;
59     ret = fpga_gpio_init();
60     if (ret != ESP_OK)
61         ESP_LOGW(TAG,
62             "FPGA GPIO expander unavailable (%s) - I2C buttons "
63             "disabled",
64             esp_err_to_name(ret));
65
66     ESP_LOGI(TAG, "all subsystems ready");
67     return ESP_OK;
```

```

68 }
69
70 static void create_tasks(void)
71 {
72     xTaskCreate(cam_spi_receive_task, "cam_rx", STACK_SIZE_MEDIUM, NULL,
73                 TASK_PRIORITY_HIGH, NULL);
74     xTaskCreate(image_processor_task, "img_proc", STACK_SIZE_LARGE, NULL,
75                 TASK_PRIORITY_MEDIUM, NULL);
76     xTaskCreate(sensors_task, "sensors", STACK_SIZE_MEDIUM, NULL,
77                 TASK_PRIORITY_LOW, NULL);
78
79 #ifdef TEST_MODE_FPGA_PATTERNS
80     xTaskCreate(fpga_test_task, "fpga_test", STACK_SIZE_MEDIUM, NULL,
81                 TASK_PRIORITY_MEDIUM, NULL);
82 #endif
83 #ifdef TEST_MODE_FPGA_GPIO
84     xTaskCreate(fpga_gpio_test_task, "fpga_gpio_test", STACK_SIZE_MEDIUM,
85                 NULL, TASK_PRIORITY_LOW, NULL);
86 #endif
87 }
88
89 static void monitor_heap(void)
90 {
91     uint32_t prev = esp_get_free_heap_size();
92
93     for (;;) {
94         vTaskDelay(SECONDS_TO_TICKS(10));
95         uint32_t now = esp_get_free_heap_size();
96         ESP_LOGI(TAG, "heap: %lu (%+ld)", now, (int32_t)(now - prev));
97         prev = now;
98     }
99 }
100
101 void app_main(void)
102 {
103 #ifdef TEST_MODE_LORA_BENCH
104     /* Bench mode: only LoRa bench task runs - no camera/FPGA/sensor init
105      */
106     xTaskCreate(lora_bench_task, "lora_bench", STACK_SIZE_LARGE, NULL,
107                 TASK_PRIORITY_MEDIUM, NULL);
108     monitor_heap();
109 #else
110     if (init_system() != ESP_OK) {
111         ESP_LOGE(TAG, "init failed - halted");
112         return;
113     }
114     create_tasks();
115     monitor_heap();
116 #endif
117 }

```

esp/esp32_devkitv1/main/src/peripherals/aht20.c

```
1  /**
2   * aht20.c - AHT20 temperature/humidity sensor driver
3   *
4   * Uses i2c_bus.h for bus access. Init → trigger → read.
5   */
6
7  #include "peripherals/aht20.h"
8  #include "peripherals/i2c_bus.h"
9
10 #include "esp_log.h"
11 #include "freertos/FreeRTOS.h"
12 #include "freertos/task.h"
13 #include <string.h>
14
15 static const char *TAG = "AHT20";
16
17 #define CMD_RESET    0xBA
18 #define CMD_INIT     0xBE
19 #define CMD_MEASURE  0xAC
20 #define READ_LEN     6
21
22 static esp_err_t soft_reset(aht20_t *dev)
23 {
24     uint8_t cmd = CMD_RESET;
25     esp_err_t err = i2c_bus_write(dev->addr, &cmd, 1, dev->timeout_ms);
26     if (err != ESP_OK) return err;
27     vTaskDelay(pdMS_TO_TICKS(40));
28     return ESP_OK;
29 }
30
31 static esp_err_t send_init(aht20_t *dev)
32 {
33     uint8_t cmd[3] = {CMD_INIT, 0x08, 0x00};
34     esp_err_t err =
35         i2c_bus_write(dev->addr, cmd, sizeof(cmd), dev->timeout_ms);
36     if (err != ESP_OK) return err;
37     vTaskDelay(pdMS_TO_TICKS(20));
38     return ESP_OK;
39 }
40
41 esp_err_t aht20_init(aht20_t *dev, uint8_t addr)
42 {
43     if (!dev) return ESP_ERR_INVALID_ARG;
44
45     memset(dev, 0, sizeof(*dev));
46     dev->addr = addr;
47     dev->timeout_ms = 1000;
48
49     /* Reset → init → wait for ready */
50     soft_reset(dev);
51
52     esp_err_t err = send_init(dev);
53     if (err != ESP_OK) return err;
54
55     uint8_t data[READ_LEN];
56     for (int i = 0; i < 5; i++) {
57         vTaskDelay(pdMS_TO_TICKS(20));
58         err = i2c_bus_read(dev->addr, data, READ_LEN, dev->timeout_ms);
59         if (err == ESP_OK) {
60             ESP_LOGI(TAG, "ready (status 0x%02X)", data[0]);
61             return ESP_OK;
62         }
63     }
64
65     ESP_LOGE(TAG, "init failed");
66     return err;
67 }
```

```

68
69 esp_err_t aht20_read(aht20_t *dev, float *temp_c, float *rh_pct)
70 {
71     if (!dev || !temp_c || !rh_pct) return ESP_ERR_INVALID_ARG;
72
73     uint8_t cmd[3] = {CMD_MEASURE, 0x33, 0x00};
74     esp_err_t err =
75         i2c_bus_write(dev->addr, cmd, sizeof(cmd), dev->timeout_ms);
76     if (err != ESP_OK) return err;
77
78     vTaskDelay(pdMS_TO_TICKS(90));
79
80     uint8_t data[READ_LEN];
81     for (int i = 0; i < 10; i++) {
82         err = i2c_bus_read(dev->addr, data, READ_LEN, dev->timeout_ms);
83         if (err != ESP_OK) return err;
84
85         /* Bit 7 of status = busy */
86         if ((data[0] & 0x80) == 0) {
87             uint32_t hum_raw = ((uint32_t)data[1] << 12) |
88                               ((uint32_t)data[2] << 4) |
89                               ((data[3] >> 4) & 0x0F);
90
91             uint32_t temp_raw =
92                 (((uint32_t)data[3] & 0x0F) << 16) |
93                 ((uint32_t)data[4] << 8) | (uint32_t)data[5];
94
95             *rh_pct = (float)hum_raw * 100.0f / 1048576.0f;
96             *temp_c =
97                 (float)temp_raw * 200.0f / 1048576.0f - 50.0f;
98             return ESP_OK;
99         }
100
101         vTaskDelay(pdMS_TO_TICKS(10));
102     }
103
104     ESP_LOGW(TAG, "sensor busy - timeout");
105     return ESP_ERR_TIMEOUT;
106 }

```

esp/esp32_devkitv1/main/src/peripherals/fpga_gpio.c

```
1  /**
2   * fpga_gpio.c - FPGA I2C GPIO Expander Driver
3   *
4   * PCA9534-compatible I2C slave on Tang Nano 9K at address 0x27.
5   * Requires i2c_bus to be initialised first.
6   */
7
8  #include "peripherals/fpga_gpio.h"
9  #include "config.h"
10 #include "peripherals/i2c_bus.h"
11 #include <string.h>
12
13 #include "esp_log.h"
14 #include "freertos/FreeRTOS.h"
15 #include "freertos/task.h"
16
17 static const char *TAG = "FPGA_GPIO";
18
19 /*****
20  * Internal helpers
21  *****/
22
23 static esp_err_t write_reg(uint8_t reg, uint8_t value)
24 {
25     uint8_t buf[2] = {reg, value};
26     return i2c_bus_write(FPGA_I2C_ADDR, buf, sizeof(buf),
27                          FPGA_I2C_TIMEOUT_MS);
28 }
29
30 static esp_err_t read_reg(uint8_t reg, uint8_t *value)
31 {
32     return i2c_bus_write_read(FPGA_I2C_ADDR, &reg, 1, value, 1,
33                               FPGA_I2C_TIMEOUT_MS);
34 }
35
36 /*****
37  * Public API
38  *****/
39
40 esp_err_t fpga_gpio_init(void)
41 {
42     esp_err_t err;
43
44     /* Direction: bits[7:4]=inputs(unused), bits[3:0]=outputs(stepppers) */
45     err = write_reg(FPGA_GPIO_REG_DIR, FPGA_GPIO_DIR_BOOT);
46     if (err != ESP_OK) {
47         ESP_LOGE(TAG, "DIR_REG write failed: %s",
48                  esp_err_to_name(err));
49         return err;
50     }
51
52     /* Polarity invert: bits[2:0] inverted so 1=pressed for active-low
53      * buttons */
54     err = write_reg(FPGA_GPIO_REG_INV, FPGA_GPIO_INV_BOOT);
55     if (err != ESP_OK) {
56         ESP_LOGE(TAG, "INV_REG write failed: %s",
57                  esp_err_to_name(err));
58         return err;
59     }
60
61     /* Clear steppers on startup */
62     err = write_reg(FPGA_GPIO_REG_OUT, 0x00);
63     if (err != ESP_OK) {
64         ESP_LOGE(TAG, "OUT_REG clear failed: %s",
65                  esp_err_to_name(err));
66         return err;
67     }
68 }
```

```

68     ESP_LOGI(TAG, "ready - addr 0x%02X, DIR=0x%02X, INV=0x%02X",
69               FPGA_I2C_ADDR, FPGA_GPIO_DIR_BOOT, FPGA_GPIO_INV_BOOT);
70     return ESP_OK;
71 }
72
73
74 esp_err_t fpga_gpio_set_steppers(uint8_t pattern)
75 {
76     return write_reg(FPGA_GPIO_REG_OUT, pattern & 0x0F);
77 }
78
79 esp_err_t fpga_gpio_read_buttons(uint8_t *buttons)
80 {
81     if (!buttons) return ESP_ERR_INVALID_ARG;
82     uint8_t raw;
83     esp_err_t err = read_reg(FPGA_GPIO_REG_IN, &raw);
84     if (err != ESP_OK) return err;
85     *buttons = raw & 0x07; /* mask to 3 button bits */
86     return ESP_OK;
87 }
88
89 /*****
90  * Test Task
91  *****/
92
93 #ifdef TEST_MODE_FPGA_GPIO
94
95 #include "fpga_spi.h"
96 #include "utils.h"
97
98 /* Test patterns sent to FPGA BRAM over SPI */
99 typedef enum {
100     BPAT_GRADIENT = 0,
101     BPAT_CHECKER,
102     BPAT_WHITE,
103     BPAT_BLACK,
104     BPAT_COUNT
105 } bpat_t;
106
107 static const char *bpat_names[] = {"gradient", "checker", "white", "black"};
108
109 static void fill_bpat(uint8_t *buf, bpat_t pat)
110 {
111     switch (pat) {
112     case BPAT_GRADIENT:
113         for (int i = 0; i < TEST_IMAGE_SIZE; i++) buf[i] = (uint8_t)i;
114         break;
115     case BPAT_CHECKER:
116         for (int i = 0; i < TEST_IMAGE_SIZE; i++)
117             buf[i] = (i & 0x10) ? 0xFF : 0x00;
118         break;
119     case BPAT_WHITE: memset(buf, 0xFF, TEST_IMAGE_SIZE); break;
120     case BPAT_BLACK: memset(buf, 0x00, TEST_IMAGE_SIZE); break;
121     default: memset(buf, 0x80, TEST_IMAGE_SIZE); break;
122     }
123 }
124
125 void fpga_gpio_test_task(void *pvParameters)
126 {
127     (void)pvParameters;
128
129     uint8_t *buf = dma_malloc(TEST_IMAGE_SIZE);
130     if (!buf) {
131         ESP_LOGE(TAG, "DMA alloc failed - task exiting");
132         vTaskDelete(NULL);
133         return;
134     }
135
136     bpat_t pat = BPAT_GRADIENT;
137     uint8_t prev_buttons = 0xFF;

```

```

138
139 /* Send initial pattern so the LCD shows something on boot */
140 fill_bpat(buf, pat);
141 fpga_spi_transmit(buf, TEST_IMAGE_SIZE);
142 ESP_LOGI(TAG, "boot pattern: %s", bpat_names[pat]);
143
144 for (;;) {
145     uint8_t buttons = 0;
146     esp_err_t err = fpga_gpio_read_buttons(&buttons);
147
148     if (err != ESP_OK) {
149         ESP_LOGE(TAG, "read failed: %s", esp_err_to_name(err));
150         vTaskDelay(pdMS_TO_TICKS(500));
151         continue;
152     }
153
154     if (buttons != prev_buttons) {
155         bool btn_l = (buttons >> FPGA_BTN_L_BIT) & 1;
156         bool btn_r = (buttons >> FPGA_BTN_R_BIT) & 1;
157         bool btn_s = (buttons >> FPGA_BTN_S_BIT) & 1;
158
159         ESP_LOGI(TAG, "buttons: L=%d R=%d S=%d (raw=0x%02X)",
160                 btn_l, btn_r, btn_s, buttons);
161
162         /* L = prev pattern, R = next pattern, S = reset to
163          * gradient */
164         if (btn_l)
165             pat = (bpat_t)((pat + BPAT_COUNT - 1) %
166                             BPAT_COUNT);
167         if (btn_r) pat = (bpat_t)((pat + 1) % BPAT_COUNT);
168         if (btn_s) pat = BPAT_GRADIENT;
169
170         if (btn_l || btn_r || btn_s) {
171             fill_bpat(buf, pat);
172             esp_err_t tx =
173                 fpga_spi_transmit(buf, TEST_IMAGE_SIZE);
174             if (tx == ESP_OK)
175                 ESP_LOGI(TAG, "sent pattern: %s",
176                         bpat_names[pat]);
177             else
178                 ESP_LOGE(TAG, "SPI tx failed: %s",
179                         esp_err_to_name(tx));
180         }
181
182         /* Mirror to steppers */
183         uint8_t step_pattern = 0x00;
184         if (btn_s) step_pattern = 0x0F;
185         else if (btn_l) step_pattern = 0x01;
186         else if (btn_r) step_pattern = 0x02;
187         fpga_gpio_set_steppers(step_pattern);
188
189         prev_buttons = buttons;
190     }
191
192     vTaskDelay(pdMS_TO_TICKS(50));
193 }
194 }
195
196 #endif

```

```

1  /**
2   * motor.c - 28BYJ-48 stepper motor driver (ULN2003)
3   *
4   * Full-step 2-coil sequence with trapezoidal ramp profile.
5   */
6
7  #include "peripherals/motor.h"
8
9  #include "driver/gpio.h"
10 #include "freertos/FreeRTOS.h"
11 #include "freertos/task.h"
12
13 /*****
14  * Step Table - 2-coil full-step (→→→ABBCCDDA)
15  *****/
16
17 static const uint8_t STEP[4][4] = {
18     {1, 1, 0, 0},
19     {0, 1, 1, 0},
20     {0, 0, 1, 1},
21     {1, 0, 0, 1},
22 };
23
24 /*****
25  * Ramp Defaults
26  *****/
27
28 #define RAMP_START_MS 14
29 #define RAMP_MIN_MS 2
30 #define RAMP_STEPS 600
31
32 /*****
33  * Internals
34  *****/
35
36 static void apply_phase(const motor_stepper_t *m)
37 {
38     const uint8_t *s = STEP[m->phase & 3];
39     int v[4] = {s[0], s[1], s[2], s[3]};
40
41     gpio_set_level((gpio_num_t)m->in1_gpio, v[m->wire_map[0]]);
42     gpio_set_level((gpio_num_t)m->in2_gpio, v[m->wire_map[1]]);
43     gpio_set_level((gpio_num_t)m->in3_gpio, v[m->wire_map[2]]);
44     gpio_set_level((gpio_num_t)m->in4_gpio, v[m->wire_map[3]]);
45 }
46
47 /*****
48  * Public Interface
49  *****/
50
51 esp_err_t motor_stepper_init(motor_stepper_t *m)
52 {
53     if (!m) return ESP_ERR_INVALID_ARG;
54
55     gpio_config_t io = {
56         .pin_bit_mask = (1ULL << m->in1_gpio) | (1ULL << m->in2_gpio) |
57             (1ULL << m->in3_gpio) | (1ULL << m->in4_gpio),
58         .mode = GPIO_MODE_OUTPUT,
59         .pull_up_en = GPIO_PULLUP_DISABLE,
60         .pull_down_en = GPIO_PULLEDOWN_DISABLE,
61         .intr_type = GPIO_INTR_DISABLE,
62     };
63
64     esp_err_t err = gpio_config(&io);
65     if (err != ESP_OK) return err;
66
67     apply_phase(m);

```



```

68     return ESP_OK;
69 }
70
71 void motor_stepper_set_phase(motor_stepper_t *m, int phase)
72 {
73     if (!m) return;
74     m->phase = phase & 3;
75     apply_phase(m);
76 }
77
78 void motor_stepper_release(motor_stepper_t *m)
79 {
80     if (!m) return;
81     gpio_set_level((gpio_num_t)m->in1_gpio, 0);
82     gpio_set_level((gpio_num_t)m->in2_gpio, 0);
83     gpio_set_level((gpio_num_t)m->in3_gpio, 0);
84     gpio_set_level((gpio_num_t)m->in4_gpio, 0);
85 }
86
87 void motor_stepper_move_ramped(motor_stepper_t *m, int steps, int dir,
88                               int start_ms, int min_ms, int ramp)
89 {
90     if (!m || steps <= 0) return;
91
92     dir = (dir < 0) ? -1 : 1;
93     if (min_ms < 1) min_ms = 1;
94     if (start_ms < min_ms) start_ms = min_ms;
95     if (ramp < 1) ramp = 1;
96     if (2 * ramp > steps) ramp = steps / 2;
97
98     for (int i = 0; i < steps; i++) {
99         m->phase = (m->phase + dir) & 3;
100        apply_phase(m);
101
102        int d;
103        if (i < ramp) d = start_ms - ((start_ms - min_ms) * i) / ramp;
104        else if (i >= steps - ramp)
105            d = start_ms -
106                ((start_ms - min_ms) * (steps - 1 - i)) / ramp;
107        else d = min_ms;
108
109        vTaskDelay(pdMS_TO_TICKS(d));
110    }
111 }
112
113 void motor_stepper_swing_and_release(motor_stepper_t *m, int steps, int dir,
114                                     int hold_ms)
115 {
116     if (!m || steps <= 0) return;
117
118     motor_stepper_move_ramped(m, steps, dir, RAMP_START_MS, RAMP_MIN_MS,
119                               RAMP_STEPS);
120     vTaskDelay(pdMS_TO_TICKS(hold_ms));
121     motor_stepper_move_ramped(m, steps, -dir, RAMP_START_MS, RAMP_MIN_MS,
122                               RAMP_STEPS);
123     motor_stepper_release(m);
124 }
125
126 void motor_stepper_swing_cw(motor_stepper_t *m, int steps, int hold_ms)
127 {
128     motor_stepper_swing_and_release(m, steps, +1, hold_ms);
129 }
130
131 void motor_stepper_swing_ccw(motor_stepper_t *m, int steps, int hold_ms)
132 {
133     motor_stepper_swing_and_release(m, steps, -1, hold_ms);
134 }

```

esp/esp32_devkitv1/main/src/peripherals/pir.c

```
1  /**
2   * pir.c - PIR motion sensor GPIO interface
3   */
4
5  #include "peripherals/pir.h"
6  #include "driver/gpio.h"
7
8  static int pins[3] = {-1, -1, -1};
9
10 esp_err_t pir_init(int pir1, int pir2, int pir3)
11 {
12     pins[0] = pir1;
13     pins[1] = pir2;
14     pins[2] = pir3;
15
16     gpio_config_t io = {
17         .pin_bit_mask = (1ULL << pir1) | (1ULL << pir2) | (1ULL << pir3),
18         .mode = GPIO_MODE_INPUT,
19         .pull_up_en = GPIO_PULLUP_DISABLE,
20         .pull_down_en = GPIO_PULLEDOWN_DISABLE,
21         .intr_type = GPIO_INTR_DISABLE,
22     };
23
24     return gpio_config(&io);
25 }
26
27 bool pir_read(pir_id_t pir)
28 {
29     int idx = (int)pir;
30     if (idx < 0 || idx > 2 || pins[idx] < 0) return 0;
31     return gpio_get_level((gpio_num_t)pins[idx]);
32 }
```

```

1  /**
2   * @file sensors_temp_humidity.c
3   * @brief Sensor subsystem - AHT20 + PIR + stepper motor
4   */
5
6  #include "peripherals/sensors_temp_humidity.h"
7  #include "config.h"
8
9  #include <inttypes.h>
10 #include <stdbool.h>
11 #include <stdio.h>
12
13 #include "freertos/FreeRTOS.h"
14 #include "freertos/task.h"
15
16 #include "driver/gpio.h"
17 #include "esp_err.h"
18 #include "esp_log.h"
19
20 #include "peripherals/aht20.h"
21 #include "peripherals/i2c_bus.h"
22 #include "peripherals/motor.h"
23
24 static const char *TAG = "sensors";
25
26 #define I2C_SDA_GPIO 21
27 #define I2C_SCL_GPIO 22
28
29 #define PIR1_GPIO 35
30 #define PIR2_GPIO 34
31 #define PIR3_GPIO 4
32
33 #define STEP_IN1_GPIO 13
34 #define STEP_IN2_GPIO 12
35 #define STEP_IN3_GPIO 14
36 #define STEP_IN4_GPIO 27
37
38 // FPGA + camera enable pins
39 #define FPGA_EN_GPIO 2
40 #define CAM_EN_GPIO 15
41
42 #define STEPS_90 1024
43 #define HOLD_MS 3000
44 #define PIR_SETTLE_TIMEOUT_MS 30000
45 #define PIR3_ACTIVE_MS (HOLD_MS + 2000)
46
47 static aht20_t s_aht20;
48 static bool s_sensor_ok;
49 static motor_stepper_t s_motor;
50
51 static void pir_init_pin(gpio_num_t pin, bool input_only)
52 {
53     gpio_config_t io = {
54         .pin_bit_mask = (1ULL << pin),
55         .mode = GPIO_MODE_INPUT,
56         .pull_up_en = GPIO_PULLUP_DISABLE,
57         .pull_down_en =
58             input_only ? GPIO_PULLDOWN_DISABLE : GPIO_PULLDOWN_ENABLE,
59         .intr_type = GPIO_INTR_DISABLE,
60     };
61     ESP_ERROR_CHECK(gpio_config(&io));
62 }
63
64 static void enable_outputs_init(void)
65 {
66     gpio_config_t io = {
67         .pin_bit_mask = (1ULL << FPGA_EN_GPIO) | (1ULL << CAM_EN_GPIO),

```

```

68         .mode = GPIO_MODE_OUTPUT,
69         .pull_up_en = GPIO_PULLUP_DISABLE,
70         .pull_down_en = GPIO_PULLDOWN_DISABLE,
71         .intr_type = GPIO_INTR_DISABLE,
72     };
73     ESP_ERROR_CHECK(gpio_config(&io));
74
75     gpio_set_level((gpio_num_t)FPGA_EN_GPIO, 1);
76     gpio_set_level((gpio_num_t)CAM_EN_GPIO, 1);
77 }
78
79 static inline void enable_outputs_set(bool on)
80 {
81     int level = on ? 1 : 0;
82     gpio_set_level((gpio_num_t)FPGA_EN_GPIO, level);
83     gpio_set_level((gpio_num_t)CAM_EN_GPIO, level);
84 }
85
86 #ifdef DEBUG_PIR_ALWAYS_ON
87 #warning "DEBUG_PIR_ALWAYS_ON - PIR forced to center trigger"
88 #endif
89
90 static int detect_triggered_pir(void)
91 {
92     #ifdef DEBUG_PIR_ALWAYS_ON
93         return 3;
94     #else
95         if (gpio_get_level((gpio_num_t)PIR1_GPIO)) return 1;
96         if (gpio_get_level((gpio_num_t)PIR2_GPIO)) return 2;
97         if (gpio_get_level((gpio_num_t)PIR3_GPIO)) return 3;
98         return 0;
99     #endif
100 }
101
102 static void wait_for_all_pirs_low(void)
103 {
104     int elapsed = 0;
105
106     while (gpio_get_level((gpio_num_t)PIR1_GPIO) ||
107            gpio_get_level((gpio_num_t)PIR2_GPIO) ||
108            gpio_get_level((gpio_num_t)PIR3_GPIO)) {
109         if (elapsed >= PIR_SETTLE_TIMEOUT_MS) { break; }
110
111         vTaskDelay(pdMS_TO_TICKS(100));
112         elapsed += 100;
113     }
114
115     vTaskDelay(pdMS_TO_TICKS(200));
116 }
117
118 static inline float c_to_f(float c) { return c * 9.0f / 5.0f + 32.0f; }
119
120 esp_err_t sensors_init(void)
121 {
122     // PIR pins
123     pir_init_pin((gpio_num_t)PIR1_GPIO, true);
124     pir_init_pin((gpio_num_t)PIR2_GPIO, true);
125     pir_init_pin((gpio_num_t)PIR3_GPIO, false);
126
127     // FPGA + camera enable pins
128     enable_outputs_init();
129
130     // Stepper configuration
131     s_motor = (motor_stepper_t){
132         .in1_gpio = STEP_IN1_GPIO,
133         .in2_gpio = STEP_IN2_GPIO,
134         .in3_gpio = STEP_IN3_GPIO,
135         .in4_gpio = STEP_IN4_GPIO,
136         .wire_map = {1, 0, 3, 2},
137         .phase = 0,

```

```

138     };
139
140     esp_err_t err = motor_stepper_init(&s_motor);
141     if (err != ESP_OK) {
142         ESP_LOGE(TAG, "motor_stepper_init failed: %s",
143             esp_err_to_name(err));
144         return err;
145     }
146
147     motor_stepper_set_phase(&s_motor, 0);
148     motor_stepper_release(&s_motor);
149
150     // I2C bus
151     if (!i2c_bus_is_init()) {
152         const i2c_bus_config_t bus_cfg = {
153             .port = I2C_NUM_0,
154             .sda_gpio = I2C_SDA_GPIO,
155             .scl_gpio = I2C_SCL_GPIO,
156             .enable_internal_pullup = true,
157         };
158
159         err = i2c_bus_init(&bus_cfg);
160         if (err != ESP_OK) {
161             ESP_LOGE(TAG, "I2C bus init failed: %s",
162                 esp_err_to_name(err));
163             return err;
164         }
165     }
166
167     // AHT20
168     s_sensor_ok = (aht20_init(&s_aht20, AHT20_I2C_ADDR_DEFAULT) == ESP_OK);
169     if (!s_sensor_ok) {
170         ESP_LOGW(
171             TAG,
172             "AHT20 unavailable - continuing without temp/humidity");
173     }
174
175     ESP_LOGI(TAG, "sensor subsystem initialised");
176     return ESP_OK;
177 }
178
179 void sensors_task(void *pvParameters)
180 {
181     (void)pvParameters;
182
183     ESP_LOGI(TAG, "sensor task started - polling PIRs");
184
185     #ifdef DEBUG_PIR_ALWAYS_ON
186         enable_outputs_set(true);
187         ESP_LOGW(TAG, "DEBUG_PIR_ALWAYS_ON - outputs held high");
188         for (;;) vTaskDelay(pdMS_TO_TICKS(10000));
189     #else
190         for (;;) {
191             int trig = detect_triggered_pir();
192
193             if (trig == 0) {
194                 vTaskDelay(pdMS_TO_TICKS(100));
195                 continue;
196             }
197
198             if (s_sensor_ok) {
199                 float t_c = 0.0f;
200                 float rh = 0.0f;
201
202                 if (aht20_read(&s_aht20, &t_c, &rh) == ESP_OK) {
203                     ESP_LOGI(TAG, "PIR%d | %.1f F | %.0f%% RH",
204                         trig, c_to_f(t_c), rh);
205                 } else {
206                     ESP_LOGW(TAG, "PIR%d | AHT20 read failed",
207                         trig);

```

```

208         }
209     } else {
210         ESP_LOGI(TAG, "PIR%d | sensor unavailable", trig);
211     }
212
213     enable_outputs_set(true);
214
215     switch (trig) {
216     case 1:
217         motor_stepper_swing_cw(&s_motor, STEPS_90, HOLD_MS);
218         break;
219
220     case 2:
221         motor_stepper_swing_ccw(&s_motor, STEPS_90, HOLD_MS);
222         break;
223
224     case 3:
225         vTaskDelay(pdMS_TO_TICKS(PIR3_ACTIVE_MS));
226         break;
227
228     default: break;
229     }
230
231     motor_stepper_release(&s_motor);
232     enable_outputs_set(false);
233     wait_for_all_pirs_low();
234 }
235 #endif
236 }

```

esp/esp32_devkitv1/main/src/utilities/image_processor.c

```
1  /**
2   * image_processor.c -- Camera -> FPGA -> LoRa pipeline
3   *
4   * Dequeues images from cam_spi, sends to FPGA for WHT compression,
5   * reads back compressed packet, transmits over LoRa.
6   */
7
8  #include "esp_log.h"
9  #include "freertos/FreeRTOS.h"
10 #include "freertos/task.h"
11
12 #include "cam_spi.h"
13 #include "fpga_spi.h"
14 #include "image_processor.h"
15 #include "lora.h"
16 #include "utils.h"
17
18 static const char *TAG = "IMG_PROC";
19
20 #define FPGA_TX_BUF_SIZE 512
21
22 static size_t compressed_len(const uint8_t *buf, size_t buf_size)
23 {
24     size_t pos = 0;
25     while (pos + 2 <= buf_size) {
26         uint8_t header_hi = buf[pos];
27         uint8_t header_lo = buf[pos + 1];
28         if (header_hi == 0 && header_lo == 0)
29             break;
30         int coeff_n = header_lo & 0x07;
31         pos += 2 + coeff_n * 3;
32     }
33     return pos;
34 }
35
36 esp_err_t image_processor_init(void)
37 {
38     ESP_LOGI(TAG, "initialized");
39     return ESP_OK;
40 }
41
42 void image_processor_task(void *pvParameters)
43 {
44     QueueHandle_t queue = cam_spi_get_queue();
45     image_data_t img;
46
47     uint8_t *rx_buf = dma_malloc(FPGA_TX_BUF_SIZE);
48     if (!rx_buf) {
49         ESP_LOGE(TAG, "DMA alloc failed for rx buffer");
50         vTaskDelete(NULL);
51         return;
52     }
53
54     ESP_LOGI(TAG, "task started");
55
56     for (;;) {
57         if (xQueueReceive(queue, &img, portMAX_DELAY) == pdTRUE) {
58             esp_err_t ret =
59                 fpga_spi_transmit(img.buffer, img.size);
60             safe_free((void **)&img.buffer);
61
62             if (ret != ESP_OK) {
63                 ESP_LOGE(TAG, "FPGA tx failed: %s",
64                     esp_err_to_name(ret));
65                 continue;
66             }
67         }
```

```

68     vTaskDelay(pdMS_TO_TICKS(50));
69
70     ret = fpga_spi_receive(rx_buf, FPGA_TX_BUF_SIZE);
71     if (ret != ESP_OK) {
72         ESP_LOGE(TAG, "FPGA rx failed: %s",
73                 esp_err_to_name(ret));
74         continue;
75     }
76
77     size_t pkt_len = compressed_len(rx_buf, FPGA_TX_BUF_SIZE);
78     ESP_LOGI(TAG, "rx: %02x %02x %02x %02x %02x %02x %02x %02x | len=%zu",
79             rx_buf[0], rx_buf[1], rx_buf[2], rx_buf[3],
80             rx_buf[4], rx_buf[5], rx_buf[6], rx_buf[7],
81             pkt_len);
82     if (pkt_len == 0) {
83         continue;
84     }
85
86     ESP_LOGI(TAG, "%zu compressed bytes", pkt_len);
87
88     ret = lora_send_packet(rx_buf, pkt_len);
89     if (ret != ESP_OK) {
90         ESP_LOGW(TAG, "LoRa send skipped (%zu bytes): %s",
91                 pkt_len, esp_err_to_name(ret));
92     }
93 }
94 }
95 }

```



```

1  /**
2   * isr.c - Interrupt service routines
3   *
4   * Central ISR container. Handlers dispatch via semaphores.
5   * No business logic here - just signal forwarding.
6   */
7
8  #include "driver/gpio.h"
9  #include "driver/gptimer.h"
10 #include "esp_log.h"
11 #include "freertos/FreeRTOS.h"
12 #include "freertos/semphr.h"
13
14 static const char *TAG = "ISR";
15
16 /*****
17  * Shared Semaphores (extern'd via isr_signals.h)
18  *****/
19
20 SemaphoreHandle_t g_trigger_sem = NULL;      /* LoRa → capture */
21 SemaphoreHandle_t g_button_sem = NULL;      /* button → FPGA test */
22 SemaphoreHandle_t g_timer_capture_sem = NULL; /* timer → capture */
23
24 /*****
25  * Init
26  *****/
27
28 esp_err_t isr_init(void)
29 {
30     g_trigger_sem = xSemaphoreCreateBinary();
31     g_button_sem = xSemaphoreCreateBinary();
32     g_timer_capture_sem = xSemaphoreCreateBinary();
33
34     if (!g_trigger_sem || !g_button_sem || !g_timer_capture_sem) {
35         ESP_LOGE(TAG, "semaphore creation failed");
36         return ESP_FAIL;
37     }
38
39     ESP_LOGI(TAG, "initialized");
40     return ESP_OK;
41 }
42
43 /*****
44  * Handlers
45  *****/
46
47 /* Task context - called from lora_receive_task */
48 void ISR_OnLoRaTrigger(void) { xSemaphoreGive(g_trigger_sem); }
49
50 /* True ISR - GPIO falling edge on test button */
51 void IRAM_ATTR ISR_OnButtonPress(void *arg)
52 {
53     BaseType_t woken = pdFALSE;
54     xSemaphoreGiveFromISR(g_button_sem, &woken);
55     portYIELD_FROM_ISR(woken);
56 }
57
58 /* True ISR - GPTimer alarm callback */
59 bool IRAM_ATTR ISR_OnTimerAlarm(gptimer_handle_t timer,
60                                 const gptimer_alarm_event_data_t *event_data,
61                                 void *user_ctx)
62 {
63     BaseType_t woken = pdFALSE;
64     xSemaphoreGiveFromISR(g_timer_capture_sem, &woken);
65     return woken == pdTRUE;
66 }

```

esp/esp32_devkitv1/main/src/utilities/utils.c

```
1  /**
2   * utils.c - Checksums, DMA memory, validation
3   *
4   * Non-inlined versions with logging. Fast/inline versions in utils.h.
5   */
6
7  #include "utils.h"
8  #include "esp_heap_caps.h"
9  #include "esp_log.h"
10 #include <inttypes.h>
11 #include <string.h>
12
13 static const char *TAG = "UTILS";
14
15 /*****
16  * Checksum
17  *****/
18
19 bool checksum_verify_xor(const uint8_t *data, size_t size, uint32_t expected)
20 {
21     uint32_t calc = checksum_calculate_xor(data, size);
22
23     if (calc != expected) {
24         ESP_LOGW(TAG,
25                 "checksum: expected 0x%08" PRIX32 " got 0x%08" PRIX32,
26                 expected, calc);
27         return false;
28     }
29
30     return true;
31 }
32
33 /*****
34  * Memory
35  *****/
36
37 void *dma_malloc(size_t size)
38 {
39     if (size == 0) return NULL;
40
41     size = align_4(size);
42     void *ptr = heap_caps_malloc(size, MALLOC_CAP_DMA);
43
44     if (!ptr) { ESP_LOGE(TAG, "DMA alloc failed: %zu bytes", size); }
45
46     return ptr;
47 }
48
49 /*****
50  * Validation
51  *****/
52
53 bool validate_image_header(const image_header_t *header)
54 {
55     if (!header) return false;
56
57     if (header->magic != IMAGE_HEADER_MAGIC) {
58         ESP_LOGW(TAG, "bad magic: 0x%08" PRIX32,
59                 (uint32_t)header->magic);
60         return false;
61     }
62
63     if (!validate_size_fast(header->size, IMAGE_MAX_SIZE_BYTES)) {
64         ESP_LOGW(TAG, "bad size: %" PRIu32, (uint32_t)header->size);
65         return false;
66     }
67 }
```

```
68         return true;
69     }
70
71     bool validate_size(size_t size, size_t max_size)
72     {
73         if (size == 0 || size > max_size) {
74             ESP_LOGW(TAG, "size %zu exceeds max %zu", size, max_size);
75             return false;
76         }
77         return true;
78     }
```

esp/esp32cam/main/inc/cam_ctrl.h

```
1  /*
2  * cam_ctrl.h
3  * ESP32-CAM camera control interface
4  */
5
6  #ifndef CAMERA_CONTROL_H
7  #define CAMERA_CONTROL_H
8
9  /* Uncomment for burst capture test mode */
10 // #define TEST_BURST_CAPTURE
11
12 #include "esp_err.h"
13 #include "freertos/FreeRTOS.h"
14 #include "freertos/queue.h"
15
16 /**
17  * Initialize OV2640 camera and apply sensor configuration.
18  * Resolution set by build mode: QVGA (test) or VGA (production).
19  */
20 esp_err_t camera_init(void);
21
22 /**
23  * Set the SPI transmit queue for downstream img data
24  */
25 void camera_set_tx_queue(QueueHandle_t queue);
26
27 /**
28  * Start the capture task.
29  * Test mode: burst dump to SD
30  * Production: SPI every frame, JPEG to SD every 3s
31  */
32 void camera_start_capture(void);
33
34 #endif /* CAMERA_CONTROL_H */
```

esp/esp32cam/main/inc/dma_spi.h

```
1  /*
2   * dma_spi.h
3   * ESP32-CAM SPI DMA Master Interface Header
4   *
5   * Provides high-performance SPI DMA transmission for img data
6   */
7
8  #ifndef DMA_SPI_MASTER_H
9  #define DMA_SPI_MASTER_H
10
11  #include <stdbool.h>
12  #include <stddef.h>
13  #include <stdint.h>
14
15  #include "esp_err.h"
16  #include "freertos/FreeRTOS.h"
17  #include "freertos/queue.h"
18  #include "img_data.h"
19
20  // SPI Pin Configuration
21  #define PIN_NUM_MOSI 13
22  #define PIN_NUM_MISO 12
23  #define PIN_NUM_CLK 14
24  #define PIN_NUM_CS 15
25
26  // Start and stop commands
27  #define CMD_START 1
28  #define CMD_STOP 2
29
30  // SPI Configuration
31  #define SPI_CLOCK_SPEED (10 * 1000 * 1000) // 10MHz
32  #define MAX_TRANSFER_SIZE (4092)
33
34  esp_err_t spi_dma_init(void);
35  void spi_dma_deinit(void);
36  esp_err_t spi_dma_send_img(const uint8_t *img_data, size_t img_size);
37  void spi_transmit_task(void *pvParameters);
38  bool spi_dma_stop_requested(void);
39  uint32_t spi_dma_epoch(void);
40
41  #endif // DMA_SPI_MASTER_H
```

esp/esp32cam/main/inc/img_buf.h

```
1  #ifndef IMAGE_BUFFER_POOL_H
2  #define IMAGE_BUFFER_POOL_H
3
4  #include <stddef.h>
5  #include <stdint.h>
6
7  #include "esp_err.h"
8
9  #include <stdbool.h>
10
11 #define IMAGE_BUFFER_COUNT 3
12 #define IMAGE_BUFFER_SIZE 76800 // 320x240 grayscale
13
14 /* Save ring buffer - 1-in-N frames buffered in PSRAM for post-capture JPEG */
15 #define SAVE_RING_SIZE      30
16 #define SAVE_INTERVAL       10
17
18 /**
19  * Initialize img buffer pool in PSRAM
20  */
21 esp_err_t img_buf_init(void);
22
23 /**
24  * Allocate a buffer from the pool
25  */
26 uint8_t *img_buffer_alloc(void);
27
28 /**
29  * Return a buffer to the pool
30  */
31 void img_buffer_free(uint8_t *buffer);
32
33 /**
34  * Get statistics about buffer pool usage
35  */
36 void img_buffer_get_stats(int *total, int *in_use, int *peak);
37
38 /**
39  * Initialize save ring buffer in PSRAM.
40  * Call once at boot, after img_buf_init().
41  */
42 esp_err_t save_ring_init(void);
43
44 /**
45  * Copy a frame into the next save ring slot.
46  * Overwrites oldest if full (circular).
47  * timestamp: esp_timer_get_time() value at capture.
48  */
49 void save_ring_push(const uint8_t *frame, size_t len, int64_t timestamp);
50
51 /**
52  * Get pointer and timestamp for a saved frame by index.
53  * idx 0 = oldest, idx count-1 = newest.
54  * Returns NULL if idx out of range.
55  */
56 const uint8_t *save_ring_get(int idx, int64_t *timestamp_out);
57
58 /**
59  * Number of frames currently in the ring.
60  */
61 int save_ring_count(void);
62
63 /**
64  * Reset ring to empty (does not free PSRAM slots).
65  */
66 void save_ring_reset(void);
67
```

```
68 /**
69  * Run unit test on buffer pool
70  */
71 void img_buf_unit_test(void);
72
73 #endif // IMAGE_BUFFER_POOL_H
```

esp/esp32cam/main/inc/img_data.h

```
1 #ifndef IMAGE_DATA_H
2 #define IMAGE_DATA_H
3
4 #include <stddef.h>
5 #include <stdint.h>
6
7 // Image data structure for passing between tasks
8 typedef struct {
9     uint8_t *buffer;
10    size_t size;
11 } img_data_t;
12
13 #endif // IMAGE_DATA_H
```


esp/esp32cam/main/inc/ov2640_cfg.h

```
1  /*
2  * ov2640_cfg.h
3  * OV2640 sensor configuration - config-by-struct pattern
4  *
5  * Wraps the Espressif sensor_t function pointer mess into a clean
6  * struct-based interface. Configure once, apply once, never touch
7  * the raw API again.
8  */
9
10 #ifndef OV2640_CONFIG_H
11 #define OV2640_CONFIG_H
12
13 #include "esp_camera.h"
14 #include "esp_err.h"
15
16 typedef struct {
17     int aec_enable;           /* 0 = manual exposure, 1 = auto */
18     int agc_enable;          /* 0 = manual gain, 1 = auto */
19     int awb_enable;          /* 0 = manual WB, 1 = auto */
20     int aec2_enable;         /* 0 = DSP AEC off, 1 = on */
21     int gain;                /* manual gain level -(030) */
22     int exposure;            /* manual exposure -(01200) */
23     gainceiling_t gain_ceiling; /* max gain multiplier */
24     int target_mean; /* auto-tune target brightness (0 = skip, use exposure
25                      as-is) */
26     int tune_tolerance; /* auto-tune stops when |mean - target| < tolerance
27                      */
28 } ov2640_cfg_t;
29
30 /* Locked defaults for diff-based compression pipeline.
31  * Gain floored, exposure auto-tuned to mean ~110. */
32 #define OV2640_LOCKED_DEFAULTS \
33     { \
34         .aec_enable = 0, \
35         .agc_enable = 0, \
36         .awb_enable = 0, \
37         .aec2_enable = 0, \
38         .gain = 0, \
39         .exposure = 50, \
40         .gain_ceiling = GAINCEILING_2X, \
41         .target_mean = 110, \
42         .tune_tolerance = 5, \
43     }
44
45 /**
46  * Apply sensor configuration.
47  * Disables auto controls, sets static parameters, and optionally
48  * auto-tunes exposure to hit target_mean brightness.
49  *
50  * Call after esp_camera_init().
51  */
52 esp_err_t ov2640_apply_config(const ov2640_cfg_t *cfg);
53
54 /**
55  * Read back current AEC value from sensor.
56  */
57 int ov2640_get_aec_value(void);
58
59 #endif /* OV2640_CONFIG_H */
```

esp/esp32cam/main/inc/sd_stg.h

```
1  /*
2   * sd_stg.h
3   * SD card storage interface for img logging
4   */
5
6  #ifndef SD_STORAGE_H
7  #define SD_STORAGE_H
8
9  #include <stddef.h>
10 #include <stdint.h>
11
12 #include "esp_err.h"
13
14 /* Debug flag - set to 1 to enable verbose logging */
15 #ifndef UNIT_TEST_SD_STORAGE
16 #define UNIT_TEST_SD_STORAGE 0
17 #endif
18
19 #if UNIT_TEST_SD_STORAGE
20 #define LOG_DEBUG(fmt, ...) ESP_LOGI(TAG, fmt, ##__VA_ARGS__)
21 #else
22 #define LOG_DEBUG(fmt, ...) \
23     do { \
24         } while (0)
25 #endif
26
27 /**
28  * Initialize SD card and mount filesystem
29  */
30 esp_err_t sd_card_init(void);
31
32 /**
33  * Unmount SD card and release resources
34  */
35 void sd_card_deinit(void);
36
37 /**
38  * Save img buffer to SD card as .jpg with auto filename
39  */
40 esp_err_t sd_save_img(const uint8_t *buffer, size_t length);
41
42 /**
43  * Save JPEG to SD with timestamped filename.
44  * epoch: UTC seconds from DevKit RTC.
45  * elapsed_us: esp_timer microseconds since boot at capture time.
46  * seq: sequence number within burst (0-based).
47  */
48 esp_err_t sd_save_timestamped(const uint8_t *jpg, size_t jpg_len,
49                               uint32_t epoch, int64_t elapsed_us, int seq);
50
51 /**
52  * Save raw buffer to SD card as .raw with auto filename
53  */
54 esp_err_t sd_save_raw(const uint8_t *buffer, size_t length);
55
56 /**
57  * Save img buffer to SD card with specific filename
58  */
59 esp_err_t sd_save_img_named(const uint8_t *buffer, size_t length,
60                             const char *filename);
61
62 /**
63  * Get current img counter value
64  */
65 int sd_get_img_counter(void);
66
67 /**
```

```
68 | * Reset img counter to zero
69 | */
70 | void sd_reset_img_counter(void);
71 |
72 | #endif /* SD_STORAGE_H */
```

esp/esp32cam/main/src/cam_ctrl.c

```
1  /*
2  * cam_ctrl.c
3  * ESP32-CAM camera initialization and frame capture
4  *
5  * Production mode:
6  *   - SPI pipeline: every grayscale frame pushed downstream at 30 fps
7  *   - Save ring: every SAVE_INTERVAL'th frame copied to PSRAM (no SD, no JPEG)
8  *   - Post-capture: after STOP command, JPEG encode + timestamped SD write
9  *
10 * Test burst mode (TEST_BURST_CAPTURE):
11 *   - No SPI - SD mounted for entire capture
12 *   - Dumps N raw grayscale frames to SD at max rate
13 */
14
15 #include "cam_ctrl.h"
16
17 #include <stdio.h>
18 #include <string.h>
19
20 #include "dma_spi.h"
21 #include "esp_camera.h"
22 #include "esp_err.h"
23 #include "esp_log.h"
24 #include "esp_sleep.h"
25 #include "esp_timer.h"
26 #include "freertos/FreeRTOS.h"
27 #include "freertos/task.h"
28 #include "img_buf.h"
29 #include "img_data.h"
30 #include "img_converters.h"
31 #include "ov2640_cfg.h"
32 #include "sd_stg.h"
33
34 static const char *TAG = "CAM_CTRL";
35
36 /*****
37  * Configuration
38  *****/
39
40 #define FRAME_PERIOD_MS      33 /* 30 fps target (1000 / 30) */
41 #define BURST_FRAME_COUNT    30
42 #define BURST_DISCARD_COUNT  5
43 #define JPEG_QUALITY          80
44
45 static const camera_config_t hw_config = {
46     .pin_pwdn = 32,
47     .pin_reset = -1,
48     .pin_xclk = 0,
49     .pin_sccb_sda = 26,
50     .pin_sccb_scl = 27,
51
52     .pin_d7 = 35,
53     .pin_d6 = 34,
54     .pin_d5 = 39,
55     .pin_d4 = 36,
56     .pin_d3 = 21,
57     .pin_d2 = 19,
58     .pin_d1 = 18,
59     .pin_d0 = 5,
60     .pin_vsync = 25,
61     .pin_href = 23,
62     .pin_pclk = 22,
63
64     .xclk_freq_hz = 20000000,
65     .ledc_timer = LEDC_TIMER_0,
66     .ledc_channel = LEDC_CHANNEL_0,
67 }
```

```

68     .pixel_format = PIXFORMAT_GRAYSCALE,
69     .frame_size = FRAMESIZE_QVGA, /* 320x240 for all modes */
70     .fb_count = 3,                /* triple buffer in PSRAM */
71     .fb_location = CAMERA_FB_IN_PSRAM,
72     .grab_mode = CAMERA_GRAB_LATEST, /* always freshest frame */
73 };
74
75 /*****
76  * File Scope State
77  *****/
78
79 static QueueHandle_t tx_queue = NULL;
80 static uint32_t frame_count = 0;
81
82 /*****
83  * SPI Queue Helper
84  *****/
85
86 static void enqueue_frame(const uint8_t *buf, size_t len)
87 {
88     uint8_t *tx = img_buffer_alloc();
89     if (!tx) {
90         ESP_LOGW(TAG, "No free buffers, dropping frame");
91         return;
92     }
93
94     size_t n = (len > IMAGE_BUFFER_SIZE) ? IMAGE_BUFFER_SIZE : len;
95     memcpy(tx, buf, n);
96
97     img_data_t img = {.buffer = tx, .size = n};
98
99     if (xQueueSend(tx_queue, &img, 0) != pdTRUE) {
100         ESP_LOGW(TAG, "SPI queue full, dropping frame");
101         img_buffer_free(tx);
102     }
103 }
104
105 /*****
106  * Post-Capture: batch JPEG encode + timestamped SD write
107  *****/
108
109 static void post_capture_save(void)
110 {
111     int n = save_ring_count();
112     if (n == 0) {
113         ESP_LOGI(TAG, "No frames in save ring");
114         return;
115     }
116
117     uint32_t epoch = spi_dma_epoch();
118
119     ESP_LOGI(TAG, "Post-capture: %d frames to encode + save", n);
120
121     /* SPI off - release GPIO 13 for SD */
122     spi_dma_deinit();
123
124     if (sd_card_init() != ESP_OK) {
125         ESP_LOGE(TAG, "SD mount failed - %d frames lost", n);
126         return;
127     }
128
129     for (int i = 0; i < n; i++) {
130         int64_t ts = 0;
131         const uint8_t *raw = save_ring_get(i, &ts);
132         if (!raw) continue;
133
134         uint8_t *jpg = NULL;
135         size_t jpg_len = 0;
136
137         if (!fmt2jpg((uint8_t *)raw, IMAGE_BUFFER_SIZE, 320, 240,

```

```

138         PIXFORMAT_GRAYSCALE, JPEG_QUALITY, &jpg,
139         &jpg_len)) {
140         ESP_LOGE(TAG, "JPEG encode failed, frame %d", i);
141         continue;
142     }
143
144     esp_err_t ret;
145     if (epoch > 0) {
146         ret = sd_save_timestamped(jpg, jpg_len, epoch, ts, i);
147     } else {
148         /* No epoch from DevKit - fallback filename */
149         char name[48];
150         snprintf(name, sizeof(name),
151                  "/sdcard/img_unknown_%03d.jpg", i);
152         ret = sd_save_img_named(jpg, jpg_len, name);
153     }
154
155     if (ret == ESP_OK) {
156         ESP_LOGI(TAG, " [%d/%d] %zu bytes (%.1f:1)", i + 1, n,
157                  jpg_len, (float)IMAGE_BUFFER_SIZE / jpg_len);
158     }
159
160     free(jpg);
161 }
162
163 sd_card_deinit();
164 save_ring_reset();
165
166 ESP_LOGI(TAG, "Post-capture save complete");
167 }
168
169 /*****
170  * Capture Tasks
171  *****/
172
173 #ifdef TEST_BURST_CAPTURE
174
175 static void burst_capture_task(void *arg)
176 {
177     ESP_LOGI(TAG, "=== BURST CAPTURE MODE ===");
178
179     /* Discard startup frames */
180     for (int i = 0; i < BURST_DISCARD_COUNT; i++) {
181         camera_fb_t *fb = esp_camera_fb_get();
182         if (fb) esp_camera_fb_return(fb);
183     }
184
185     ESP_LOGI(TAG, "Capturing %d raw frames...", BURST_FRAME_COUNT);
186     int64_t t0 = esp_timer_get_time();
187     int saved = 0, failed = 0;
188
189     for (int i = 0; i < BURST_FRAME_COUNT; i++) {
190         camera_fb_t *fb = esp_camera_fb_get();
191         if (!fb) {
192             failed++;
193             vTaskDelay(pdMS_TO_TICKS(10));
194             continue;
195         }
196
197         if (sd_save_raw(fb->buf, fb->len) == ESP_OK) saved++;
198         else failed++;
199
200         esp_camera_fb_return(fb);
201     }
202
203     float elapsed = (esp_timer_get_time() - t0) / 1e6f;
204
205     ESP_LOGI(TAG, "=== BURST COMPLETE ===");
206     ESP_LOGI(TAG, " %d saved, %d failed, %.2fs, %.1f fps", saved, failed,
207              elapsed, saved / elapsed);

```

```

208     ESP_LOGI(TAG, "Pull SD card and run analysis.");
209
210     for (;;) vTaskDelay(pdMS_TO_TICKS(10000));
211 }
212
213 #else
214
215 static void capture_loop(void *arg)
216 {
217     ESP_LOGI(TAG,
218         "Capture loop started - %d fps target, save every %d frames",
219         1000 / FRAME_PERIOD_MS, SAVE_INTERVAL);
220
221     TickType_t last_wake = xTaskGetTickCount();
222
223     for (;;) {
224         /* Check for STOP command from DevKit */
225         if (spi_dma_stop_requested()) {
226             ESP_LOGI(TAG, "STOP received - exiting capture loop");
227             break;
228         }
229
230         camera_fb_t *fb = esp_camera_fb_get();
231         if (!fb) {
232             vTaskDelayUntil(&last_wake,
233                 pdMS_TO_TICKS(FRAME_PERIOD_MS));
234             continue;
235         }
236
237         /* Every frame goes downstream over SPI */
238         if (tx_queue) enqueue_frame(fb->buf, fb->len);
239
240         /* Every SAVE_INTERVAL'th frame → PSRAM ring (no SD, no JPEG)
241          */
242         frame_count++;
243         if (frame_count % SAVE_INTERVAL == 0) {
244             save_ring_push(fb->buf, fb->len, esp_timer_get_time());
245         }
246
247         esp_camera_fb_return(fb);
248
249         vTaskDelayUntil(&last_wake, pdMS_TO_TICKS(FRAME_PERIOD_MS));
250     }
251
252     /* Post-capture phase: JPEG encode + SD save + sleep */
253     ESP_LOGI(TAG, "Entering post-capture phase");
254
255     post_capture_save();
256
257     ESP_LOGI(TAG, "Entering deep sleep");
258     esp_deep_sleep_start();
259 }
260
261 #endif /* TEST_BURST_CAPTURE */
262
263 /*****
264  * Global Interface
265  *****/
266
267 esp_err_t camera_init(void)
268 {
269     esp_err_t err = esp_camera_init(&hw_config);
270     if (err != ESP_OK) {
271         ESP_LOGE(TAG, "Camera init failed: 0x%x", err);
272         return err;
273     }
274
275     ESP_LOGI(TAG, "Camera initialized: 320x240 grayscale");
276
277     /* Sensor config - locked exposure, auto-tuned to scene brightness */

```

```
278         ov2640_cfg_t sensor_cfg = OV2640_LOCKED_DEFAULTS;
279         ov2640_apply_config(&sensor_cfg);
280
281         return ESP_OK;
282     }
283
284     void camera_set_tx_queue(QueueHandle_t queue) { tx_queue = queue; }
285
286     void camera_start_capture(void)
287     {
288         #ifdef TEST_BURST_CAPTURE
289             xTaskCreatePinnedToCore(burst_capture_task, "burst", 8192, NULL, 6,
290                                     NULL, 0);
291         #else
292             xTaskCreatePinnedToCore(capture_loop, "capture", 8192, NULL, 6, NULL,
293                                     0);
294         #endif
295     }
```


esp/esp32cam/main/src/dma_spi.c

```
1  /*
2  * dma_spi.c
3  * ESP32-CAM SPI DMA Master Implementation
4  *
5  * Sends img data over SPI using DMA for maximum throughput.
6  *
7  * Uses spi_device_transmit() (synchronous, calls get_trans_result internally)
8  * so the ESP-IDF driver never has "unfinished transactions" - required for
9  * spi_bus_remove_device / spi_bus_free to succeed during the SD card pin swap.
10 *
11 * bus_mutex serialises transfers and makes spi_dma_deinit() wait for any
12 * in-progress transfer to complete before tearing down the bus.
13 */
14
15 #include "dma_spi.h"
16
17 #include <string.h>
18
19 #include "driver/spi_master.h"
20 #include "esp_log.h"
21 #include "freertos/FreeRTOS.h"
22 #include "freertos/semphr.h"
23 #include "freertos/task.h"
24 #include "img_buf.h"
25
26 // Debug flag
27 #define UNIT_TEST_SPI 0
28
29 #if UNIT_TEST_SPI
30 #define LOG_DEBUG(fmt, ...) ESP_LOGI(TAG, fmt, ##__VA_ARGS__)
31 #else
32 #define LOG_DEBUG(fmt, ...) \
33     do { \
34         } while (0)
35 #endif
36
37 #define SPI_MASTER_HOST HSPI_HOST
38 #define DMA_CHANNEL SPI_DMA_CH_AUTO
39
40 static const char *TAG = "SPI_TX";
41
42 typedef struct {
43     spi_device_handle_t spi;
44     /* held for full duration of each transfer */
45     SemaphoreHandle_t bus_mutex;
46     bool initialized;
47     uint8_t rx_buf[12];
48     uint32_t last_cmd;
49     uint32_t epoch;
50     bool stop_flag;
51 } spi_dma_context_t;
52
53 static spi_dma_context_t ctx = {.initialized = false};
54
55 esp_err_t spi_dma_init(void)
56 {
57     if (ctx.initialized) {
58         ESP_LOGW(TAG, "SPI DMA already initialized");
59         return ESP_OK;
60     }
61
62     ctx.bus_mutex = xSemaphoreCreateMutex();
63     if (ctx.bus_mutex == NULL) {
64         ESP_LOGE(TAG, "Failed to create bus_mutex");
65         return ESP_FAIL;
66     }
67 }
```

```

68     spi_bus_config_t bus_cfg = {
69         .mosi_io_num = PIN_NUM_MOSI,
70         .miso_io_num = PIN_NUM_MISO,
71         .sclk_io_num = PIN_NUM_CLK,
72         .quadwp_io_num = -1,
73         .quadhd_io_num = -1,
74         .max_transfer_sz = MAX_TRANSFER_SIZE,
75         .flags = SPICOMMON_BUSFLAG_MASTER,
76     };
77
78     esp_err_t ret =
79         spi_bus_initialize(SPI_MASTER_HOST, &bus_cfg, DMA_CHANNEL);
80     if (ret != ESP_OK) {
81         ESP_LOGE(TAG, "Failed to initialize SPI bus: %s",
82             esp_err_to_name(ret));
83         vSemaphoreDelete(ctx.bus_mutex);
84         ctx.bus_mutex = NULL;
85         return ret;
86     }
87
88     spi_device_interface_config_t dev_cfg = {
89         .clock_speed_hz = SPI_CLOCK_SPEED,
90         .mode = 0,
91         .spics_io_num = PIN_NUM_CS,
92         .queue_size = 1,
93         .flags = 0,
94     };
95
96     ret = spi_bus_add_device(SPI_MASTER_HOST, &dev_cfg, &ctx.spi);
97     if (ret != ESP_OK) {
98         ESP_LOGE(TAG, "Failed to add SPI device: %s",
99             esp_err_to_name(ret));
100         spi_bus_free(SPI_MASTER_HOST);
101         vSemaphoreDelete(ctx.bus_mutex);
102         ctx.bus_mutex = NULL;
103         return ret;
104     }
105
106     ctx.initialized = true;
107     ESP_LOGI(TAG, "SPI DMA initialized: %d MHz",
108         SPI_CLOCK_SPEED / 1000000);
109     return ESP_OK;
110 }
111
112 void spi_dma_deinit(void)
113 {
114     if (!ctx.initialized) return;
115
116     /* Block until any in-progress spi_device_transmit() finishes */
117     xSemaphoreTake(ctx.bus_mutex, portMAX_DELAY);
118
119     /* All transactions are complete (spi_device_transmit calls
120      * get_trans_result internally), so remove_device and bus_free are safe
121      * to call now. */
122     spi_bus_remove_device(ctx.spi);
123     spi_bus_free(SPI_MASTER_HOST);
124
125     ctx.initialized = false;
126
127     xSemaphoreGive(ctx.bus_mutex);
128     vSemaphoreDelete(ctx.bus_mutex);
129     ctx.bus_mutex = NULL;
130
131     ESP_LOGI(TAG, "SPI DMA deinitialized");
132 }
133
134 static void check_miso_cmd(const uint8_t *rx)
135 {
136     uint32_t magic;
137     memcpy(&magic, rx, 4);

```

```

138     if (magic != 0xDEADBEEF) return;
139
140     uint32_t cmd;
141     memcpy(&cmd, rx + 4, 4);
142
143     if (cmd == CMD_START) memcpy(&ctx.epoch, rx + 8, 4);
144     else if (cmd == CMD_STOP) ctx.stop_flag = true;
145 }
146
147 static esp_err_t spi_dma_transmit(const uint8_t *data, size_t length,
148                                   uint8_t *rx, size_t rx_len)
149 {
150     if (data == NULL || length == 0 || length > MAX_TRANSFER_SIZE)
151         return ESP_ERR_INVALID_ARG;
152
153     spi_transaction_t trans = {
154         .length = length * 8,
155         .tx_buffer = data,
156         .rx_buffer = rx,
157         .rxlength = rx ? rx_len * 8 : 0,
158     };
159
160     return spi_device_transmit(ctx.spi, &trans);
161 }
162
163 esp_err_t spi_dma_send_img(const uint8_t *img_data, size_t img_size)
164 {
165     if (!ctx.initialized || !ctx.bus_mutex) return ESP_ERR_INVALID_STATE;
166
167     if (xSemaphoreTake(ctx.bus_mutex, pdMS_TO_TICKS(500)) != pdTRUE) {
168         ESP_LOGW(TAG, "Bus mutex timeout - SPI busy");
169         return ESP_ERR_TIMEOUT;
170     }
171
172     /* Re-check under mutex: deinit may have run between the check above
173      * and lock */
174     if (!ctx.initialized) {
175         xSemaphoreGive(ctx.bus_mutex);
176         return ESP_ERR_INVALID_STATE;
177     }
178
179     struct {
180         uint32_t magic;
181         uint32_t size;
182         uint32_t checksum;
183         uint32_t timestamp;
184         uint16_t width;
185         uint16_t height;
186         uint8_t format;
187         uint8_t reserved[7];
188     } __attribute__((packed)) header = {0};
189
190     header.magic = 0xCAFEDEED;
191     header.size = img_size;
192     header.width = 320;
193     header.height = 240;
194
195     header.checksum = 0;
196     for (size_t i = 0; i < img_size; i++) header.checksum ^= img_data[i];
197
198     esp_err_t ret = spi_dma_transmit((uint8_t *)&header, sizeof(header),
199                                     ctx.rx_buf, sizeof(ctx.rx_buf));
200     if (ret != ESP_OK) {
201         xSemaphoreGive(ctx.bus_mutex);
202         return ret;
203     }
204
205     size_t offset = 0;
206     while (offset < img_size) {
207         size_t chunk = (img_size - offset) > MAX_TRANSFER_SIZE

```

```

208             ? MAX_TRANSFER_SIZE
209             : (img_size - offset);
210
211     ret = spi_dma_transmit(img_data + offset, chunk, NULL, 0);
212     if (ret != ESP_OK) {
213         xSemaphoreGive(ctx.bus_mutex);
214         return ret;
215     }
216     offset += chunk;
217 }
218
219 LOG_DEBUG("Sent %d bytes via SPI", img_size);
220
221 xSemaphoreGive(ctx.bus_mutex);
222 return ESP_OK;
223 }
224
225 void spi_transmit_task(void *pvParameters)
226 {
227     QueueHandle_t img_queue = (QueueHandle_t)pvParameters;
228     img_data_t img_data;
229
230     ESP_LOGI(TAG, "SPI transmit task started");
231     vTaskDelay(pdMS_TO_TICKS(1000));
232
233     for (;;) {
234         if (xQueueReceive(img_queue, &img_data, portMAX_DELAY) ==
235             pdTRUE) {
236             esp_err_t ret =
237                 spi_dma_send_img(img_data.buffer, img_data.size);
238             check_miso_cmd(ctx.rx_buf);
239
240             if (ret != ESP_OK)
241                 ESP_LOGE(TAG, "SPI transmission failed: %s",
242                     esp_err_to_name(ret));
243             else LOG_DEBUG("SPI transmission successful");
244
245             img_buffer_free(img_data.buffer);
246
247 #if UNIT_TEST_SPI
248             int total, in_use, peak;
249             img_buffer_get_stats(&total, &in_use, &peak);
250             ESP_LOGI(TAG, "Buffer pool: %d/%d in use (peak: %d)",
251                 in_use, total, peak);
252 #endif
253         }
254     }
255 }
256
257 bool spi_dma_stop_requested(void) { return ctx.stop_flag; }
258
259 uint32_t spi_dma_epoch(void) { return ctx.epoch; }

```

esp/esp32cam/main/src/img_buf.c

```
1  /*
2  * img_buf.c
3  * PSRAM memory pool for img buffers
4  * Thread-safe allocation/deallocation with usage tracking
5  */
6
7  #include "img_buf.h"
8
9  #include <string.h>
10
11 #include "esp_heap_caps.h"
12 #include "esp_log.h"
13 #include "freertos/FreeRTOS.h"
14 #include "freertos/semphr.h"
15
16 static const char *TAG = "IMG_POOL";
17
18 /* Debug flag - set to 1 to enable verbose logging */
19 #define UNIT_TEST_BUFF_POOL 0
20
21 #if UNIT_TEST_BUFF_POOL
22 #define LOG_DEBUG(fmt, ...) ESP_LOGI(TAG, fmt, ##__VA_ARGS__)
23 #else
24 #define LOG_DEBUG(fmt, ...) \
25     do { \
26         } while (0)
27 #endif
28
29 /*
30     *****
31
32     File Scope Variables
33     *****
34     */
35
36 /* Buffer pool structure */
37 static struct {
38     uint8_t *buffers[IMAGE_BUFFER_COUNT];
39     bool in_use[IMAGE_BUFFER_COUNT];
40     SemaphoreHandle_t mutex;
41     int peak_usage;
42     bool initialized;
43 } pool = {0};
44
45 /*
46     *****
47
48     Global Interface
49     *****
50     */
51
52 esp_err_t img_buf_init(void)
53 {
54     /* Redundancy check */
55     if (pool.initialized) {
56         ESP_LOGW(TAG, "Buffer pool already initialized");
57         return ESP_OK;
58     }
59
60     LOG_DEBUG("Initializing buffer pool in PSRAM...");
61
62     /* Create mutex for thread-safe access */
63     pool.mutex = xSemaphoreCreateMutex();
64     if (pool.mutex == NULL) {
65         ESP_LOGE(TAG, "Failed to create mutex");
66         return ESP_FAIL;
67     }
68 }
```

```

62
63 /* Allocate buffers in PSRAM (external RAM) */
64 for (int i = 0; i < IMAGE_BUFFER_COUNT; i++) {
65     pool.buffers[i] =
66         heap_caps_malloc(IMAGE_BUFFER_SIZE, MALLOC_CAP_SPIRAM);
67
68     if (pool.buffers[i] == NULL) {
69         ESP_LOGE(TAG, "Failed to allocate buffer %d in PSRAM",
70             i);
71
72         /* Clean up any allocated buffers */
73         for (int j = 0; j < i; j++) {
74             heap_caps_free(pool.buffers[j]);
75         }
76
77         vSemaphoreDelete(pool.mutex);
78         return ESP_ERR_NO_MEM;
79     }
80
81     pool.in_use[i] = false;
82     LOG_DEBUG("Buffer %d allocated at %p", i, pool.buffers[i]);
83 }
84
85 pool.peak_usage = 0;
86 pool.initialized = true;
87
88 ESP_LOGI(
89     TAG,
90     "Buffer pool initialized: %d buffers x %d bytes = %d KB in PSRAM",
91     IMAGE_BUFFER_COUNT, IMAGE_BUFFER_SIZE,
92     (IMAGE_BUFFER_COUNT * IMAGE_BUFFER_SIZE) / 1024);
93
94 return ESP_OK;
95 }
96
97 uint8_t *img_buffer_alloc(void)
98 {
99     if (!pool.initialized) {
100         ESP_LOGE(TAG, "Buffer pool not initialized!");
101         return NULL;
102     }
103
104     xSemaphoreTake(pool.mutex, portMAX_DELAY);
105
106     /* Find free buffer */
107     for (int i = 0; i < IMAGE_BUFFER_COUNT; i++) {
108         if (!pool.in_use[i]) {
109             pool.in_use[i] = true;
110
111             /* Update peak usage stats */
112             int current_usage = 0;
113             for (int j = 0; j < IMAGE_BUFFER_COUNT; j++) {
114                 if (pool.in_use[j]) current_usage++;
115             }
116             if (current_usage > pool.peak_usage) {
117                 pool.peak_usage = current_usage;
118             }
119
120             xSemaphoreGive(pool.mutex);
121
122             LOG_DEBUG("Allocated buffer %d (%d/%d in use)", i,
123                 current_usage, IMAGE_BUFFER_COUNT);
124
125             return pool.buffers[i];
126         }
127     }
128
129     xSemaphoreGive(pool.mutex);
130
131     ESP_LOGW(TAG, "No free buffers available! All %d buffers in use",

```

```

132         IMAGE_BUFFER_COUNT);
133
134     return NULL;
135 }
136
137 void img_buffer_free(uint8_t *buffer)
138 {
139     if (!pool.initialized) {
140         ESP_LOGE(TAG, "Buffer pool not initialized!");
141         return;
142     }
143
144     if (buffer == NULL) {
145         ESP_LOGW(TAG, "Attempted to free NULL buffer");
146         return;
147     }
148
149     xSemaphoreTake(pool.mutex, portMAX_DELAY);
150
151     /* Find buffer in pool */
152     for (int i = 0; i < IMAGE_BUFFER_COUNT; i++) {
153         if (pool.buffers[i] == buffer) {
154             if (!pool.in_use[i]) {
155                 ESP_LOGW(TAG, "Buffer %d was already free!",
156                     i);
157             }
158
159             pool.in_use[i] = false;
160             xSemaphoreGive(pool.mutex);
161
162             LOG_DEBUG("Freed buffer %d", i);
163             return;
164         }
165     }
166
167     xSemaphoreGive(pool.mutex);
168
169     ESP_LOGE(TAG, "Tried to free unknown buffer at %p!", buffer);
170 }
171
172 void img_buffer_get_stats(int *total, int *in_use, int *peak)
173 {
174     if (!pool.initialized) {
175         if (total) *total = 0;
176         if (in_use) *in_use = 0;
177         if (peak) *peak = 0;
178         return;
179     }
180
181     xSemaphoreTake(pool.mutex, portMAX_DELAY);
182
183     if (total) *total = IMAGE_BUFFER_COUNT;
184
185     if (in_use) {
186         int count = 0;
187         for (int i = 0; i < IMAGE_BUFFER_COUNT; i++) {
188             if (pool.in_use[i]) count++;
189         }
190         *in_use = count;
191     }
192
193     if (peak) *peak = pool.peak_usage;
194
195     xSemaphoreGive(pool.mutex);
196 }
197
198 /*
199     *****
200
201     Save Ring Buffer

```

```

200 *****
    */
201
202 static struct {
203     uint8_t *slots[SAVE_RING_SIZE];
204     int64_t timestamps[SAVE_RING_SIZE];
205     int head;
206     int count;
207     bool initialized;
208 } ring = {0};
209
210 esp_err_t save_ring_init(void)
211 {
212     if (ring.initialized) {
213         ESP_LOGW(TAG, "Save ring already initialized");
214         return ESP_OK;
215     }
216
217     for (int i = 0; i < SAVE_RING_SIZE; i++) {
218         ring.slots[i] =
219             heap_caps_malloc(IMAGE_BUFFER_SIZE, MALLOC_CAP_SPIRAM);
220         if (ring.slots[i] == NULL) {
221             ESP_LOGE(TAG, "Save ring alloc failed at slot %d", i);
222             for (int j = 0; j < i; j++)
223                 heap_caps_free(ring.slots[j]);
224             return ESP_ERR_NO_MEM;
225         }
226     }
227
228     ring.head = 0;
229     ring.count = 0;
230     ring.initialized = true;
231
232     ESP_LOGI(TAG, "Save ring: %d slots x %d bytes = %d KB in PSRAM",
233             SAVE_RING_SIZE, IMAGE_BUFFER_SIZE,
234             (SAVE_RING_SIZE * IMAGE_BUFFER_SIZE) / 1024);
235
236     return ESP_OK;
237 }
238
239 void save_ring_push(const uint8_t *frame, size_t len, int64_t timestamp)
240 {
241     if (!ring.initialized) return;
242
243     size_t n = (len > IMAGE_BUFFER_SIZE) ? IMAGE_BUFFER_SIZE : len;
244     memcpy(ring.slots[ring.head], frame, n);
245     ring.timestamps[ring.head] = timestamp;
246
247     ring.head = (ring.head + 1) % SAVE_RING_SIZE;
248     if (ring.count < SAVE_RING_SIZE) ring.count++;
249 }
250
251 const uint8_t *save_ring_get(int idx, int64_t *timestamp_out)
252 {
253     if (!ring.initialized || idx < 0 || idx >= ring.count) return NULL;
254
255     /* idx 0 = oldest */
256     int pos;
257     if (ring.count < SAVE_RING_SIZE) pos = idx;
258     else pos = (ring.head + idx) % SAVE_RING_SIZE;
259
260     if (timestamp_out) *timestamp_out = ring.timestamps[pos];
261     return ring.slots[pos];
262 }
263
264 int save_ring_count(void) { return ring.count; }
265
266 void save_ring_reset(void)
267 {
268     ring.head = 0;

```



```

269     ring.count = 0;
270 }
271
272 /*
    *****
273     Testing
274     *****
    */
275
276 #ifdef UNIT_TEST_BUFF_POOL
277 void img_buf_unit_test(void)
278 {
279     ESP_LOGI(TAG, "=== Buffer Pool Unit Test ===");
280
281     if (!pool.initialized) {
282         ESP_LOGE(TAG, "Pool not initialized!");
283         return;
284     }
285
286     /* Test 1: Allocate all buffers */
287     ESP_LOGI(TAG, "Test 1: Allocating all buffers...");
288     uint8_t *bufs[IMAGE_BUFFER_COUNT];
289     for (int i = 0; i < IMAGE_BUFFER_COUNT; i++) {
290         bufs[i] = img_buffer_alloc();
291         if (bufs[i] == NULL) {
292             ESP_LOGE(TAG, "Failed to allocate buffer %d", i);
293             return;
294         }
295         ESP_LOGI(TAG, " Buffer %d: %p", i, bufs[i]);
296     }
297
298     /* Test 2: Try to allocate one more (should fail) */
299     ESP_LOGI(TAG, "Test 2: Attempting overflow allocation...");
300     uint8_t *overflow = img_buffer_alloc();
301     if (overflow == NULL) {
302         ESP_LOGI(TAG,
303             " Correctly returned NULL (all buffers in use)");
304     } else {
305         ESP_LOGE(TAG, " ERROR: Allocated beyond pool size!");
306     }
307
308     /* Test 3: Free all buffers */
309     ESP_LOGI(TAG, "Test 3: Freeing all buffers...");
310     for (int i = 0; i < IMAGE_BUFFER_COUNT; i++) {
311         img_buffer_free(bufs[i]);
312         ESP_LOGI(TAG, " Freed buffer %d", i);
313     }
314
315     /* Test 4: Check stats */
316     int total, in_use, peak;
317     img_buffer_get_stats(&total, &in_use, &peak);
318     ESP_LOGI(TAG, "Test 4: Stats - Total: %d, In use: %d, Peak: %d", total,
319         in_use, peak);
320
321     if (in_use == 0 && peak == IMAGE_BUFFER_COUNT) {
322         ESP_LOGI(TAG, " All tests PASSED");
323     } else {
324         ESP_LOGE(TAG,
325             " Test FAILED: in_use=%d (expected 0), peak=%d "
326             "(expected %d)",
327             in_use, IMAGE_BUFFER_COUNT, IMAGE_BUFFER_COUNT);
328     }
329
330     ESP_LOGI(TAG, "=== Unit Test Complete ===");
331 }
332 #endif

```

esp/esp32cam/main/src/main.c

```
1  /*
2  * ESP32-CAM Main Application
3  *
4  * Production mode:
5  *   Capture → SPI DMA to DevKit (every frame, 30 fps)
6  *   →          PSRAM save ring (every 10th frame, raw)
7  *   On STOP → JPEG batch encode → timestamped SD write → deep sleep
8  *
9  * Test burst mode (define TEST_BURST_CAPTURE):
10 *   Camera capture → raw frames to SD at max rate
11 *   No SPI, no buffer pool – SD stays mounted
12 */
13
14 #include "dma_spi.h"
15 #include "cam_ctrl.h"
16 #include "esp_log.h"
17 #include "freertos/FreeRTOS.h"
18 #include "freertos/queue.h"
19 #include "img_buf.h"
20 #include "img_data.h"
21 #include "sd_stg.h"
22
23 static const char *TAG = "MAIN";
24
25 #define IMAGE_QUEUE_DEPTH 2
26
27 void app_main(void)
28 {
29     ESP_LOGI(TAG, "ESP32-CAM Starting...");
30
31     #ifdef TEST_BURST_CAPTURE
32
33         ESP_LOGI(TAG, "*** TEST BURST CAPTURE MODE ***");
34
35         /* Mount SD once – stays mounted for entire burst */
36         if (sd_card_init() != ESP_OK) {
37             ESP_LOGE(TAG, "SD card init failed – halting");
38             return;
39         }
40
41         /* Camera with locked exposure/gain */
42         if (camera_init() != ESP_OK) {
43             ESP_LOGE(TAG, "Camera init failed – halting");
44             return;
45         }
46
47         /* Fire – burst_capture_task handles everything */
48         camera_start_capture();
49
50         ESP_LOGI(TAG, "Burst capture running");
51
52     #else
53
54         /* PSRAM buffer pool for SPI DMA transfers */
55         if (img_buf_init() != ESP_OK) {
56             ESP_LOGE(TAG, "Buffer pool init failed – halting");
57             return;
58         }
59
60
61         /* PSRAM save ring for post-capture JPEG batch */
62         if (save_ring_init() != ESP_OK) {
63             ESP_LOGE(TAG, "Save ring init failed – halting");
64             return;
65         }
66
67     #endif
```

```

68     /* SPI DMA master - GPIO 13 shared with SD, no conflict:
69     * SD only accessed in post-capture phase when SPI is idle */
70     if (spi_dma_init() != ESP_OK) {
71         ESP_LOGE(TAG, "SPI DMA init failed - halting");
72         return;
73     }
74
75
76     /* Camera - 320x240 grayscale, locked exposure */
77     if (camera_init() != ESP_OK) {
78         ESP_LOGE(TAG, "Camera init failed - halting");
79         return;
80     }
81
82
83     /* Image queue: capture task → SPI transmit task */
84     QueueHandle_t img_queue =
85         xQueueCreate(IMAGE_QUEUE_DEPTH, sizeof(img_data_t));
86     if (img_queue == NULL) {
87         ESP_LOGE(TAG, "Queue creation failed - halting");
88         return;
89     }
90
91     camera_set_tx_queue(img_queue);
92
93     /* SPI transmit task - dequeues frames, sends via DMA */
94     xTaskCreate(spi_transmit_task, "spi_tx", 4096, (void *)img_queue, 5,
95         NULL);
96
97     /* Capture loop - every frame to SPI, 1-in-10 to PSRAM save ring.
98     * On STOP command: JPEG encode batch → SD write → deep sleep */
99     camera_start_capture();
100
101     ESP_LOGI(TAG, "Pipeline running");
102
103 #endif
104 }

```

esp/esp32cam/main/src/ov2640_cfg.c

```
1  /*
2  * ov2640_cfg.c
3  * OV2640 sensor configuration implementation
4  *
5  * All interaction with the Espressif sensor_t API lives here.
6  * The rest of the codebase uses ov2640_cfg_t structs.
7  */
8
9  #include "ov2640_cfg.h"
10
11  #include <stdlib.h>
12
13  #include "esp_log.h"
14  #include "freertos/FreeRTOS.h"
15  #include "freertos/task.h"
16
17  static const char *TAG = "OV2640";
18
19  /*
20     *****
21     File Scope
22     *****
23     */
24
25  static sensor_t *get_sensor(void)
26  {
27      sensor_t *s = esp_camera_sensor_get();
28      if (!s) {
29          ESP_LOGE(
30              TAG,
31              "sensor handle is NULL - was esp_camera_init() called?");
32      }
33      return s;
34  }
35
36  static int compute_frame_mean(const camera_fb_t *fb)
37  {
38      uint32_t sum = 0;
39      for (size_t i = 0; i < fb->len; i++) { sum += fb->buf[i]; }
40      return (int)(sum / fb->len);
41  }
42
43  /**
44   * Proportional exposure auto-tune.
45   * Captures frames, measures mean brightness, adjusts aec_value
46   * until within tolerance of target. Converges in ~35 iterations.
47   */
48  static esp_err_t tune_exposure(sensor_t *s, int target, int tolerance,
49                                int initial_aec)
50  {
51      int aec = initial_aec;
52
53      for (int i = 0; i < 20; i++) {
54          s->set_aec_value(s, aec);
55          vTaskDelay(pdMS_TO_TICKS(100));
56
57          camera_fb_t *fb = esp_camera_fb_get();
58          if (!fb) continue;
59
60          int mean = compute_frame_mean(fb);
61          esp_camera_fb_return(fb);
62
63          ESP_LOGI(TAG, "tune [%2d]: aec=%-4d  mean=%-3d  target=%d", i,
64                   aec, mean, target);
65
66          if (abs(mean - target) < tolerance) {
```

```

65         ESP_LOGI(TAG, "converged: aec=%d mean=%d", aec, mean);
66         return ESP_OK;
67     }
68
69     aec = aec * target / (mean > 0 ? mean : 1);
70     if (aec < 1) aec = 1;
71     if (aec > 1200) aec = 1200;
72 }
73
74 ESP_LOGW(TAG, "auto-tune did not converge (aec=%d)", aec);
75 return ESP_OK; /* still usable, just not perfect */
76 }
77
78 /*
79  ****
80
81     Global Interface
82 ****
83 */
84
85 esp_err_t ov2640_apply_config(const ov2640_cfg_t *cfg)
86 {
87     sensor_t *s = get_sensor();
88     if (!s) return ESP_FAIL;
89
90     /* Disable auto controls first - order matters on OV2640 */
91     s->set_whitebal(s, cfg->awb_enable);
92     s->set_aec2(s, cfg->aec2_enable);
93     s->set_gain_ctrl(s, cfg->agc_enable);
94     s->set_exposure_ctrl(s, cfg->aec_enable);
95
96     /* Static parameters */
97     s->set_agc_gain(s, cfg->gain);
98     s->set_gainceiling(s, cfg->gain_ceiling);
99
100     /* Exposure: either fixed or auto-tuned */
101     if (cfg->target_mean > 0) {
102         tune_exposure(s, cfg->target_mean, cfg->tune_tolerance,
103             cfg->exposure);
104     } else {
105         s->set_aec_value(s, cfg->exposure);
106     }
107
108     ESP_LOGI(TAG, "config applied: gain=%d agc=%d aec=%d awb=%d",
109         cfg->gain, cfg->agc_enable, cfg->aec_enable, cfg->awb_enable);
110
111     return ESP_OK;
112 }
113
114 int ov2640_get_aec_value(void)
115 {
116     sensor_t *s = get_sensor();
117     if (!s) return -1;
118     return s->status.aec_value;
119 }

```

esp/esp32cam/main/src/sd_stg.c

```
1  /*
2  * sd_stg.c
3  * SD card storage implementation for img logging
4  */
5
6  #include "sd_stg.h"
7
8  #include <stdio.h>
9  #include <string.h>
10
11 #include "driver/sdmmc_host.h"
12 #include "esp_log.h"
13 #include "esp_vfs_fat.h"
14 #include "sdmmc_cmd.h"
15
16 static const char *TAG = "SD_STORAGE";
17
18 /*
19  *****
20
21  File Scope Variables
22  *****
23  */
24
25 static sdmmc_card_t *card = NULL;
26 static int img_counter = 0;
27 static bool initialized = false;
28
29 /*
30  *****
31
32  Global Interface
33  *****
34  */
35
36 esp_err_t sd_card_init(void)
37 {
38     if (initialized) {
39         ESP_LOGW(TAG, "SD card already initialized");
40         return ESP_OK;
41     }
42
43     LOG_DEBUG("Initializing SD card...");
44
45     esp_vfs_fat_sdmmc_mount_config_t mount_config = {
46         .format_if_mount_failed = false,
47         .max_files = 5,
48         .allocation_unit_size = 16 * 1024};
49
50     sdmmc_host_t host = SDMMC_HOST_DEFAULT();
51     host.max_freq_khz = SDMMC_FREQ_HIGHSPEED; /* 40 MHz */
52
53     sdmmc_slot_config_t slot_config = SDMMC_SLOT_CONFIG_DEFAULT();
54     slot_config.width = 4; /* 4-bit mode */
55
56     esp_err_t ret = esp_vfs_fat_sdmmc_mount("/sdcard", &host, &slot_config,
57         &mount_config, &card);
58
59     if (ret != ESP_OK) {
60         if (ret == ESP_FAIL) {
61             ESP_LOGE(TAG, "Failed to mount filesystem. Insert SD "
62                 "card and reset.");
63         } else {
64             ESP_LOGE(TAG, "Failed to initialize SD card: %s",
65                 esp_err_to_name(ret));
66         }
67     }
68     return ret;
69 }
```

```

62     }
63
64 #if UNIT_TEST_SD_STORAGE
65     sdmmc_card_print_info(stdout, card);
66 #endif
67
68     initialized = true;
69     ESP_LOGI(TAG, "SD card mounted successfully");
70
71     return ESP_OK;
72 }
73
74 void sd_card_deinit(void)
75 {
76     if (!initialized) { return; }
77
78     esp_vfs_fat_sdcard_unmount("/sdcard", card);
79     card = NULL;
80     initialized = false;
81     ESP_LOGI(TAG, "SD card unmounted");
82 }
83
84 esp_err_t sd_save_img(const uint8_t *buffer, size_t length)
85 {
86     if (!initialized) {
87         ESP_LOGE(TAG, "SD card not initialized!");
88         return ESP_FAIL;
89     }
90
91     char filename[32];
92     snprintf(filename, sizeof(filename), "/sdcard/img_%04d.jpg",
93              img_counter++);
94
95     return sd_save_img_named(buffer, length, filename);
96 }
97
98 esp_err_t sd_save_raw(const uint8_t *buffer, size_t length)
99 {
100     if (!initialized) {
101         ESP_LOGE(TAG, "SD card not initialized!");
102         return ESP_FAIL;
103     }
104
105     char filename[32];
106     snprintf(filename, sizeof(filename), "/sdcard/img_%04d.raw",
107              img_counter++);
108
109     return sd_save_img_named(buffer, length, filename);
110 }
111
112 esp_err_t sd_save_img_named(const uint8_t *buffer, size_t length,
113                             const char *filename)
114 {
115     if (!initialized) {
116         ESP_LOGE(TAG, "SD card not initialized!");
117         return ESP_FAIL;
118     }
119
120     if (buffer == NULL || filename == NULL) {
121         ESP_LOGE(TAG, "Invalid parameters");
122         return ESP_ERR_INVALID_ARG;
123     }
124
125     FILE *f = fopen(filename, "wb");
126     if (f == NULL) {
127         ESP_LOGE(TAG, "Failed to open file: %s", filename);
128         return ESP_FAIL;
129     }
130
131     size_t written = fwrite(buffer, 1, length, f);

```

```

132     fclose(f);
133
134     if (written != length) {
135         ESP_LOGE(TAG, "Write failed: %zu/%zu bytes", written, length);
136         return ESP_ERR_INVALID_SIZE;
137     }
138
139     LOG_DEBUG("Saved: %s (%zu bytes)", filename, written);
140     return ESP_OK;
141 }
142
143 esp_err_t sd_save_timestamped(const uint8_t *jpg, size_t jpg_len,
144                               uint32_t epoch, int64_t elapsed_us, int seq)
145 {
146     if (!initialized) {
147         ESP_LOGE(TAG, "SD card not initialized!");
148         return ESP_FAIL;
149     }
150
151     uint32_t abs_sec = epoch + (uint32_t)(elapsed_us / 1000000);
152
153     /* Break epoch into date/time components.
154      * Minimal implementation - no timezone, no leap second. */
155     uint32_t s = abs_sec;
156     int sec = s % 60;
157     s /= 60;
158     int min = s % 60;
159     s /= 60;
160     int hour = s % 24;
161     s /= 24;
162
163     /* Days since 1970-01-01 */
164     int days = (int)s;
165     int year = 1970;
166     while (1) {
167         int yd =
168             (year % 4 == 0 && (year % 100 != 0 || year % 400 == 0))
169             ? 366
170             : 365;
171         if (days < yd) break;
172         days -= yd;
173         year++;
174     }
175     int leap = (year % 4 == 0 && (year % 100 != 0 || year % 400 == 0));
176     int mdays[] = {31, 28 + leap, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
177     int month = 0;
178     while (month < 11 && days >= mdays[month]) {
179         days -= mdays[month];
180         month++;
181     }
182     int day = days + 1;
183     month += 1;
184
185     char filename[64];
186     snprintf(filename, sizeof(filename),
187              "/sdcard/img_%04d%02d%02d_%02d%02d%02d_%03d.jpg", year, month,
188              day, hour, min, sec, seq);
189
190     ESP_LOGI(TAG, "Saving %s (%zu bytes)", filename, jpg_len);
191     return sd_save_img_named(jpg, jpg_len, filename);
192 }
193
194 int sd_get_img_counter(void) { return img_counter; }
195
196 void sd_reset_img_counter(void)
197 {
198     img_counter = 0;
199     ESP_LOGI(TAG, "Image counter reset to 0");
200 }

```


esp/esp32s3_wroom/main/inc/config.h

```
1 #pragma once
2
3 /*****
4  * WiFi AP Configuration
5  *****/
6 #define WIFI_AP_SSID      "TrailCamera"
7 #define WIFI_AP_PASS      "wildlife1"      /* min 8 chars for WPA2 */
8 #define WIFI_AP_CHANNEL   6
9 #define WIFI_AP_MAX_CONN  4
10
11 /*****
12  * WebSocket Server
13  *****/
14 #define WS_PORT           80
15 #define WS_PATH            "/stream"
16 #define WS_MAX_CLIENTS    4
17
18 /*****
19  * Bench Test Mode
20  *
21  * Define TEST_MODE_LORA_BENCH on BOTH boards to run the link bench.
22  * BENCH_SF and BENCH_BW MUST match the DevKitV1 config.h values.
23  * In bench mode: WiFi/WebSocket disabled; LoRa packets are echoed back.
24  *****/
25 // #define TEST_MODE_LORA_BENCH
26 // #define TEST_MODE_WS_PATTERNS
27
28 #ifndef TEST_MODE_LORA_BENCH
29     #warning "LoRa bench mode -- S3 is echo slave, WiFi/WebSocket disabled"
30 #endif
31
32 /*****
33  * LoRa RF Parameters (must match DevKitV1)
34  *****/
35 #define LORA_SF           7      /* spreading factor (7-12) */
36 #define LORA_BW           9      /* 7=125kHz 8=250kHz 9=500kHz */
37 #define LORA_CR           1      /* coding rate: 1=4/5 */
38 #define LORA_PREAMBLE     12
39
40 /*****
41  * LoRa UART
42  *****/
43 #define LORA_UART_NUM      UART_NUM_2
44 #define LORA_PIN_TX        GPIO_NUM_17
45 #define LORA_PIN_RX        GPIO_NUM_18
46 #define LORA_BAUD_RATE     115200
47 #define LORA_BUF_SIZE      1024
48 #define LORA_ADDRESS_RECEIVER 1      /* this receiver */
49 #define LORA_NETWORK_ID    6
50 #define LORA_AT_TIMEOUT_MS  500
51 #define LORA_INIT_DELAY_MS  300
52 #define LORA_MAX_PAYLOAD    240      /* RYLR998 max payload */
53
54 /*****
55  * Image / Frame Constants (must match camera unit)
56  *****/
57 #define FRAME_W            640
58 #define FRAME_H            480
59 #define FRAME_BYTES        (FRAME_W * FRAME_H) /* 307,200 - grayscale */
60 #define BLOCK_SIZE         8
61 #define BLOCKS_W            (FRAME_W / BLOCK_SIZE) /* 80 */
62 #define BLOCKS_H            (FRAME_H / BLOCK_SIZE) /* 60 */
63 #define NUM_POSITIONS      3
64
65 /*****
66  * WHT Reconstruction (must match FPGA and sim)
67  *****/
```

```

68 #define Q_SHIFT          5                /* q_step = 2^5 = 32 */
69 #define GAUSS_SIGMA      4.0f             /* matches sim */
70
71 /*****
72  * Task Config
73  *****/
74 #define STACK_LORA        16384
75 #define STACK_WS          8192
76 #define PRIO_LORA         5
77 #define PRIO_WS           4
78 #define CORE_LORA         0
79 #define CORE_WS           1
80
81 /*****
82  * System Configuration - Computed Constants
83  *****/
84
85 // Memory allocation
86 #define LW_BYTES           (10 << 1)      // (10 * 2)
87 #define IMAGE_MAX_SIZE_BYTES (1 << LW_BYTES) // (1024 * 1024)
88 #define IMAGE_QUEUE_LENGTH 3
89 #define UART_BUFFER_SIZE  2048
90
91 // Computed timeout values (compile-time calculations)
92 #define MS_TO_TICKS(ms)   pdMS_TO_TICKS(ms)
93 #define SECONDS_TO_TICKS(s) pdMS_TO_TICKS((s) * 1000)
94
95 // Task priorities
96 #define TASK_PRIORITY_HIGH 5
97 #define TASK_PRIORITY_MEDIUM 4
98 #define TASK_PRIORITY_LOW 3
99
100 #define STACK_SIZE_BIT_WIDTH_S 11
101 #define STACK_SIZE_BIT_WIDTH_M 12
102 #define STACK_SIZE_BIT_WIDTH_L 13
103
104 // Task stack sizes
105 #define STACK_SIZE_SMALL (1 << STACK_SIZE_BIT_WIDTH_S) // 2048
106 #define STACK_SIZE_MEDIUM (1 << STACK_SIZE_BIT_WIDTH_M) // 4096
107 #define STACK_SIZE_LARGE (1 << STACK_SIZE_BIT_WIDTH_L) // 8192

```

esp/esp32s3_wroom/main/inc/image_reconstruct.h

```
1 #pragma once
2
3 #include <stddef.h>
4 #include <stdint.h>
5 #include "esp_err.h"
6
7 esp_err_t alloc_reference_frames(void);
8
9 uint8_t *reconstruct(const uint8_t *packet, size_t pkt_len,
10                    uint8_t *position_out);
```

esp/esp32s3_wroom/main/inc/lora.h

```
1 #pragma once
2
3 #include "config.h"
4 #include "esp_err.h"
5
6 esp_err_t lora_init(void);
7 void lora_receive_task(void *arg);
8
9 #ifdef TEST_MODE_LORA_BENCH
10 void lora_bench_task(void *arg);
11 #endif
```

esp/esp32s3_wroom/main/inc/wifi_ap.h

```
1 #pragma once
2 #include "esp_err.h"
3
4 esp_err_t wifi_ap_init(void);
```

esp/esp32s3_wroom/main/inc/ws_server.h

```
1 #pragma once
2 #include <stdint.h>
3 #include <stddef.h>
4 #include "esp_err.h"
5
6 esp_err_t ws_server_init(void);
7 esp_err_t ws_broadcast(uint8_t position, uint32_t seq,
8                       const uint8_t *pixels, size_t len);
9 void      ws_test_task(void *arg);
```

```

1  #include <math.h>
2  #include <stddef.h>
3  #include <stdint.h>
4  #include <stdlib.h>
5  #include <string.h>
6
7  #include "esp_heap_caps.h"
8  #include "esp_log.h"
9
10 #include "config.h"
11 #include "image_reconstruct.h"
12
13 static const char *TAG = "RECON";
14
15 static uint8_t *ref_frames[NUM_POSITIONS];
16
17 esp_err_t alloc_reference_frames(void)
18 {
19     for (int i = 0; i < NUM_POSITIONS; i++) {
20         ref_frames[i] =
21             heap_caps_malloc(FRAME_BYTES, 1, MALLOC_CAP_SPIRAM);
22         if (!ref_frames[i]) {
23             ESP_LOGE(TAG, "PSRAM alloc failed for position %d", i);
24             return ESP_ERR_NO_MEM;
25         }
26     }
27     ESP_LOGI(TAG, "reference frames allocated: %d x %d KB in PSRAM",
28             NUM_POSITIONS, FRAME_BYTES / 1024);
29     return ESP_OK;
30 }
31
32 /*****
33  * Inverse 8-point WHT -- maps to wht8.v butterfly
34  * 3 stages, stride 1/2/4, pure add/sub, no multiply.
35  *****/
36
37 static void iwht8(const float *x, float *y)
38 {
39     float s1[8], s2[8];
40
41     s1[0] = x[0] + x[1]; s1[1] = x[0] - x[1];
42     s1[2] = x[2] + x[3]; s1[3] = x[2] - x[3];
43     s1[4] = x[4] + x[5]; s1[5] = x[4] - x[5];
44     s1[6] = x[6] + x[7]; s1[7] = x[6] - x[7];
45
46     s2[0] = s1[0] + s1[2]; s2[2] = s1[0] - s1[2];
47     s2[1] = s1[1] + s1[3]; s2[3] = s1[1] - s1[3];
48     s2[4] = s1[4] + s1[6]; s2[6] = s1[4] - s1[6];
49     s2[5] = s1[5] + s1[7]; s2[7] = s1[5] - s1[7];
50
51     y[0] = s2[0] + s2[4]; y[4] = s2[0] - s2[4];
52     y[1] = s2[1] + s2[5]; y[5] = s2[1] - s2[5];
53     y[2] = s2[2] + s2[6]; y[6] = s2[2] - s2[6];
54     y[3] = s2[3] + s2[7]; y[7] = s2[3] - s2[7];
55 }
56
57 static void iwht_2d(float C[8][8], float P[8][8])
58 {
59     float tmp[8][8];
60     float row_in[8], row_out[8];
61
62     for (int r = 0; r < 8; r++) {
63         for (int c = 0; c < 8; c++) row_in[c] = C[r][c];
64         iwht8(row_in, row_out);
65         for (int c = 0; c < 8; c++) tmp[r][c] = row_out[c];
66     }
67 }

```

```

68     for (int c = 0; c < 8; c++) {
69         for (int r = 0; r < 8; r++) row_in[r] = tmp[r][c];
70         iwht8(row_in, row_out);
71         for (int r = 0; r < 8; r++) P[r][c] = row_out[r] / 64.0f;
72     }
73 }
74
75 /*****
76  * Reconstruct from FPGA compressed packet
77  *
78  * Wire format (from blk_schd.v):
79  *   Per block: [header_hi][header_lo] then per coeff [rc][q_hi][q_lo]
80  *   header_hi = {2'b0, by[4:0], bx[5]}
81  *   header_lo = {bx[4:0], coeff_n[2:0]}
82  *   rc        = {0, 0, r[2:0], c[2:0]}
83  *   q         = signed 16-bit (q_hi:q_lo)
84  *
85  * Reconstruction (matches decode_frame.m):
86  *   1. Dequantize: val = q << Q_SHIFT
87  *   2. Inverse 2D WHT -> spatial patch
88  *   3. Gaussian smooth diff
89  *   4. Add to reference frame
90  *****/
91
92 uint8_t *reconstruct(const uint8_t *packet, size_t pkt_len,
93                     uint8_t *position_out)
94 {
95     if (pkt_len < 2) return NULL;
96
97     /* Position is encoded in first block's header if present,
98        but the FPGA doesn't send position in the compressed stream.
99        For now, hardcode to center (1). */
100    *position_out = 1;
101    uint8_t *ref = ref_frames[*position_out];
102
103    /* Build diff image from compressed blocks */
104    float *diff = heap_caps_malloc(FRAME_BYTES, sizeof(float),
105                                   MALLOC_CAP_SPIRAM);
106    if (!diff) return NULL;
107
108    size_t pos = 0;
109    int n_blocks = 0;
110
111    while (pos + 2 <= pkt_len) {
112        uint8_t hdr_hi = packet[pos];
113        uint8_t hdr_lo = packet[pos + 1];
114
115        if (hdr_hi == 0 && hdr_lo == 0)
116            break;
117
118        int by = (hdr_hi >> 1) & 0x1F;
119        int bx = ((hdr_hi & 0x01) << 5) | ((hdr_lo >> 3) & 0x1F);
120        int coeff_n = hdr_lo & 0x07;
121        pos += 2;
122
123        if (pos + coeff_n * 3 > pkt_len)
124            break;
125
126        float C[8][8];
127        memset(C, 0, sizeof(C));
128
129        for (int k = 0; k < coeff_n; k++) {
130            uint8_t rc = packet[pos];
131            uint8_t q_hi = packet[pos + 1];
132            uint8_t q_lo = packet[pos + 2];
133            pos += 3;
134
135            int r = (rc >> 3) & 0x07;
136            int c = rc & 0x07;
137            int16_t q = (int16_t)((q_hi << 8) | q_lo);

```



```

138         C[r][c] = (float)((int32_t)q << Q_SHIFT);
139     }
140
141     float P[8][8];
142     iwht_2d(C, P);
143
144     int py0 = by * 8;
145     int px0 = bx * 8;
146     for (int r = 0; r < 8; r++) {
147         for (int c = 0; c < 8; c++) {
148             int y = py0 + r;
149             int x = px0 + c;
150             if (y < FRAME_H && x < FRAME_W)
151                 diff[y * FRAME_W + x] = P[r][c];
152         }
153     }
154     n_blocks++;
155 }
156
157 ESP_LOGI(TAG, "%d blocks, %zu bytes", n_blocks, pos);
158
159 /* TODO: LPF smoothing pass here */
160
161 /* Add diff to reference frame */
162 for (int i = 0; i < FRAME_BYTES; i++) {
163     int val = (int)ref[i] + (int)roundf(diff[i]);
164     ref[i] = (uint8_t)(val < 0 ? 0 : val > 255 ? 255 : val);
165 }
166
167 free(diff);
168 return ref;
169
170 }

```

esp/esp32s3_wroom/main/src/lora.c

```
1  /**
2   * lora.c -- ESP32-S3 LoRa receiver
3   *
4   * Listens for compressed diff packets from DevKitV1,
5   * reconstructs frames, broadcasts via WebSocket.
6   */
7  #include "config.h"
8
9  #include <stdio.h>
10 #include <stdlib.h>
11 #include <string.h>
12
13 #include "freertos/FreeRTOS.h"
14 #include "freertos/task.h"
15
16 #include "driver/gpio.h"
17 #include "driver/uart.h"
18
19 #include "esp_log.h"
20 #include "esp_timer.h"
21
22 #include "image_reconstruct.h"
23 #include "lora.h"
24 #include "ws_server.h"
25
26 static const char *TAG = "LORA";
27
28 /* ----- */
29 static void uart_init_lora(void)
30 {
31     uart_config_t cfg = {
32         .baud_rate = LORA_BAUD_RATE,
33         .data_bits = UART_DATA_8_BITS,
34         .parity = UART_PARITY_DISABLE,
35         .stop_bits = UART_STOP_BITS_1,
36         .flow_ctrl = UART_HW_FLOWCTRL_DISABLE,
37         .source_clk = UART_SCLK_DEFAULT,
38     };
39
40     uart_driver_install(LORA_UART_NUM, LORA_BUF_SIZE, LORA_BUF_SIZE, 0,
41         NULL, 0);
42     uart_param_config(LORA_UART_NUM, &cfg);
43     uart_set_pin(LORA_UART_NUM, LORA_PIN_TX, LORA_PIN_RX, -1, -1);
44     vTaskDelay(pdMS_TO_TICKS(LORA_INIT_DELAY_MS));
45 }
46
47 static bool readline(char *buf, int lenmax, int timeout_ms)
48 {
49     int pos = 0;
50     int64_t deadline = esp_timer_get_time() + (int64_t)timeout_ms * 1000;
51
52     while (esp_timer_get_time() < deadline) {
53         uint8_t b;
54         if (uart_read_bytes(LORA_UART_NUM, &b, 1, pdMS_TO_TICKS(10)) <=
55             0)
56             continue;
57
58         if (b == '\n') {
59             if (pos > 0 && buf[pos - 1] == '\r') pos--;
60             buf[pos] = '\0';
61             return pos > 0;
62         }
63
64         if (pos < lenmax - 1) buf[pos++] = (char)b;
65     }
66
67     buf[pos] = '\0';
```

```

68     return false;
69 }
70
71 static bool at_cmd(const char *cmd, const char *expect, int timeout_ms)
72 {
73     static const int POLL_MS = 50;
74
75     uart_flush_input(LORA_UART_NUM);
76     uart_write_bytes(LORA_UART_NUM, cmd, strlen(cmd));
77
78     char line[INT8_MAX + 1];
79     int elapsed = 0;
80
81     while (elapsed < timeout_ms) {
82         if (readline(line, sizeof(line), POLL_MS)) {
83             ESP_LOGI(TAG, " %s", line);
84             if (expect && strstr(line, expect)) return true;
85         }
86         elapsed += POLL_MS;
87     }
88     return false;
89 }
90
91 /* ----- */
92 static void module_init(void)
93 {
94     at_cmd("AT+RESET\r\n", "+RESET", 1000);
95     vTaskDelay(pdMS_TO_TICKS(1500));
96
97     char cmd[64];
98
99     snprintf(cmd, sizeof(cmd), "AT+ADDRESS=%d\r\n", LORA_ADDRESS_RECEIVER);
100    at_cmd(cmd, "+OK", LORA_AT_TIMEOUT_MS);
101
102    snprintf(cmd, sizeof(cmd), "AT+NETWORKID=%d\r\n", LORA_NETWORK_ID);
103    at_cmd(cmd, "+OK", LORA_AT_TIMEOUT_MS);
104
105    snprintf(cmd, sizeof(cmd), "AT+PARAMETER=%d,%d,%d,%d\r\n", LORA_SF,
106             LORA_BW, LORA_CR, LORA_PREAMBLE);
107    at_cmd(cmd, "+OK", LORA_AT_TIMEOUT_MS);
108
109    at_cmd("AT+CRFOP=7\r\n", "+OK", LORA_AT_TIMEOUT_MS);
110
111    at_cmd("AT+ADDRESS?\r\n", "+ADDRESS=", LORA_AT_TIMEOUT_MS);
112    at_cmd("AT+NETWORKID?\r\n", "+NETWORKID=", LORA_AT_TIMEOUT_MS);
113    at_cmd("AT+PARAMETER?\r\n", "+PARAMETER=", LORA_AT_TIMEOUT_MS);
114    at_cmd("AT+BAND?\r\n", "+BAND=", LORA_AT_TIMEOUT_MS);
115    at_cmd("AT+CRFOP?\r\n", "+CRFOP=", LORA_AT_TIMEOUT_MS);
116 }
117
118 /* ----- */
119 static bool parse_rcv(char *line, int *src, char **data, int *len)
120 {
121     if (strncmp(line, "+RCV=", 5) != 0) return false;
122
123     char *p = line + 5;
124
125     char *c1 = strchr(p, ',');
126     if (!c1) return false;
127     *c1 = '\0';
128     *src = atoi(p);
129
130     char *p_len = c1 + 1;
131     char *c2 = strchr(p_len, ',');
132     if (!c2) return false;
133     *c2 = '\0';
134
135     *len = atoi(p_len);
136     if (*len <= 0 || *len > LORA_MAX_PAYLOAD) return false;
137

```

```

138     *data = c2 + 1;
139
140     char *end = *data + *len;
141     if (*end == ',') {
142         *end = '\0';
143     } else if (*end == '\0') {
144         /* firmware omitted rssi/snr */
145     } else {
146         return false;
147     }
148
149     return true;
150 }
151
152 /* ----- */
153 esp_err_t lora_init(void)
154 {
155     uart_init_lora();
156     vTaskDelay(pdMS_TO_TICKS(500));
157     module_init();
158     vTaskDelay(pdMS_TO_TICKS(500));
159
160     ESP_LOGI(TAG, "LoRa receiver ready addr=%d net=%d SF=%d BW=%d",
161              LORA_ADDRESS_RECEIVER, LORA_NETWORK_ID, LORA_SF, LORA_BW);
162     return ESP_OK;
163 }
164
165 void lora_receive_task(void *arg)
166 {
167     (void)arg;
168
169     char line[300];
170     uint32_t seq = 0;
171
172     ESP_LOGI(TAG, "receive task started");
173
174     for (;;) {
175         if (!readline(line, sizeof(line), 10000))
176             continue;
177
178         if (strncmp(line, "+RCV=", 5) != 0) {
179             ESP_LOGD(TAG, "ignoring: %s", line);
180             continue;
181         }
182
183         int src, len;
184         char *data;
185         if (!parse_rcv(line, &src, &data, &len)) {
186             ESP_LOGW(TAG, "parse failed");
187             continue;
188         }
189
190         ESP_LOGI(TAG, "rx %d bytes from %d", len, src);
191
192         uint8_t pos;
193         uint8_t *frame = reconstruct((const uint8_t *)data, len, &pos);
194         if (!frame) {
195             ESP_LOGW(TAG, "reconstruct failed");
196             continue;
197         }
198
199         ws_broadcast(pos, ++seq, frame, FRAME_BYTES);
200     }
201 }
202
203 /* ----- */
204 #ifndef TEST_MODE_LORA_BENCH
205
206 static void send_echo(int dest, const char *data, int len)
207 {

```

```

208     char hdr[32];
209     int hdr_len = snprintf(hdr, sizeof(hdr), "AT+SEND=%d,%d,", dest, len);
210     uart_write_bytes(LORA_UART_NUM, hdr, hdr_len);
211     uart_write_bytes(LORA_UART_NUM, data, len);
212     uart_write_bytes(LORA_UART_NUM, "\r\n", 2);
213 }
214
215 static bool wait_for_start(char *line, int line_size)
216 {
217     int src, len;
218     char *data;
219
220     static int beacon_count = 0;
221     beacon_count++;
222     ESP_LOGW(TAG, "beacon #%d sending READY", beacon_count);
223
224     const char *cmd = "AT+SEND=2,5,READY\r\n";
225     uart_write_bytes(LORA_UART_NUM, cmd, strlen(cmd));
226
227     int64_t deadline = esp_timer_get_time() + 1000000LL;
228     while (esp_timer_get_time() < deadline) {
229         if (!readline(line, line_size, 200)) continue;
230         ESP_LOGW(TAG, "handshake rx: %s", line);
231         if (parse_rcv(line, &src, &data, &len) && len == 5 &&
232             !memcmp(data, "START", 5))
233             return false;
234     }
235     return true;
236 }
237
238 void lora_bench_task(void *arg)
239 {
240     (void)arg;
241
242     ESP_LOGI(TAG, "=== S3 echo slave ===");
243
244     uart_init_lora();
245     vTaskDelay(pdMS_TO_TICKS(500));
246     module_init();
247     vTaskDelay(pdMS_TO_TICKS(500));
248
249     uart_flush_input(LORA_UART_NUM);
250     vTaskDelay(pdMS_TO_TICKS(200));
251
252     char line[300];
253     while (wait_for_start(line, sizeof(line)));
254     ESP_LOGI(TAG, "handshake complete");
255
256     for (;;) {
257         if (!readline(line, sizeof(line), 10000))
258             continue;
259         if (strncmp(line, "+RCV=", 5) != 0)
260             continue;
261
262         ESP_LOGI(TAG, "rx: %s", line);
263
264         int src, len;
265         char *data;
266         if (!parse_rcv(line, &src, &data, &len))
267             continue;
268
269         vTaskDelay(pdMS_TO_TICKS(15));
270         send_echo(src, data, len);
271     }
272 }
273 #endif

```

esp/esp32s3_wroom/main/src/main.c

```
1  /**
2   * main.c - ESP32-S3 Trail Camera Receiver
3   *
4   * Core 0: LoRa receive → Gaussian reconstruction
5   * Core 1: WiFi AP + WebSocket server → Flutter app
6   *
7   * Phone connects to "TrailCamera" AP, opens ws://192.168.4.1:80/stream,
8   * receives reconstructed grayscale frames as binary WebSocket messages.
9   */
10
11 #include <math.h>
12 #include <stdio.h>
13 #include <stdlib.h>
14 #include <string.h>
15
16 #include "freertos/FreeRTOS.h"
17 #include "freertos/task.h"
18
19 #include "esp_heap_caps.h"
20 #include "esp_log.h"
21 #include "esp_psram.h"
22
23 #include "config.h"
24 #include "image_reconstruct.h"
25 #include "lora.h"
26 #include "wifi_ap.h"
27 #include "ws_server.h"
28
29 static const char *TAG = "MAIN";
30
31 static esp_err_t init_system(void)
32 {
33     ESP_LOGI(TAG, "=== Trail Camera IV - ESP32 S3 Wroom ===");
34     ESP_LOGI(TAG, "IDF %s | heap %lu bytes", esp_get_idf_version(),
35              esp_get_free_heap_size());
36
37     esp_err_t ret;
38
39     ret = wifi_ap_init();
40     if (ret != ESP_OK) return ret;
41     ret = ws_server_init();
42     if (ret != ESP_OK) return ret;
43     ret = alloc_reference_frames();
44     if (ret != ESP_OK) return ret;
45     ret = lora_init();
46     if (ret != ESP_OK) return ret;
47
48     return ESP_OK;
49 }
50
51 static void create_tasks(void)
52 {
53     xTaskCreate(lora_receive_task, "lora_rx", STACK_SIZE_MEDIUM, NULL,
54               TASK_PRIORITY_MEDIUM, NULL);
55 #ifdef TEST_MODE_WS_PATTERNS
56     xTaskCreate(ws_test_task, "ws_test", STACK_SIZE_LARGE, NULL,
57               TASK_PRIORITY_LOW, NULL);
58 #endif
59 }
60
61 static void monitor_heap(void)
62 {
63     uint32_t prev = esp_get_free_heap_size();
64
65     for (;;) {
66         vTaskDelay(SECONDS_TO_TICKS(10));
67         uint32_t now = esp_get_free_heap_size();
```

```

68         ESP_LOGI(TAG, "heap: %lu (%+ld)", now, (int32_t)(now - prev));
69         prev = now;
70     }
71 }
72
73 void app_main(void)
74 {
75     #ifdef TEST_MODE_LORA_BENCH
76         /* Bench mode: only LoRa bench task runs - no camera/FPGA/sensor init
77          */
78         xTaskCreate(lora_bench_task, "lora_bench", STACK_SIZE_LARGE, NULL,
79                     TASK_PRIORITY_MEDIUM, NULL);
80         monitor_heap();
81     #else
82         if (init_system() != ESP_OK) {
83             ESP_LOGE(TAG, "init failed - halted");
84             return;
85         }
86         create_tasks();
87         monitor_heap();
88     #endif
89 }

```

esp/esp32s3_wroom/main/src/wifi_ap.c

```
1  /**
2   * wifi_ap.c - WiFi Access Point
3   *
4   * Creates a WPA2 hotspot "TrailCamera" so the phone connects directly
5   * in the field - no existing infrastructure needed.
6   */
7
8  #include "wifi_ap.h"
9  #include "config.h"
10 #include "esp_event.h"
11 #include "esp_log.h"
12 #include "esp_wifi.h"
13 #include "nvs_flash.h"
14 #include <string.h>
15
16 static const char *TAG = "WIFI_AP";
17
18 static void event_handler(void *arg, esp_event_base_t base, int32_t id,
19                          void *data)
20 {
21     if (base == WIFI_EVENT && id == WIFI_EVENT_AP_STACONNECTED) {
22         wifi_event_ap_staconnected_t *e = data;
23         ESP_LOGI(
24             TAG,
25             "client connected - MAC %02x:%02x:%02x:%02x:%02x:%02x",
26             e->mac[0], e->mac[1], e->mac[2], e->mac[3], e->mac[4],
27             e->mac[5]);
28     } else if (base == WIFI_EVENT && id == WIFI_EVENT_AP_STADISCONNECTED) {
29         wifi_event_ap_stadisconnected_t *e = data;
30         ESP_LOGI(
31             TAG,
32             "client disconnected - MAC %02x:%02x:%02x:%02x:%02x:%02x",
33             e->mac[0], e->mac[1], e->mac[2], e->mac[3], e->mac[4],
34             e->mac[5]);
35     }
36 }
37
38 esp_err_t wifi_ap_init(void)
39 {
40     esp_err_t ret = nvs_flash_init();
41     if (ret == ESP_ERR_NVS_NO_FREE_PAGES ||
42         ret == ESP_ERR_NVS_NEW_VERSION_FOUND) {
43         nvs_flash_erase();
44         ret = nvs_flash_init();
45     }
46     if (ret != ESP_OK) return ret;
47
48     ESP_ERROR_CHECK(esp_netif_init());
49     ESP_ERROR_CHECK(esp_event_loop_create_default());
50     esp_netif_create_default_wifi_ap();
51
52     wifi_init_config_t cfg = WIFI_INIT_CONFIG_DEFAULT();
53     ESP_ERROR_CHECK(esp_wifi_init(&cfg));
54
55     ESP_ERROR_CHECK(esp_event_handler_instance_register(
56         WIFI_EVENT, ESP_EVENT_ANY_ID, event_handler, NULL, NULL));
57
58     wifi_config_t ap_cfg = {
59         .ap = {
60             {
61                 .ssid = WIFI_AP_SSID,
62                 .ssid_len = strlen(WIFI_AP_SSID),
63                 .password = WIFI_AP_PASS,
64                 .channel = WIFI_AP_CHANNEL,
65                 .authmode = WIFI_AUTH_WPA2_PSK,
66                 .max_connection = WIFI_AP_MAX_CONN,
67             }
68         }
69     },
```



```
68     };
69
70     ESP_ERROR_CHECK(esp_wifi_set_mode(WIFI_MODE_AP));
71     ESP_ERROR_CHECK(esp_wifi_set_config(WIFI_IF_AP, &ap_cfg));
72     ESP_ERROR_CHECK(esp_wifi_start());
73
74     ESP_LOGI(TAG, "AP started - SSID: %s", WIFI_AP_SSID);
75     return ESP_OK;
76 }
```

esp/esp32s3_wroom/main/src/ws_server.c

```
1  /**
2   * ws_server.c - WebSocket server
3   *
4   * Serves reconstructed grayscale frames to connected Flutter clients.
5   * Push-based: call ws_broadcast() whenever a new frame is ready.
6   *
7   * Frame format over WebSocket (binary):
8   *   Byte 0:      position (0=left, 1=center, 2=right)
9   *   Bytes -14:   uint32 frame sequence number
10  *   Bytes -5end:  raw grayscale pixels, FRAME_W x FRAME_H, row-major
11  */
12
13 #include "ws_server.h"
14 #include "config.h"
15 #include "esp_heap_caps.h"
16 #include "esp_http_server.h"
17 #include "esp_log.h"
18 #include "esp_spiffs.h"
19 #include "freertos/FreeRTOS.h"
20 #include "freertos/task.h"
21 #include <stdio.h>
22 #include <string.h>
23 #include <sys/stat.h>
24
25 static const char *TAG = "WS";
26
27 static httpd_handle_t server = NULL;
28 static int client_fds[WS_MAX_CLIENTS];
29 static int client_count = 0;
30
31 /* ----- */
32
33 static void add_client(int fd)
34 {
35     if (client_count >= WS_MAX_CLIENTS) return;
36     client_fds[client_count++] = fd;
37     ESP_LOGI(TAG, "client connected fd=%d (%d total)", fd, client_count);
38 }
39
40 static void remove_client(int fd)
41 {
42     for (int i = 0; i < client_count; i++) {
43         if (client_fds[i] == fd) {
44             client_fds[i] = client_fds[--client_count];
45             ESP_LOGI(TAG, "client disconnected fd=%d (%d total)",
46                     fd, client_count);
47             return;
48         }
49     }
50 }
51
52 /* ----- */
53
54 static esp_err_t ws_handler(httpd_req_t *req)
55 {
56     if (req->method == HTTP_GET) {
57         /* Handshake */
58         add_client(httpd_req_to_sockfd(req));
59         return ESP_OK;
60     }
61
62     httpd_ws_frame_t frame = {.type = HTTPD_WS_TYPE_BINARY};
63
64     /* Get frame length */
65     esp_err_t ret = httpd_ws_recv_frame(req, &frame, 0);
66     if (ret != ESP_OK) return ret;
67 }
```

```

68     /* We don't expect data from client currently - just drain it */
69     if (frame.len > 0) {
70         uint8_t *buf = malloc(frame.len);
71         if (!buf) return ESP_ERR_NO_MEM;
72         frame.payload = buf;
73         ret = httpd_ws_recv_frame(req, &frame, frame.len);
74         free(buf);
75     }
76
77     return ret;
78 }
79
80 static const httpd_uri_t ws_uri = {
81     .uri = WS_PATH,
82     .method = HTTP_GET,
83     .handler = ws_handler,
84     .is_websocket = true,
85 };
86
87 /* ----- */
88
89 static esp_err_t spiffs_mount(void)
90 {
91     esp_vfs_spiffs_conf_t cfg = {
92         .base_path = "/www",
93         .partition_label = "www",
94         .max_files = 8,
95         .format_if_mount_failed = false,
96     };
97
98     esp_err_t ret = esp_vfs_spiffs_register(&cfg);
99     if (ret != ESP_OK) {
100         ESP_LOGE(TAG, "SPIFFS mount failed: %s", esp_err_to_name(ret));
101         return ret;
102     }
103
104     size_t total = 0, used = 0;
105     esp_spiffs_info("www", &total, &used);
106     ESP_LOGI(TAG, "SPIFFS: %u KB used / %u KB total",
107         (unsigned)(used / 1024), (unsigned)(total / 1024));
108     return ESP_OK;
109 }
110
111 static const char *mime_type(const char *path)
112 {
113     const char *dot = strrchr(path, '.');
114     if (!dot) return "application/octet-stream";
115     if (!strcmp(dot, ".html")) return "text/html";
116     if (!strcmp(dot, ".js")) return "application/javascript";
117     if (!strcmp(dot, ".json")) return "application/json";
118     if (!strcmp(dot, ".png")) return "image/png";
119     if (!strcmp(dot, ".css")) return "text/css";
120     if (!strcmp(dot, ".wasm")) return "application/wasm";
121     return "application/octet-stream";
122 }
123
124 static esp_err_t static_handler(httpd_req_t *req)
125 {
126     char filepath[600];
127     const char *uri = req->uri;
128
129     if (!strcmp(uri, "/")) uri = "/index.html";
130
131     snprintf(filepath, sizeof(filepath), "/www%.590s", uri);
132
133     struct stat st;
134     if (stat(filepath, &st) != 0) {
135         httpd_resp_send_404(req);
136         return ESP_OK;
137     }

```

```

138     FILE *f = fopen(filepath, "r");
139     if (!f) {
140         httpd_resp_send_404(req);
141         return ESP_OK;
142     }
143
144     httpd_resp_set_type(req, mime_type(filepath));
145
146     char buf[1024];
147     size_t n;
148     while ((n = fread(buf, 1, sizeof(buf), f)) > 0)
149         httpd_resp_send_chunk(req, buf, n);
150     httpd_resp_send_chunk(req, NULL, 0);
151
152     fclose(f);
153     return ESP_OK;
154 }
155
156 /* ----- */
157
158 esp_err_t ws_server_init(void)
159 {
160     memset(client_fds, -1, sizeof(client_fds));
161
162     esp_err_t ret = spiffs_mount();
163     if (ret != ESP_OK) return ret;
164
165     httpd_config_t cfg = HTTPD_DEFAULT_CONFIG();
166     cfg.server_port = WS_PORT;
167     cfg.ctrl_port = WS_PORT + 1;
168     cfg.max_open_sockets = WS_MAX_CLIENTS + 3;
169     cfg.uri_match_fn = httpd_uri_match_wildcard;
170
171     ret = httpd_start(&server, &cfg);
172     if (ret != ESP_OK) {
173         ESP_LOGE(TAG, "start failed: %s", esp_err_to_name(ret));
174         return ret;
175     }
176
177     httpd_register_uri_handler(server, &ws_uri);
178
179     static const httpd_uri_t static_uri = {
180         .uri = "/*",
181         .method = HTTP_GET,
182         .handler = static_handler,
183     };
184     httpd_register_uri_handler(server, &static_uri);
185
186     ESP_LOGI(TAG, "WebSocket server ready on ws://192.168.4.1:%d%s",
187             WS_PORT, WS_PATH);
188     return ESP_OK;
189 }
190
191 /* ----- */
192
193 #ifdef TEST_MODE_WS_PATTERNS
194 void ws_test_task(void *arg)
195 {
196     (void)arg;
197
198     uint8_t *buf = heap_caps_malloc(FRAME_BYTES, MALLOC_CAP_SPIRAM);
199     if (!buf) {
200         ESP_LOGE(TAG, "PSRAM alloc failed for test frame");
201         vTaskDelete(NULL);
202         return;
203     }
204
205     for (int i = 0; i < FRAME_BYTES; i++)
206         buf[i] = (uint8_t)((i % FRAME_W) * 255 / (FRAME_W - 1));
207

```

```

208
209     uint32_t seq = 0;
210
211     for (;;) {
212         ws_broadcast(1, ++seq, buf, FRAME_BYTES);
213         vTaskDelay(pdMS_TO_TICKS(1000));
214     }
215 }
216 #endif
217
218 esp_err_t ws_broadcast(uint8_t position, uint32_t seq, const uint8_t *pixels,
219                       size_t len)
220 {
221     if (client_count == 0) return ESP_OK;
222
223     /* Build header: 1 byte position + 4 bytes seq */
224     const size_t header_len = 5;
225     uint8_t header[5];
226     header[0] = position;
227     header[1] = (seq >> 24) & 0xFF;
228     header[2] = (seq >> 16) & 0xFF;
229     header[3] = (seq >> 8) & 0xFF;
230     header[4] = (seq) & 0xFF;
231
232     /* Send header + pixels as two fragments or allocate one buffer.
233      * esp_http_server doesn't support scatter-gather so combine them. */
234     const size_t total = header_len + len;
235     uint8_t *buf = malloc(total);
236     if (!buf) return ESP_ERR_NO_MEM;
237     memcpy(buf, header, header_len);
238     memcpy(buf + header_len, pixels, len);
239
240     httpd_ws_frame_t frame = {
241         .type = HTTPD_WS_TYPE_BINARY,
242         .payload = buf,
243         .len = total,
244     };
245
246     for (int i = client_count - 1; i >= 0; i--) {
247         esp_err_t ret =
248             httpd_ws_send_frame_async(server, client_fds[i], &frame);
249         if (ret != ESP_OK) {
250             ESP_LOGW(TAG, "send failed fd=%d - removing client",
251                     client_fds[i]);
252             remove_client(client_fds[i]);
253         }
254     }
255
256     free(buf);
257     return ESP_OK;
258 }

```

fpga/src/rtl/blk_schd.v

```

1  /*
2  * blk_schd.v -- Block scheduler FSM
3  *
4  * Iterates 40x30 blocks over a 320x240 frame pair in PSRAM.
5  * Per block: LOAD current + previous pixels, diff, WHT, coeff, EMIT.
6  * Two PSRAM regions: SPI writes to cur_base, scheduler reads both.
7  * top.v swaps cur_base/prev_base each frame.
8  *
9  * Flat bus convention: element [r][c] = flat[(r*8+c)*W +: W]
10 */
11
12 module block_sched
13     #(parameter W = 16, TOP_K = 5)
14     (input wire clk,
15      input wire rst_n,
16      input wire start,
17      output reg frame_done,
18
19      output reg psram_read,
20      output reg [21:0] psram_addr,
21      input wire [15:0] psram_rdata,
22      input wire psram_busy,
23
24      output reg tx_valid,
25      output reg [7:0] tx_data,
26      input wire tx_ready,
27
28      input wire [21:0] cur_base,
29      input wire [21:0] prev_base);
30
31     // =====
32     // FSM states
33     // =====
34     localparam [2:0] IDLE = 3'd0,
35                     LOAD = 3'd1,
36                     WHT = 3'd2,
37                     WAIT = 3'd3,
38                     EMIT = 3'd4,
39                     NEXT = 3'd5;
40     reg [2:0] state;
41
42     // =====
43     // Block counters
44     // =====
45     reg [5:0] bx;
46     reg [4:0] by;
47
48     // =====
49     // LOAD counters + diff logic
50     // =====
51     reg [2:0] load_row;
52     reg [1:0] load_col;
53     reg load_pass;
54     reg psram_pending;
55     reg [15:0] cur_word;
56
57     // =====
58     // 8x8 pixel register array -- flat packed bus
59     // =====
60     reg signed [W*64-1:0] pixels;
61
62     // =====
63     // Address calculation
64     // =====
65     wire [21:0] by_offset = ({17'b0, by} << 11) + ({17'b0, by} << 9);
66     wire [21:0] bx_offset = {16'b0, bx} << 3;
67     wire [21:0] row_offset = ({19'b0, load_row} << 8)

```

```

68             + ({19'b0, load_row} << 6);
69 wire [21:0] col_offset = {20'b0, load_col, 1'b0};
70 wire [21:0] block_offset = by_offset + bx_offset
71             + row_offset + col_offset;
72 wire [21:0] cur_pixel_addr = cur_base + block_offset;
73 wire [21:0] prev_pixel_addr = prev_base + block_offset;
74
75 wire signed [W-1:0] diff_lo = {{(W-8){1'b0}}, cur_word[7:0]}
76                             - {{(W-8){1'b0}}, psram_rdata[7:0]};
77 wire signed [W-1:0] diff_hi = {{(W-8){1'b0}}, cur_word[15:8]}
78                             - {{(W-8){1'b0}}, psram_rdata[15:8]};
79
80 wire [4:0] pix_pair = {load_row, load_col};
81
82 // =====
83 // WHT -- single wht8, 4-bit pass counter
84 //   pass 0-7: rows 0-7      pass 8-15: cols 0-7
85 // =====
86 reg [3:0] wht_pass;
87 reg signed [W*8-1:0] wht_in;
88 wire signed [W*8-1:0] wht_out;
89
90 integer m;
91 always @(*) begin
92     case (wht_pass)
93         4'd0: for(m=0;m<8;m=m+1) wht_in[m*W+:W] = pixels[(0*8+m)*W+:W];
94         4'd1: for(m=0;m<8;m=m+1) wht_in[m*W+:W] = pixels[(1*8+m)*W+:W];
95         4'd2: for(m=0;m<8;m=m+1) wht_in[m*W+:W] = pixels[(2*8+m)*W+:W];
96         4'd3: for(m=0;m<8;m=m+1) wht_in[m*W+:W] = pixels[(3*8+m)*W+:W];
97         4'd4: for(m=0;m<8;m=m+1) wht_in[m*W+:W] = pixels[(4*8+m)*W+:W];
98         4'd5: for(m=0;m<8;m=m+1) wht_in[m*W+:W] = pixels[(5*8+m)*W+:W];
99         4'd6: for(m=0;m<8;m=m+1) wht_in[m*W+:W] = pixels[(6*8+m)*W+:W];
100        4'd7: for(m=0;m<8;m=m+1) wht_in[m*W+:W] = pixels[(7*8+m)*W+:W];
101        4'd8: for(m=0;m<8;m=m+1) wht_in[m*W+:W] = pixels[(m*8+0)*W+:W];
102        4'd9: for(m=0;m<8;m=m+1) wht_in[m*W+:W] = pixels[(m*8+1)*W+:W];
103        4'd10: for(m=0;m<8;m=m+1) wht_in[m*W+:W] = pixels[(m*8+2)*W+:W];
104        4'd11: for(m=0;m<8;m=m+1) wht_in[m*W+:W] = pixels[(m*8+3)*W+:W];
105        4'd12: for(m=0;m<8;m=m+1) wht_in[m*W+:W] = pixels[(m*8+4)*W+:W];
106        4'd13: for(m=0;m<8;m=m+1) wht_in[m*W+:W] = pixels[(m*8+5)*W+:W];
107        4'd14: for(m=0;m<8;m=m+1) wht_in[m*W+:W] = pixels[(m*8+6)*W+:W];
108        4'd15: for(m=0;m<8;m=m+1) wht_in[m*W+:W] = pixels[(m*8+7)*W+:W];
109    endcase
110 end
111
112 wht2d #(.W(W)) wht_inst(
113     .x(wht_in),
114     .y(wht_out)
115 );
116
117 // =====
118 // Sequency mask (combinational, reads pixels after WHT done)
119 // =====
120 wire signed [W*64-1:0] masked;
121
122 seq_msk #(.W(W)) msk_inst(
123     .c_i(pixels),
124     .c_o(masked)
125 );
126
127 // =====
128 // Coeff extractor
129 // =====
130 reg coeff_bgn;
131 wire coeff_done;
132 wire [3*TOP_K-1:0] coeff_r;
133 wire [3*TOP_K-1:0] coeff_c;
134 wire signed [W*TOP_K-1:0] coeff_q;
135 wire [2:0] coeff_n;
136
137 coeff #(.W(W), .TOP_K(TOP_K)) coeff_inst(

```

```

138         .clk(clk),
139         .rst(~rst_n),
140         .bgn(coeff_bgn),
141         .c_i(masked),
142         .r_o(coeff_r),
143         .c_o(coeff_c),
144         .q_o(coeff_q),
145         .n_out(coeff_n),
146         .done(coeff_done)
147     );
148
149     // =====
150     // EMIT sub-counter
151     // =====
152     reg [2:0] emit_idx;
153     reg [3:0] emit_byte;
154     reg [1:0] emit_sub;
155     reg emit_active;
156
157     wire [7:0] header_hi = {2'b0, by, bx[5]};
158     wire [7:0] header_lo = {bx[4:0], coeff_n};
159
160     wire [2:0] ei = emit_idx;
161     wire [7:0] rc_byte = {2'b0, coeff_r[ei*3 +: 3], coeff_c[ei*3 +: 3]};
162     wire signed [W-1:0] emit_q = coeff_q[ei*W +: W];
163     wire [7:0] q_hi = emit_q[W-1:8];
164     wire [7:0] q_lo = emit_q[7:0];
165
166     // =====
167     // Main FSM
168     // =====
169     integer j;
170
171     always @(posedge clk or negedge rst_n) begin
172         if (!rst_n) begin
173             state <= IDLE;
174             bx <= 6'd0;
175             by <= 5'd0;
176             load_row <= 3'd0;
177             load_col <= 2'd0;
178             load_pass <= 1'b0;
179             psram_read <= 1'b0;
180             psram_addr <= 22'd0;
181             psram_pending <= 1'b0;
182             cur_word <= 16'd0;
183             coeff_bgn <= 1'b0;
184             wht_pass <= 4'd0;
185             tx_valid <= 1'b0;
186             tx_data <= 8'd0;
187             frame_done <= 1'b0;
188             emit_idx <= 3'd0;
189             emit_byte <= 4'd0;
190             emit_sub <= 2'd0;
191             emit_active <= 1'b0;
192             pixels <= {(W*64){1'b0}};
193         end else begin
194
195             psram_read <= 1'b0;
196             coeff_bgn <= 1'b0;
197             tx_valid <= 1'b0;
198             frame_done <= 1'b0;
199
200             case (state)
201             // -----
202             IDLE: begin
203                 bx <= 6'd0;
204                 by <= 5'd0;
205                 state <= start ? LOAD : IDLE;
206                 load_pass <= 1'b0;
207

```



```

208 // -----
209
210 LOAD: begin
211     if (~psram_busy & ~psram_pending) begin
212         psram_addr <= load_pass ? prev_pixel_addr
213                               : cur_pixel_addr;
214         psram_read <= 1'b1;
215         psram_pending <= 1'b1;
216     end
217
218     if (psram_pending & ~psram_busy) begin
219         psram_pending <= 1'b0;
220
221         if (~load_pass) begin
222             cur_word <= psram_rdata;
223             load_pass <= 1'b1;
224         end else begin
225             case (pix_pair)
226             5'd0: begin pixels[ 0*W+:W]<=diff_lo; pixels[ 1*W+:W]<=
227                   diff_hi; end
228             5'd1: begin pixels[ 2*W+:W]<=diff_lo; pixels[ 3*W+:W]<=
229                   diff_hi; end
230             5'd2: begin pixels[ 4*W+:W]<=diff_lo; pixels[ 5*W+:W]<=
231                   diff_hi; end
232             5'd3: begin pixels[ 6*W+:W]<=diff_lo; pixels[ 7*W+:W]<=
233                   diff_hi; end
234             5'd4: begin pixels[ 8*W+:W]<=diff_lo; pixels[ 9*W+:W]<=
235                   diff_hi; end
236             5'd5: begin pixels[10*W+:W]<=diff_lo; pixels[11*W+:W]<=
237                   diff_hi; end
238             5'd6: begin pixels[12*W+:W]<=diff_lo; pixels[13*W+:W]<=
239                   diff_hi; end
240             5'd7: begin pixels[14*W+:W]<=diff_lo; pixels[15*W+:W]<=
241                   diff_hi; end
242             5'd8: begin pixels[16*W+:W]<=diff_lo; pixels[17*W+:W]<=
243                   diff_hi; end
244             5'd9: begin pixels[18*W+:W]<=diff_lo; pixels[19*W+:W]<=
245                   diff_hi; end
246             5'd10: begin pixels[20*W+:W]<=diff_lo; pixels[21*W+:W]<=
247                   diff_hi; end
248             5'd11: begin pixels[22*W+:W]<=diff_lo; pixels[23*W+:W]<=
249                   diff_hi; end
250             5'd12: begin pixels[24*W+:W]<=diff_lo; pixels[25*W+:W]<=
251                   diff_hi; end
252             5'd13: begin pixels[26*W+:W]<=diff_lo; pixels[27*W+:W]<=
253                   diff_hi; end
254             5'd14: begin pixels[28*W+:W]<=diff_lo; pixels[29*W+:W]<=
255                   diff_hi; end
256             5'd15: begin pixels[30*W+:W]<=diff_lo; pixels[31*W+:W]<=
257                   diff_hi; end
258             5'd16: begin pixels[32*W+:W]<=diff_lo; pixels[33*W+:W]<=
259                   diff_hi; end
260             5'd17: begin pixels[34*W+:W]<=diff_lo; pixels[35*W+:W]<=
261                   diff_hi; end
262             5'd18: begin pixels[36*W+:W]<=diff_lo; pixels[37*W+:W]<=
263                   diff_hi; end
264             5'd19: begin pixels[38*W+:W]<=diff_lo; pixels[39*W+:W]<=
265                   diff_hi; end
266             5'd20: begin pixels[40*W+:W]<=diff_lo; pixels[41*W+:W]<=
267                   diff_hi; end
268             5'd21: begin pixels[42*W+:W]<=diff_lo; pixels[43*W+:W]<=
269                   diff_hi; end
270             5'd22: begin pixels[44*W+:W]<=diff_lo; pixels[45*W+:W]<=
271                   diff_hi; end
272             5'd23: begin pixels[46*W+:W]<=diff_lo; pixels[47*W+:W]<=
273                   diff_hi; end
274             5'd24: begin pixels[48*W+:W]<=diff_lo; pixels[49*W+:W]<=
275                   diff_hi; end
276             5'd25: begin pixels[50*W+:W]<=diff_lo; pixels[51*W+:W]<=
277                   diff_hi; end

```

```

252         5'd26: begin pixels[52*W+:W]<=diff_lo; pixels[53*W+:W]<=
253             diff_hi; end
254         5'd27: begin pixels[54*W+:W]<=diff_lo; pixels[55*W+:W]<=
255             diff_hi; end
256         5'd28: begin pixels[56*W+:W]<=diff_lo; pixels[57*W+:W]<=
257             diff_hi; end
258         5'd29: begin pixels[58*W+:W]<=diff_lo; pixels[59*W+:W]<=
259             diff_hi; end
260         5'd30: begin pixels[60*W+:W]<=diff_lo; pixels[61*W+:W]<=
261             diff_hi; end
262         5'd31: begin pixels[62*W+:W]<=diff_lo; pixels[63*W+:W]<=
263             diff_hi; end
264     endcase
265     load_pass <= 1'b0;
266
267     if (load_col == 2'd3) begin
268         load_col <= 2'd0;
269         if (load_row == 3'd7) begin
270             load_row <= 3'd0;
271             wht_pass <= 4'd0;
272             state <= WHT;
273         end else begin
274             load_row <= load_row + 3'd1;
275         end
276     end else begin
277         load_col <= load_col + 2'd1;
278     end
279 end
280
281 // -----
282 WHT: begin
283     case (wht_pass)
284         4'd0: for(j=0;j<8;j=j+1) pixels[(0*8+j)*W+:W] <= wht_out[j*W+:W];
285         4'd1: for(j=0;j<8;j=j+1) pixels[(1*8+j)*W+:W] <= wht_out[j*W+:W];
286         4'd2: for(j=0;j<8;j=j+1) pixels[(2*8+j)*W+:W] <= wht_out[j*W+:W];
287         4'd3: for(j=0;j<8;j=j+1) pixels[(3*8+j)*W+:W] <= wht_out[j*W+:W];
288         4'd4: for(j=0;j<8;j=j+1) pixels[(4*8+j)*W+:W] <= wht_out[j*W+:W];
289         4'd5: for(j=0;j<8;j=j+1) pixels[(5*8+j)*W+:W] <= wht_out[j*W+:W];
290         4'd6: for(j=0;j<8;j=j+1) pixels[(6*8+j)*W+:W] <= wht_out[j*W+:W];
291         4'd7: for(j=0;j<8;j=j+1) pixels[(7*8+j)*W+:W] <= wht_out[j*W+:W];
292         4'd8: for(j=0;j<8;j=j+1) pixels[(j*8+0)*W+:W] <= wht_out[j*W+:W];
293         4'd9: for(j=0;j<8;j=j+1) pixels[(j*8+1)*W+:W] <= wht_out[j*W+:W];
294         4'd10: for(j=0;j<8;j=j+1) pixels[(j*8+2)*W+:W] <= wht_out[j*W+:W];
295         4'd11: for(j=0;j<8;j=j+1) pixels[(j*8+3)*W+:W] <= wht_out[j*W+:W];
296         4'd12: for(j=0;j<8;j=j+1) pixels[(j*8+4)*W+:W] <= wht_out[j*W+:W];
297         4'd13: for(j=0;j<8;j=j+1) pixels[(j*8+5)*W+:W] <= wht_out[j*W+:W];
298         4'd14: for(j=0;j<8;j=j+1) pixels[(j*8+6)*W+:W] <= wht_out[j*W+:W];
299         4'd15: for(j=0;j<8;j=j+1) pixels[(j*8+7)*W+:W] <= wht_out[j*W+:W];
300     endcase
301
302     if (wht_pass == 4'd15) begin
303         coeff_bgn <= 1'b1;
304         state <= WAIT;
305     end else begin
306         wht_pass <= wht_pass + 4'd1;
307     end
308 end
309
310 // -----
311 WAIT: begin
312     if (coeff_done) begin
313         if (coeff_n == 0) begin
314             state <= NEXT;
315         end else begin
316             emit_byte <= 4'd0;
317             emit_idx <= 3'd0;
318             emit_sub <= 2'd0;
319             emit_active <= 1'b1;

```

```

316                                     state <= EMIT;
317                                     end
318                             end
319     end
320
321     // -----
322     EMIT: begin
323         if (tx_ready & emit_active) begin
324             tx_valid <= 1'b1;
325
326             case (emit_byte)
327             4'd0: tx_data <= header_hi;
328             4'd1: tx_data <= header_lo;
329             default: begin
330                 case (emit_sub)
331                 2'd0: tx_data <= rc_byte;
332                 2'd1: tx_data <= q_hi;
333                 2'd2: begin
334                     tx_data <= q_lo;
335                     emit_idx <= emit_idx + 3'd1;
336
337                     end
338                 default: tx_data <= 8'd0;
339                 endcase
340                 emit_sub <= (emit_sub == 2'd2) ? 2'd0
341                                     : emit_sub + 2'd1;
342             end
343             endcase
344
345             emit_byte <= emit_byte + 4'd1;
346
347             if (emit_byte == 4'd1 + {1'b0, coeff_n} * 3) begin
348                 emit_active <= 1'b0;
349                 state <= NEXT;
350             end
351         end
352     end
353
354     // -----
355     NEXT: begin
356         if (bx == 6'd39) begin
357             bx <= 6'd0;
358             if (by == 5'd29) begin
359                 frame_done <= 1'b1;
360                 state <= IDLE;
361             end else begin
362                 by <= by + 5'd1;
363                 state <= LOAD;
364             end
365         end else begin
366             bx <= bx + 6'd1;
367             state <= LOAD;
368         end
369     end
370
371     default: state <= IDLE;
372     endcase
373 end
374 endmodule

```

fpga/src/rtl/bram_image_buffer.v

```
1  /*
2  * bram_image_buffer.v
3  *
4  * Single-clock dual-port RAM for reliable BSRAM inference
5  */
6
7  module bram_image_buffer
8  (
9      // Write port (SPI side)
10     input wire clk_wr,
11     input wire we,
12     input wire [14 : 0] addr_wr,
13     input wire [7 : 0] data_wr,
14     // Read port (LCD side)
15     input wire clk_rd,
16     // Ignored - using clk_wr for both
17     input wire [14 : 0] addr_rd,
18     output reg [7 : 0] data_rd
19 );
20
21     // 32KB RAM - 14.6x Tang Nano 9K BSRAMs, fits in 26 available
22     reg [7 : 0] mem[0 : 32767];
23     // Single clock for both ports
24     always @(posedge clk_wr) begin
25         if (we) begin
26             mem[addr_wr[14 : 0]] <= data_wr;
27         end
28         data_rd <= mem[addr_rd[14 : 0]];
29     end
30 endmodule
```

fpga/src/rtl/coeff.v

```

1  /*
2  * coeff.v -- Top-K coefficient extraction FSM
3  *
4  * Ports flattened for yosys: element [r][c] = flat[(r*8+c)*W +: W]
5  * Output arrays flattened: slot k = flat[k*N +: N]
6  */
7
8  module coeff
9      #(parameter W = 16, TOP_K = 5)
10     (input wire clk,
11      input wire rst,
12      input wire bgn,
13
14      input wire signed [W*64-1:0] c_i,
15
16      output reg [3*TOP_K-1:0] r_o,
17      output reg [3*TOP_K-1:0] c_o,
18      output reg signed [W*TOP_K-1:0] q_o,
19      output reg [2:0] n_out,
20      output reg done);
21
22     localparam [W-1:0] thresh = 16'd400;
23     localparam [3:0] q_shift = 4'd3;
24     localparam [3:0] seq_thresh = 4'd6;
25
26     reg [1:0] state;
27     localparam IDLE = 2'd0, SCAN = 2'd1, DONE = 2'd2;
28
29     reg [2:0] r_cnt;
30     reg [2:0] c_cnt;
31
32     wire skp = (r_cnt + c_cnt > seq_thresh) | (~|{r_cnt, c_cnt});
33
34     wire signed [W-1:0] coeff_val = c_i[(r_cnt*8+c_cnt)*W +: W];
35     wire [W-1:0] mag = (coeff_val ^ {W{coeff_val[W-1]}}) + coeff_val[W-1];
36     wire [W-1:0] sft = mag - thresh;
37     wire [W-1:0] q = sft >>> q_shift;
38     wire signed [W-1:0] q_sgn = (q ^ {W{coeff_val[W-1]}}) + coeff_val[W-1];
39
40     wire vld = ~skp & (mag > thresh) & (q != 0);
41
42     wire state_scan = (state == SCAN);
43     wire state_idle = (state == IDLE);
44
45     wire wrt_en = state_scan & vld;
46     wire clr = state_idle & bgn;
47
48     wire fill = wrt_en & (n_out < TOP_K);
49     wire repl = wrt_en & (n_out == TOP_K) & (mag > slot_abs[min_idx]);
50
51     reg [2:0] slot_r [0:TOP_K-1];
52     reg [2:0] slot_c [0:TOP_K-1];
53     reg signed [W-1:0] slot_q [0:TOP_K-1];
54     reg [W-1:0] slot_abs [0:TOP_K-1];
55     reg [2:0] min_idx;
56
57     // min rescan: find slot with smallest abs_q
58     wire [2:0] m01 = slot_abs[0] < slot_abs[1] ? 0 : 1;
59     wire [2:0] m23 = slot_abs[2] < slot_abs[3] ? 2 : 3;
60     wire [2:0] m013 = slot_abs[m01] < slot_abs[m23] ? m01 : m23;
61     wire [2:0] nmin = slot_abs[m013] < slot_abs[4] ? m013 : 4;
62
63     integer k;
64
65     always @(posedge clk) begin
66         // reset
67         state <= rst ? IDLE : state;

```

```

68         n_out <= rst ? 0 : n_out;
69         done <= (state == DONE);
70
71         // IDLE -> SCAN on bgn
72         state <= clr ? SCAN : state;
73         r_cnt <= r_cnt & {3{~clr}};
74         c_cnt <= c_cnt & {3{~clr}};
75         n_out <= n_out & {3{~clr}};
76
77         // SCAN: advance counters
78         c_cnt <= c_cnt + state_scan;
79         r_cnt <= r_cnt + (state_scan & c_cnt == 7);
80
81         // SCAN -> DONE
82         if (state_scan & r_cnt == 7 & c_cnt == 7)
83             state <= DONE;
84
85         // DONE -> IDLE
86         if (state == DONE)
87             state <= IDLE;
88
89         // top-K: fill empty slot
90         slot_r[n_out] <= fill ? r_cnt : slot_r[n_out];
91         slot_c[n_out] <= fill ? c_cnt : slot_c[n_out];
92         slot_q[n_out] <= fill ? q_sgn : slot_q[n_out];
93         slot_abs[n_out] <= fill ? mag : slot_abs[n_out];
94         n_out <= fill ? n_out + 1 : n_out;
95
96         // top-K: replace min slot
97         slot_r[min_idx] <= repl ? r_cnt : slot_r[min_idx];
98         slot_c[min_idx] <= repl ? c_cnt : slot_c[min_idx];
99         slot_q[min_idx] <= repl ? q_sgn : slot_q[min_idx];
100        slot_abs[min_idx] <= repl ? mag : slot_abs[min_idx];
101
102        // update min_idx after any write
103        min_idx <= (fill | repl) ? nmin : min_idx;
104
105        // copy slots to flattened outputs in DONE
106        if (state == DONE) begin
107            for (k = 0; k < TOP_K; k = k + 1) begin
108                r_o[k*3 +: 3] <= slot_r[k];
109                c_o[k*3 +: 3] <= slot_c[k];
110                q_o[k*W +: W] <= slot_q[k];
111            end
112        end
113    end
114endmodule

```

fpga/src/rtl/esp_interface.v

```

1 //
2 // esp_interface.v
3 //
4 // SPI slave - Mode 0 (CPOL=0 CPHA=0): sample rising, shift falling
5 // Branchless datapath: AND-mask / OR-combine, if only for async reset
6 //
7
8 module esp_interface
9     (input wire clk,
10      input wire rst_n,
11
12      input wire esp_mosi,
13      output reg esp_miso,
14      input wire esp_sclk,
15      input wire esp_cs_n,
16
17      output reg [7 : 0] rx_data,
18      output reg rx_valid,
19      input wire rx_ready,
20
21      input wire [7 : 0] tx_data,
22      output wire tx_next,
23
24      output wire cs_active_sync // synchronized CS active level for top.v
25 );
26
27     // =====
28     // Synchronizers - 3-stage sclk/cs, 2-stage mosi
29     // Pure shift, no branches
30     // =====
31     reg [2 : 0] sclk_sync, cs_sync;
32     reg [1 : 0] mosi_sync;
33
34     always @(posedge clk or negedge rst_n) begin
35         if (!rst_n) begin
36             sclk_sync <= 3'b000;
37             cs_sync <= 3'b111;
38             mosi_sync <= 2'b00;
39         end else begin
40             sclk_sync <= {sclk_sync[1 : 0], esp_sclk};
41             cs_sync <= {cs_sync[1 : 0], esp_cs_n};
42             mosi_sync <= {mosi_sync[0], esp_mosi};
43         end
44     end
45
46     // edge / level detection - combinational
47     wire sclk_rise = (sclk_sync[2 : 1] == 2'b01);
48     wire sclk_fall = (sclk_sync[2 : 1] == 2'b10);
49     wire cs_active = ~cs_sync[2];
50
51     assign cs_active_sync = cs_active;
52
53     // =====
54     // Shift register + bit counter
55     //
56     // On sclk_rise & cs_active: shift in mosi, increment count
57     // On ~cs_active: hold zeros (AND-mask clears)
58     // byte_complete when bit_count == 7 on a rising edge
59     //
60     // shift_next: shift in new bit, AND-masked by (sclk_rise & cs_active)
61     //                held at 0 when ~cs_active
62     // count_next: (count + 1) on shift, AND-masked to wrap at 8,
63     //                zeroed when ~cs_active
64     // =====
65     reg [7 : 0] shift_reg;
66     reg [2 : 0] bit_count;
67     reg byte_ready;

```

```

68
69 wire shifting = sclk_rise & cs_active;
70 wire byte_done = shifting & (bit_count == 3'd7);
71 wire [7 : 0] shifted_in = {shift_reg[6 : 0], mosi_sync[1]};
72
73 // AND-mask: shifting selects new shifted value, ~shifting holds
74 // current ~cs_active zeros everything (overrides shifting since
75 // shifting requires cs_active)
76 wire [7 : 0] shift_next = (shifted_in & {8{shifting}}) |
77                             (shift_reg & {8{~shifting & cs_active}});
78 wire [2 : 0] count_inc = (bit_count + 3'd1) &
79                             {3{~byte_done}}; // wraps to 0 on byte_done
80 wire [2 : 0] count_next = (count_inc & {3{shifting}}) |
81                             (bit_count & {3{~shifting}});
82 wire [2 : 0] count_out = count_next &
83                             {3{cs_active}}; // zero when CS deasserted
84
85 always @(posedge clk or negedge rst_n) begin
86     if (!rst_n) begin
87         shift_reg <= 8'h00;
88         bit_count <= 3'd0;
89         byte_ready <= 1'b0;
90     end else begin
91         shift_reg <= shift_next & {8{cs_active}};
92         bit_count <= count_out;
93         byte_ready <= byte_done;
94     end
95 end
96
97 // =====
98 // Output latch - rx_data holds on byte_ready, rx_valid strobes
99 //
100 // data: AND-mask selects new capture vs hold
101 // valid: just byte_ready delayed one cycle (already is - byte_ready
102 //         is registered, so rx_valid = byte_ready is the strobe)
103 // =====
104 wire [7 : 0] captured = {shift_reg[6 : 0],
105                             mosi_sync[1]}; // include last bit
106 wire [7 : 0] data_next = (captured & {8{byte_ready}}) |
107                             (rx_data & {8{~byte_ready}});
108
109 always @(posedge clk or negedge rst_n) begin
110     if (!rst_n) begin
111         rx_data <= 8'h00;
112         rx_valid <= 1'b0;
113     end else begin
114         rx_data <= data_next;
115         rx_valid <= byte_ready;
116     end
117 end
118
119 // =====
120 // MISO TX - shift out tx_data, MSB first on falling SCLK
121 //
122 // tx_next pulses on byte_done so top.v advances to next byte
123 // Preload from tx_data on CS assert and each byte boundary
124 // =====
125 reg [7 : 0] tx_sr;
126
127 assign tx_next = byte_done;
128
129 wire tx_reload = byte_done | (~cs_active);
130
131 always @(posedge clk or negedge rst_n) begin
132     if (!rst_n) begin
133         tx_sr <= 8'd0;
134         esp_miso <= 1'b0;
135     end else begin
136         if (tx_reload)
137             tx_sr <= tx_data;

```



```
138         else if (sclk_fall & cs_active)
139             tx_sr <= {tx_sr[6 : 0], 1'b0};
140
141             esp_miso <= cs_active ? tx_sr[7] : 1'b0;
142         end
143     end
144 endmodule
```

fpga/src/rtl/gowin_rpll.v

```
1 //Copyright (C)2014-2022 Gowin Semiconductor Corporation.
2 //All rights reserved.
3 //File Title: IP file
4 //GOWIN Version: V1.9.8.07
5 //Part Number: GW1NR-LV9QN88PC6/I5
6 //Device: GW1NR-9C
7 //Created Time: Wed Aug 17 00:30:59 2022
8
9 module Gowin_rPLL (clkout, clkoutp, clkin);
10
11 output clkout;
12 output clkoutp;
13 input clkin;
14
15 wire lock_o;
16 wire clkoutd_o;
17 wire clkoutd3_o;
18 wire gw_vcc;
19 wire gw_gnd;
20
21 assign gw_vcc = 1'b1;
22 assign gw_gnd = 1'b0;
23
24 rPLL rpll_inst (
25     .CLKOUT(clkout),
26     .LOCK(lock_o),
27     .CLKOUTP(clkoutp),
28     .CLKOUTD(clkoutd_o),
29     .CLKOUTD3(clkoutd3_o),
30     .RESET(gw_gnd),
31     .RESET_P(gw_gnd),
32     .CLKIN(clkin),
33     .CLKFB(gw_gnd),
34     .FBDSEL({gw_gnd,gw_gnd,gw_gnd,gw_gnd,gw_gnd,gw_gnd}),
35     .IDSEL({gw_gnd,gw_gnd,gw_gnd,gw_gnd,gw_gnd,gw_gnd}),
36     .ODSEL({gw_gnd,gw_gnd,gw_gnd,gw_gnd,gw_gnd,gw_gnd}),
37     .PSDA({gw_gnd,gw_gnd,gw_gnd,gw_gnd,gw_gnd,gw_gnd}),
38     .DUTYDA({gw_gnd,gw_gnd,gw_gnd,gw_gnd,gw_gnd,gw_gnd}),
39     .FDLY({gw_vcc,gw_vcc,gw_vcc,gw_vcc,gw_vcc,gw_vcc})
40 );
41
42 defparam rpll_inst.FCLKIN = "27";
43 defparam rpll_inst.DYN_IDIV_SEL = "false";
44 // 81 Mhz, LATENCY=3
45 defparam rpll_inst.FBDIV_SEL = 2;
46 defparam rpll_inst.IDIV_SEL = 0;
47 defparam rpll_inst.ODIV_SEL = 8;
48
49 // 102.6 Mhz, LATENCY=4
50 //defparam rpll_inst.FBDIV_SEL = 18;
51 //defparam rpll_inst.IDIV_SEL = 4;
52 //defparam rpll_inst.ODIV_SEL = 8;
53
54 // Other possible clock speed:
55 // 51 Mhz
56 //defparam rpll_inst.IDIV_SEL = 8;
57 //defparam rpll_inst.FBDIV_SEL = 16;
58 //defparam rpll_inst.ODIV_SEL = 8;
59
60 // 74.25 Mhz, 720p pixel clock
61 //defparam rpll_inst.IDIV_SEL = 3;
62 //defparam rpll_inst.FBDIV_SEL = 10;
63 //defparam rpll_inst.ODIV_SEL = 8;
64
65 // 11 Mhz
66 //defparam rpll_inst.IDIV_SEL = 7;
67 //defparam rpll_inst.FBDIV_SEL = 2;
```

```

68 //defparam rp11_inst.ODIV_SEL = 48;
69
70
71 defparam rp11_inst.DYN_FBDIV_SEL = "false";
72 defparam rp11_inst.DYN_ODIV_SEL = "false";
73 defparam rp11_inst.PSDA_SEL = "0100";
74 defparam rp11_inst.DYN_DA_EN = "false";
75 defparam rp11_inst.DUTYDA_SEL = "1000";
76 defparam rp11_inst.CLKOUT_FT_DIR = 1'b1;
77 defparam rp11_inst.CLKOUTP_FT_DIR = 1'b1;
78 defparam rp11_inst.CLKOUT_DLY_STEP = 0;
79 defparam rp11_inst.CLKOUTP_DLY_STEP = 0;
80 defparam rp11_inst.CLKFB_SEL = "internal";
81 defparam rp11_inst.CLKOUT_BYPASS = "false";
82 defparam rp11_inst.CLKOUTP_BYPASS = "false";
83 defparam rp11_inst.CLKOUTD_BYPASS = "false";
84 defparam rp11_inst.DYN_SDIV_SEL = 2;
85 defparam rp11_inst.CLKOUTD_SRC = "CLKOUT";
86 defparam rp11_inst.CLKOUTD3_SRC = "CLKOUT";
87 defparam rp11_inst.DEVICE = "GW1NR-9C";
88
89 endmodule //Gowin_rPLL

```

fpga/src/rtl/i2c_gpio_expander.v

```
1  /*
2  * i2c_gpio_expander.v
3  *
4  * Trail Camera IV - I2C Slave GPIO Expander
5  * PCA9534-compatible register map
6  *
7  * Register Map:
8  *   0x00  OUTPUT_PORT    Drive gpio_out[7:0] (output pins only, gated by DIR)
9  *   0x01  INPUT_PORT     Read gpio_in[7:0] with invert mask applied (read-only)
10 *   0x02  DIR_REG        1=input, 0=output (default 0xFF - all inputs on
11 * reset) 0x03  INVERT_REG Input polarity invert mask (default 0x00)
12 *
13 * Usage for trail camera:
14 *   gpio_in[2:0]  -> button_L, button_R, button_S (pins 84, 83, 85)
15 *   gpio_in[7:3]  -> unused (read as 0)
16 *   gpio_out[3:0] -> step_1, step_2, step_3, step_4 (pins 25-28)
17 *   DIR_REG       -> ESP32 writes 0xF0 on boot:
18 *                   bits[7:4]=1 unused inputs, bits[3:0]=0 stepper outputs
19 *                   i.e. 0b11110000 = 0xF0
20 *
21 * Electrical:
22 *   SDA -> pin 82 (IOT11A) LVCMOS18, external 4.7k pull-up to 3.3V
23 *   SCL -> pin 81 (IOT11B) LVCMOS18, external 4.7k pull-up to 3.3V
24 *   Tristate buffer in top.v: assign i2c_sda = sda_oe ? 1'b0 : 1'bz
25 *
26 * I2C address: 7'h27 (configurable)
27 */
28
29 module i2c_gpio_expander
30     #(parameter DEVICE_ADDR = 7'h27)
31     (input wire clk,
32      input wire rst_n,
33
34      input wire scl_i,
35      input wire sda_i,
36      output reg sda_oe,
37      // 1 = drive SDA low; 0 = release (pull-up handles high)
38
39      output reg [7 : 0] gpio_out,
40      // output register - wire to physical pins via dir gate
41      output reg [7 : 0] gpio_dir,
42      // direction: 1=input, 0=output
43      output reg [7 : 0] gpio_invert,
44      // input invert mask
45      input wire [7 : 0]
46          gpio_in // physical input pin values (registered externally)
47     );
48
49     // =====
50     // 3-stage synchronizers - metastability on async I2C lines
51     // =====
52     reg [2 : 0] scl_sync;
53     reg [2 : 0] sda_sync;
54
55     always @(posedge clk or negedge rst_n) begin
56         if (!rst_n) begin
57             scl_sync <= 3'b111;
58             sda_sync <= 3'b111;
59         end else begin
60             scl_sync <= {scl_sync[1 : 0], scl_i};
61             sda_sync <= {sda_sync[1 : 0], sda_i};
62         end
63     end
64
65     wire scl_s = scl_sync[1];
66     wire sda_s = sda_sync[1];
67     wire scl_rise = (scl_sync[2 : 1] == 2'b01);
```

```

68 wire scl_fall = (scl_sync[2 : 1] == 2'b10);
69 wire start_det = scl_sync[1] &
70                 (sda_sync[2 : 1] == 2'b10); // SDA falls, SCL high
71 wire stop_det = scl_sync[1] &
72                 (sda_sync[2 : 1] == 2'b01); // SDA rises, SCL high
73
74 // =====
75 // States
76 // =====
77 localparam [3 : 0] S_IDLE = 4'd0,
78                   S_ADDR = 4'd1, // receive 7-bit addr + R/W bit
79                   S_ADDR_ACK = 4'd2, // drive ACK/NACK
80                   S_REG = 4'd3, // receive register pointer
81                   S_REG_ACK = 4'd4,
82                   S_WRITE = 4'd5, // receive data byte
83                   S_WRITE_ACK = 4'd6,
84                   S_READ = 4'd7, // transmit data byte
85                   S_READ_ACK = 4'd8; // wait for master ACK/NACK
86
87 reg [3 : 0] state;
88 reg [7 : 0] shift_reg; // rx shift register (MSB first)
89 reg [2 : 0] bit_cnt; // counts 0..7
90 reg rw_bit; // latched R/W from address phase
91 reg [7 : 0]
92   reg_addr; // current register pointer (auto-increments on burst)
93 reg [7 : 0] tx_byte; // byte being clocked out during reads
94
95 // =====
96 // Register read mux - combinational
97 // =====
98 function [7 : 0] read_reg;
99     input [7 : 0] addr;
100     begin
101         case (addr)
102             8'h00:
103                 read_reg = gpio_out;
104             8'h01:
105                 read_reg =
106                     gpio_in ^
107                     gpio_invert; // invert applied, dir
108                                 // not masked (master
109                                 // knows dir)
110             8'h02:
111                 read_reg = gpio_dir;
112             8'h03:
113                 read_reg = gpio_invert;
114             default:
115                 read_reg = 8'hFF;
116         endcase
117     end
118 endfunction
119
120 // =====
121 // State machine
122 //
123 // Timing contract:
124 //   ACK/data bits driven on scl_fall (low phase), stable when SCL
125 //   rises RX bits sampled on scl_rise START/STOP always override
126 //   current state
127 // =====
128 always @(posedge clk or negedge rst_n) begin
129     if (!rst_n) begin
130         state <= S_IDLE;
131         sda_oe <= 1'b0;
132         gpio_out <= 8'h00;
133         gpio_dir <=
134             8'hFF; // all inputs on reset - safe default
135         gpio_invert <= 8'h00;
136         shift_reg <= 8'h00;
137         bit_cnt <= 3'd0;

```

```

138         rw_bit <= 1'b0;
139         reg_addr <= 8'h00;
140         tx_byte <= 8'h00;
141     end else begin
142
143         // START/STOP override all states - always check first
144         if (start_det) begin
145             state <= S_ADDR;
146             bit_cnt <= 3'd0;
147             sda_oe <= 1'b0;
148         end else if (stop_det) begin
149             state <= S_IDLE;
150             sda_oe <= 1'b0;
151         end else begin
152             case (state)
153
154                 // -----
155                 S_IDLE: begin
156                     sda_oe <= 1'b0;
157                 end
158
159                 // -----
160                 // Shift in address byte (7 addr bits +
161                 // R/W)
162                 S_ADDR: begin
163                     if (scl_rise) begin
164                         shift_reg <=
165                             {shift_reg[6 : 0],
166                             sda_s};
167                     if (bit_cnt == 3'd7)
168                         begin
169                             rw_bit <=
170                                 sda_s; // last bit = R/W
171                             bit_cnt <=
172                                 3'd0;
173                             state <=
174                                 S_ADDR_ACK;
175                         end else
176                             bit_cnt <=
177                                 bit_cnt +
178                                 1'b1;
179                     end
180                 end
181
182                 // -----
183                 // Drive ACK low during SCL low phase
184                 // after address. Hold sda_oe through
185                 // SCL high - releasing here would let
186                 // SDA rise while SCL is still high,
187                 // which stop_det would misread as a
188                 // STOP condition. The destination
189                 // state (S_REG / S_READ) releases
190                 // sda_oe on its first scl_fall (SCL
191                 // low), so no false STOP is generated.
192                 S_ADDR_ACK: begin
193                     if (scl_fall) begin
194                         if (shift_reg[7 : 1] ==
195                             DEVICE_ADDR) begin
196                             sda_oe <=
197                                 1'b1; // ACK
198                                 // -
199                                 // pull
200                                 // low
201                             tx_byte <=
202                                 read_reg(
203                                     reg_addr);
204                         end else begin
205                             sda_oe <=
206                                 1'b0; // NACK
207                                 // -

```

```

208                                     // not
209                                     // our
210                                     // address
211                                     state <=
212                                         S_IDLE;
213                                     end
214     end
215     if (scl_rise &
216         (shift_reg[7 : 1] ==
217         DEVICE_ADDR)) begin
218         // Do NOT release
219         // sda_oe here - hold
220         // ACK low through SCL
221         // high; next state
222         // releases on its
223         // first scl_fall
224         state <= rw_bit
225                                     ? S_READ
226                                     : S_REG;
227     end
228 end
229
230 // -----
231 S_REG: begin
232     if (scl_fall)
233         sda_oe <=
234             1'b0; // ensure
235                 // released
236     if (scl_rise) begin
237         shift_reg <=
238             {shift_reg[6 : 0],
239             sda_s};
240         if (bit_cnt == 3'd7)
241             begin
242                 reg_addr <=
243                     {shift_reg
244                     [6
245                     : 0],
246                     sda_s};
247                 bit_cnt <=
248                     3'd0;
249                 state <=
250                     S_REG_ACK;
251             end else
252                 bit_cnt <=
253                     bit_cnt +
254                     1'b1;
255     end
256 end
257
258 // -----
259 S_REG_ACK: begin
260     if (scl_fall)
261         sda_oe <= 1'b1; // ACK
262     if (scl_rise) begin
263         // Hold sda_oe -
264         // S_WRITE releases on
265         // its first scl_fall
266         state <= S_WRITE;
267     end
268 end
269
270 // -----
271 S_WRITE: begin
272     if (scl_fall) sda_oe <= 1'b0;
273     if (scl_rise) begin
274         shift_reg <=
275             {shift_reg[6 : 0],
276             sda_s};
277         if (bit_cnt == 3'd7)

```

```

278         begin
279             bit_cnt <=
280                 3'd0;
281             state <=
282                 S_WRITE_ACK;
283         end else
284             bit_cnt <=
285                 bit_cnt +
286                 1'b1;
287         end
288     end
289
290     // -----
291     // Latch write on scl_fall after 8th
292     // bit
293     S_WRITE_ACK: begin
294         if (scl_fall) begin
295             // commit the byte to
296             // the addressed
297             // register
298             case (reg_addr)
299                 8'h00:
300                     gpio_out <=
301                         shift_reg;
302                 8'h02:
303                     gpio_dir <=
304                         shift_reg;
305                 8'h03:
306                     gpio_invert <=
307                         shift_reg;
308                 default:
309                     ; // 0x01
310                     // INPUT_PORT
311                     // read-only
312             endcase
313             reg_addr <=
314                 reg_addr +
315                 1'b1; // auto-increment
316                     // for burst
317                     // writes
318             sda_oe <= 1'b1; // ACK
319         end
320         if (scl_rise) begin
321             // Hold sda_oe -
322             // S_WRITE releases on
323             // its first scl_fall
324             state <= S_WRITE;
325         end
326     end
327
328     // -----
329     // Clock out tx_byte MSB first
330     // Drive on scl_fall, stable during SCL
331     // high
332     S_READ: begin
333         if (scl_fall) begin
334             sda_oe <=
335                 ~tx_byte
336                 [7]; // 0=drive
337                     // low,
338                     // 1=release
339             tx_byte <=
340                 {tx_byte[6 : 0],
341                 1'b1}; // shift,
342                     // pad with
343                     // 1
344             if (bit_cnt == 3'd7)
345                 begin
346                     bit_cnt <=
347                         3'd0;

```



```

348                                     state <=
349                                     S_READ_ACK;
350                                 end else
351                                     bit_cnt <=
352                                     bit_cnt +
353                                     1'b1;
354                                 end
355                             end
356                         end
357
358                         // -----
359                         // Sample master ACK/NACK on scl_rise
360                         // ACK (SDA low) = master wants more
361                         // bytes NACK (SDA high) = master done
362                         S_READ_ACK: begin
363                             if (scl_fall)
364                                 sda_oe <=
365                                 1'b0; // release
366                                     // SDA for
367                                     // master to
368                                     // drive
369                             if (scl_rise) begin
370                                 if (!sda_s) begin
371                                     // Master ACK -
372                                     // load next
373                                     // register and
374                                     // continue
375                                     reg_addr <=
376                                     reg_addr +
377                                     1'b1;
378                                     tx_byte <=
379                                     read_reg(
380                                     reg_addr +
381                                     1'b1);
382                                     state <=
383                                     S_READ;
384                                 end else
385                                     state <=
386                                     S_IDLE; // Master NACK - done
387                             end
388                         end
389                     default:
390                         state <= S_IDLE;
391                 endcase
392             end
393         end
394     end
395 endmodule

```

fpga/src/rtl/lcd_controller.v

```

1 //
2 // lcd_controller.v
3 // RGB LCD controller - 480x272 @ 13.5MHz (27MHz / 2)
4 //
5 // Style: if(!rst_n) for async reset (yosys requires this pattern),
6 //         AND-mask / OR-combine / XOR everywhere else. No if/else in datapath.
7 //
8
9 module lcd_controller
10     (input wire clk,
11      // 27MHz
12      input wire rst_n,
13      input wire btn,
14
15      output reg [14 : 0] bram_addr,
16      input wire [7 : 0] bram_data,
17
18      output wire lcd_clk,
19      output reg lcd_hsync,
20      output reg lcd_vsync,
21      output reg lcd_de,
22      output reg [4 : 0] lcd_r,
23      output reg [5 : 0] lcd_g,
24      output reg [4 : 0] lcd_b);
25
26     localparam H_ACTIVE = 480, H_FRONT = 2, H_SYNC = 41, H_TOTAL = 525;
27     localparam V_ACTIVE = 272, V_FRONT = 2, V_SYNC = 10, V_TOTAL = 286;
28     // =====
29     // Pixel clock: toggle → 13.5 MHz
30     // =====
31     reg pclk;
32     always @(posedge clk or negedge rst_n)
33         if (!rst_n)
34             pclk <= 1'b0;
35         else
36             pclk <= ~pclk;
37
38     assign lcd_clk = pclk;
39
40     // =====
41     // H/V counters
42     // (count + 1) & {N{~last}} → wraps to 0 with no branch
43     // =====
44     reg [9 : 0] h_count, v_count;
45     wire h_last = (h_count == H_TOTAL - 1);
46     wire v_last = (v_count == V_TOTAL - 1);
47
48     wire [9 : 0] h_next = (h_count + 10'd1) & {10{~h_last}};
49     wire [9 : 0] v_next = (v_count + 10'd1) & {10{~v_last}};
50
51     // pclk gates h; pclk & h_last gates v
52     // hold = current & mask; load = next & ~mask; result = hold | load
53     wire [9 : 0] h_load = (h_next & {10{pclk}}) | (h_count & {10{~pclk}});
54     wire v_en = pclk & h_last;
55     wire [9 : 0] v_load = (v_next & {10{v_en}}) | (v_count & {10{~v_en}});
56
57     always @(posedge clk or negedge rst_n) begin
58         if (!rst_n) begin
59             h_count <= 10'd0;
60             v_count <= 10'd0;
61         end else begin
62             h_count <= h_load;
63             v_count <= v_load;
64         end
65     end
66
67     // =====

```

```

68 // Derived signals - pure combinational
69 // =====
70 wire visible = (h_count < H_ACTIVE) & (v_count < V_ACTIVE);
71 wire frame_start = ~|h_count & ~|v_count;
72 wire h_sync = (h_count >= H_ACTIVE + H_FRONT) &
73               (h_count < H_ACTIVE + H_FRONT + H_SYNC);
74 wire v_sync = (v_count >= V_ACTIVE + V_FRONT) &
75               (v_count < V_ACTIVE + V_FRONT + V_SYNC);
76
77 // =====
78 // Register sync/DE - AND-mask select: →pclknew, ~→pclkhold
79 // =====
80 wire next_hsync = (h_sync & pclk) | (lcd_hsync & ~pclk);
81 wire next_vsync = (v_sync & pclk) | (lcd_vsync & ~pclk);
82 wire next_de = (visible & pclk) | (lcd_de & ~pclk);
83
84 always @(posedge clk or negedge rst_n) begin
85     if (!rst_n) begin
86         lcd_hsync <= 1'b0;
87         lcd_vsync <= 1'b0;
88         lcd_de <= 1'b0;
89     end else begin
90         lcd_hsync <= next_hsync;
91         lcd_vsync <= next_vsync;
92         lcd_de <= next_de;
93     end
94 end
95
96 // =====
97 // BRAM address
98 // +visible increments only in active region
99 // & ~frame_start zeros at frame boundary
100 // & pclk / ~pclk selects new vs hold
101 // =====
102 wire [14 : 0] addr_inc =
103     (bram_addr + {14'd0, visible}) & {15{~frame_start}};
104 wire [14 : 0] next_addr =
105     (addr_inc & {15{pclk}}) | (bram_addr & {15{~pclk}});
106
107 always @(posedge clk or negedge rst_n)
108     if (!rst_n)
109         bram_addr <= 15'd0;
110     else
111         bram_addr <= next_addr;
112
113 // =====
114 // Button debounce
115 // XOR detects change. AND-mask zeros counter on match.
116 // Stable latch via SDD501 XOR trick.
117 // =====
118 reg [19 : 0] db_cnt;
119 reg btn_s1, btn_s2, btn_stable, btn_prev;
120
121 wire btn_changed = btn_s2 ^ btn_stable;
122 wire db_done = (db_cnt >= 20'd540000);
123
124 always @(posedge clk or negedge rst_n) begin
125     if (!rst_n) begin
126         btn_s1 <= 1'b1;
127         btn_s2 <= 1'b1;
128         btn_stable <= 1'b1;
129         btn_prev <= 1'b1;
130         db_cnt <= 20'd0;
131     end else begin
132         btn_s1 <= btn;
133         btn_s2 <= btn_s1;
134         btn_prev <= btn_stable;
135         db_cnt <= (db_cnt + {19'd0, btn_changed}) &
136                 {20{btn_changed}};
137         btn_stable <=

```

```

138             btn_stable ^ ((btn_stable ^ btn_s2) & db_done);
139         end
140     end
141
142     wire btn_pressed = btn_prev & ~btn_stable;
143
144     // =====
145     // Pattern selector - increment masked by btn_pressed
146     // Start at 7 (BRAM display) so SPI data shows on boot
147     // =====
148     reg [2 : 0] pat;
149     always @(posedge clk or negedge rst_n)
150         if (!rst_n)
151             pat <= 3'd7;
152         else
153             pat <= pat + {2'd0, btn_pressed};
154
155     // =====
156     // Pixel output - fully branchless
157     //
158     // All 8 patterns computed as parallel wires.
159     // AND-mask with one-hot select, OR-combine.
160     // AND-mask with visibility. AND-mask with pclk for hold.
161     // =====
162
163     // one-hot decode
164     wire [7 : 0] sel;
165     assign sel[0] = (pat == 3'd0);
166     assign sel[1] = (pat == 3'd1);
167     assign sel[2] = (pat == 3'd2);
168     assign sel[3] = (pat == 3'd3);
169     assign sel[4] = (pat == 3'd4);
170     assign sel[5] = (pat == 3'd5);
171     assign sel[6] = (pat == 3'd6);
172     assign sel[7] = (pat == 3'd7);
173
174     // visibility AND masks
175     wire [4 : 0] vis5 = {5{visible}};
176     wire [5 : 0] vis6 = {6{visible}};
177
178     // --- p0: solid red ---
179     wire [4 : 0] p0_r = {5{sel[0]}};
180     wire [5 : 0] p0_g = 6'd0;
181     wire [4 : 0] p0_b = 5'd0;
182
183     // --- p1: solid green ---
184     wire [4 : 0] p1_r = 5'd0;
185     wire [5 : 0] p1_g = {6{sel[1]}};
186     wire [4 : 0] p1_b = 5'd0;
187
188     // --- p2: solid blue ---
189     wire [4 : 0] p2_r = 5'd0;
190     wire [5 : 0] p2_g = 6'd0;
191     wire [4 : 0] p2_b = {5{sel[2]}};
192
193     // --- p3: solid white ---
194     wire [4 : 0] p3_r = {5{sel[3]}};
195     wire [5 : 0] p3_g = {6{sel[3]}};
196     wire [4 : 0] p3_b = {5{sel[3]}};
197
198     // --- p4: vertical color bars ---
199     wire [2 : 0] bar = h_count[6 : 4];
200     wire bar_r =
201         (bar == 3'd0) | (bar == 3'd3) | (bar == 3'd4) | (bar == 3'd6);
202     wire bar_g =
203         (bar == 3'd1) | (bar == 3'd3) | (bar == 3'd5) | (bar == 3'd6);
204     wire bar_b =
205         (bar == 3'd2) | (bar == 3'd4) | (bar == 3'd5) | (bar == 3'd6);
206
207     wire [4 : 0] p4_r = {5{bar_r & sel[4]}};

```

```

208     wire [5 : 0] p4_g = {6{bar_g & sel[4]}};
209     wire [4 : 0] p4_b = {5{bar_b & sel[4]}};
210
211     // --- p5: horizontal red gradient ---
212     wire [4 : 0] p5_r = h_count[8 : 4] & {5{sel[5]}};
213     wire [5 : 0] p5_g = 6'd0;
214     wire [4 : 0] p5_b = 5'd0;
215
216     // --- p6: checkerboard ---
217     wire chk = h_count[5] ^ v_count[5];
218     wire [4 : 0] p6_r = {5{chk & sel[6]}};
219     wire [5 : 0] p6_g = {6{chk & sel[6]}};
220     wire [4 : 0] p6_b = {5{chk & sel[6]}};
221
222     // --- p7: BRAM grayscale ---
223     wire [4 : 0] p7_r = bram_data[7 : 3] & {5{sel[7]}};
224     wire [5 : 0] p7_g = bram_data[7 : 2] & {6{sel[7]}};
225     wire [4 : 0] p7_b = bram_data[7 : 3] & {5{sel[7]}};
226
227     // --- OR-combine all patterns, AND with visibility ---
228     wire [4 : 0] raw_r = (p0_r | p1_r | p2_r | p3_r | p4_r | p5_r | p6_r |
229                          p7_r) &
230                          vis5;
231     wire [5 : 0] raw_g = (p0_g | p1_g | p2_g | p3_g | p4_g | p5_g | p6_g |
232                          p7_g) &
233                          vis6;
234     wire [4 : 0] raw_b = (p0_b | p1_b | p2_b | p3_b | p4_b | p5_b | p6_b |
235                          p7_b) &
236                          vis5;
237
238     // --- pclk AND-mask: new on pclk, hold on ~pclk ---
239     wire [4 : 0] next_r = (raw_r & {5{pclk}}) | (lcd_r & {5{~pclk}});
240     wire [5 : 0] next_g = (raw_g & {6{pclk}}) | (lcd_g & {6{~pclk}});
241     wire [4 : 0] next_b = (raw_b & {5{pclk}}) | (lcd_b & {5{~pclk}});
242
243     always @(posedge clk or negedge rst_n) begin
244         if (!rst_n) begin
245             lcd_r <= 5'd0;
246             lcd_g <= 6'd0;
247             lcd_b <= 5'd0;
248         end else begin
249             lcd_r <= next_r;
250             lcd_g <= next_g;
251             lcd_b <= next_b;
252         end
253     end
254 endmodule

```

fpga/src/rtl/psram-tang-nano-9k/src/gowin_rpll/gowin_rpll.v

```
1 //Copyright (C)2014-2022 Gowin Semiconductor Corporation.
2 //All rights reserved.
3 //File Title: IP file
4 //GOWIN Version: V1.9.8.07
5 //Part Number: GW1NR-LV9QN88PC6/I5
6 //Device: GW1NR-9C
7 //Created Time: Wed Aug 17 00:30:59 2022
8
9 module Gowin_rPLL (clkout, clkoutp, clkin);
10
11 output clkout;
12 output clkoutp;
13 input clkin;
14
15 wire lock_o;
16 wire clkoutd_o;
17 wire clkoutd3_o;
18 wire gw_vcc;
19 wire gw_gnd;
20
21 assign gw_vcc = 1'b1;
22 assign gw_gnd = 1'b0;
23
24 rPLL rpll_inst (
25     .CLKOUT(clkout),
26     .LOCK(lock_o),
27     .CLKOUTP(clkoutp),
28     .CLKOUTD(clkoutd_o),
29     .CLKOUTD3(clkoutd3_o),
30     .RESET(gw_gnd),
31     .RESET_P(gw_gnd),
32     .CLKIN(clkin),
33     .CLKFB(gw_gnd),
34     .FBDSEL({gw_gnd,gw_gnd,gw_gnd,gw_gnd,gw_gnd,gw_gnd}),
35     .IDSEL({gw_gnd,gw_gnd,gw_gnd,gw_gnd,gw_gnd,gw_gnd}),
36     .ODSEL({gw_gnd,gw_gnd,gw_gnd,gw_gnd,gw_gnd,gw_gnd}),
37     .PSDA({gw_gnd,gw_gnd,gw_gnd,gw_gnd,gw_gnd}),
38     .DUTYDA({gw_gnd,gw_gnd,gw_gnd,gw_gnd}),
39     .FDLY({gw_vcc,gw_vcc,gw_vcc,gw_vcc})
40 );
41
42 defparam rpll_inst.FCLKIN = "27";
43 defparam rpll_inst.DYN_IDIV_SEL = "false";
44 // 81 Mhz, LATENCY=3
45 defparam rpll_inst.FBDIV_SEL = 2;
46 defparam rpll_inst.IDIV_SEL = 0;
47 defparam rpll_inst.ODIV_SEL = 8;
48
49 // 102.6 Mhz, LATENCY=4
50 //defparam rpll_inst.FBDIV_SEL = 18;
51 //defparam rpll_inst.IDIV_SEL = 4;
52 //defparam rpll_inst.ODIV_SEL = 8;
53
54 // Other possible clock speed:
55 // 51 Mhz
56 //defparam rpll_inst.IDIV_SEL = 8;
57 //defparam rpll_inst.FBDIV_SEL = 16;
58 //defparam rpll_inst.ODIV_SEL = 8;
59
60 // 74.25 Mhz, 720p pixel clock
61 //defparam rpll_inst.IDIV_SEL = 3;
62 //defparam rpll_inst.FBDIV_SEL = 10;
63 //defparam rpll_inst.ODIV_SEL = 8;
64
65 // 11 Mhz
66 //defparam rpll_inst.IDIV_SEL = 7;
67 //defparam rpll_inst.FBDIV_SEL = 2;
```

```

68 //defparam rp11_inst.ODIV_SEL = 48;
69
70
71 defparam rp11_inst.DYN_FBDIV_SEL = "false";
72 defparam rp11_inst.DYN_ODIV_SEL = "false";
73 defparam rp11_inst.PSDA_SEL = "0100";
74 defparam rp11_inst.DYN_DA_EN = "false";
75 defparam rp11_inst.DUTYDA_SEL = "1000";
76 defparam rp11_inst.CLKOUT_FT_DIR = 1'b1;
77 defparam rp11_inst.CLKOUTP_FT_DIR = 1'b1;
78 defparam rp11_inst.CLKOUT_DLY_STEP = 0;
79 defparam rp11_inst.CLKOUTP_DLY_STEP = 0;
80 defparam rp11_inst.CLKFB_SEL = "internal";
81 defparam rp11_inst.CLKOUT_BYPASS = "false";
82 defparam rp11_inst.CLKOUTP_BYPASS = "false";
83 defparam rp11_inst.CLKOUTD_BYPASS = "false";
84 defparam rp11_inst.DYN_SDIV_SEL = 2;
85 defparam rp11_inst.CLKOUTD_SRC = "CLKOUT";
86 defparam rp11_inst.CLKOUTD3_SRC = "CLKOUT";
87 defparam rp11_inst.DEVICE = "GW1NR-9C";
88
89 endmodule //Gowin_rPLL

```

```
1 //Copyright (C)2014-2022 Gowin Semiconductor Corporation.
2 //All rights reserved.
3 //File Title: Template file for instantiation
4 //GOWIN Version: V1.9.8.07
5 //Part Number: GW1NR-LV9QN88PC6/I5
6 //Device: GW1NR-9C
7 //Created Time: Wed Aug 17 00:30:59 2022
8
9 //Change the instance name and port connections to the signal names
10 //-----Copy here to design-----
11
12     Gowin_rPLL your_instance_name(
13         .clkout(clkout_o), //output clkout
14         .clkoutp(clkoutp_o), //output clkoutp
15         .clkin(clkin_i) //input clkin
16     );
17
18 //-----Copy end-----
```



```

1  parameter STR = 0;
2  parameter HEX = 1;
3
4  wire print_clk;
5
6  reg[7:0] print_seq[255:0];
7  reg[7:0] seq_head=8'd0;
8  reg[7:0] seq_tail=8'd0;
9
10 reg[1023:0] print_buffer=1024'h0;
11 reg[6:0] print_buffer_pointer = 7'd0;
12
13 reg last_spin_state=0;
14 reg spin_state=0;
15 reg[6:0] print_length;
16 reg print_type;
17
18 parameter PRINT_IDLE_STATE = 0;
19 parameter PRINT_WAIT_STATE = 1;
20 parameter PRINT_WORK_STATE = 2;
21 parameter PRINT_CONV_STATE = 3;
22 reg[1:0] print_state=PRINT_IDLE_STATE;
23
24 wire[7:0] hex_lib[15:0];
25 assign hex_lib[4'h0] = 8'h30;
26 assign hex_lib[4'h1] = 8'h31;
27 assign hex_lib[4'h2] = 8'h32;
28 assign hex_lib[4'h3] = 8'h33;
29 assign hex_lib[4'h4] = 8'h34;
30 assign hex_lib[4'h5] = 8'h35;
31 assign hex_lib[4'h6] = 8'h36;
32 assign hex_lib[4'h7] = 8'h37;
33 assign hex_lib[4'h8] = 8'h38;
34 assign hex_lib[4'h9] = 8'h39;
35 assign hex_lib[4'hA] = 8'h61;
36 assign hex_lib[4'hB] = 8'h62;
37 assign hex_lib[4'hC] = 8'h63;
38 assign hex_lib[4'hD] = 8'h64;
39 assign hex_lib[4'hE] = 8'h65;
40 assign hex_lib[4'hF] = 8'h66;
41
42 //always block to handle the print task
43 always@(posedge print_clk)begin
44     last_spin_state<=spin_state;
45
46     case(print_state)
47         PRINT_IDLE_STATE:begin//IDLE, check if spin_state is changed
48             if(spin_state!=last_spin_state)begin
49                 print_state<=PRINT_WAIT_STATE;
50             end
51         end
52         PRINT_WAIT_STATE:begin//WAIT, wait 1 clk then start to fill print_seq
53             print_state<=PRINT_WORK_STATE;
54             if(print_type==STR)
55                 print_buffer_pointer<=7'd127;
56             else
57                 print_buffer_pointer<=7'd127;
58         end
59         PRINT_WORK_STATE:begin//WORK, fill print_seq
60             if(print_type==STR)begin//type is string, fill as it is
61                 if(print_buffer[
62                     print_buffer_pointer*8+7 -: 8
63                 ]!=8'd0)begin
64                     print_seq[seq_tail]<=print_buffer[
65                         print_buffer_pointer*8+7 -: 8
66                     ];
67                     seq_tail<=seq_tail+8'd1;

```

```

68         end else begin
69             print_state<=PRINT_IDLE_STATE;
70         end
71
72         print_buffer_pointer<=print_buffer_pointer-7'd1;
73
74         if(print_buffer_pointer==7'd0)begin
75             print_state<=PRINT_IDLE_STATE;
76         end
77     end else begin //type is data, fill as hex
78         print_seq[seq_tail]<=hex_lib[print_buffer[
79             print_buffer_pointer*8+7 -: 4
80         ]];
81         seq_tail<=seq_tail+8'd1;
82
83         //another convert clock cycle is needed
84         print_state<=PRINT_CONV_STATE;
85     end
86 end
87 PRINT_CONV_STATE:begin//CONV, convert data to hex
88     print_seq[seq_tail]<=hex_lib[print_buffer[
89         print_buffer_pointer*8+3 -: 4
90     ]];
91     seq_tail<=seq_tail+8'd1;
92     print_state<=PRINT_WORK_STATE;
93
94     print_buffer_pointer<=print_buffer_pointer-7'd1;
95
96     if(print_buffer_pointer==print_length)
97         print_state<=PRINT_IDLE_STATE;
98     end
99 endcase
100 end
101
102 reg uart_en;
103 wire uart_bz;
104 wire uart_txp;
105 uart_tx_V2 tx(print_clk, print_seq[seq_head], uart_en, uart_bz, uart_txp);
106
107 //always block to send the data via UART
108 always@(posedge print_clk)begin
109     uart_en<=1'b0;
110     if(uart_en && uart_bz)
111         seq_head<=seq_head+8'd1;
112     if(seq_head!=seq_tail && !uart_bz)
113         uart_en<=1'b1;
114 end
115
116 task int_print(
117     input[1023:0] strin,//max 128 characters
118     input[7:0] type_length //8bit width to show 128 characters
119 );
120 begin
121     if(print_state==PRINT_IDLE_STATE)begin//print when busy will be ignored
122         spin_state<=~spin_state;
123
124         if(type_length==STR)begin
125             print_type<=STR;
126         end else begin
127             print_type<=HEX;
128             print_length<=8'd128-type_length;
129         end
130
131         print_buffer<=strin;
132     end
133 end
134
135 'define print(a,b) int_print({>>{a}},b)
136 endtask

```

fpga/src/rtl/psram-tang-nano-9k/src/psram_controller.v

```

1 //
2 // PSRAM/HyperRAM controller for Tang Nano 9K / Gowin GW1NR-LV9QN88PC6/15.
3 // Feng Zhou, 2022.8
4 //
5 // This is a word or byte based, non-bursting controller for accessing the on-chip HyperRAM.
6 // - 1:1 clock design. Memory and main logic work under the same clock.
7 // - Low latency. Under default settings, write latency is 7 cycles (1x) or 10 cycles (2x).
8 //   Read latency is 12 cycles (1x) or 15 cycles(2x). In my test, 2x latency happens about
9 //   0.05% of time.
10
11 module PsramController #(
12     parameter FREQ=81_000_000, // Actual clk frequency, to time 150us initialization delay
13     parameter LATENCY=3        // tACC (Initial Latency) in W955D8MBYA datasheet:
14                                // 3 (max 83Mhz), 4 (max 104Mhz), 5 (max 133Mhz) or 6 (max 166Mhz)
15 ) (
16     input clk,
17     input clk_p,                // phase-shifted clock for driving O_psram_ck
18     input resetn,
19     input read,                 // Set to 1 to read from RAM
20     input write,               // Set to 1 to write to RAM
21     input [21:0] addr,         // Byte address to read / write
22     input [15:0] din,          // Data word to write
23     input byte_write,          // When writing, only write one byte instead of the whole word.
24                                // addr[0]==1 means we write the upper half of din. lower half
25                                // otherwise.
26     output reg [15:0] dout,     // Last read data. Read is always word-based.
27     output busy,               // 1 while an operation is in progress
28
29     // HyperRAM physical interface. Gowin interface is for 2 dies.
30     // We currently only use the first die (4MB).
31     output [1:0] O_psram_ck,
32     inout [1:0] IO_psram_rwds,
33     inout [15:0] IO_psram_dq,
34     output [1:0] O_psram_cs_n
35 );
36
37 reg [2:0] state;
38 localparam [2:0] INIT_ST = 3'd0;
39 localparam [2:0] CONFIG_ST= 3'd1;
40 localparam [2:0] IDLE_ST = 3'd2;
41 localparam [2:0] READ_ST = 3'd3;
42 localparam [2:0] WRITE_ST = 3'd4;
43
44 reg cfg_now, dq_oen, ram_cs_n, ck_e, ck_e_p;
45 reg wait_for_rd_data;
46 reg ub;                        // 1 for upper byte
47 reg [15:0] w_din;
48 reg [23:0] cycles_sr;         // shift register counting cycles
49 reg [63:0] dq_sr;             // shifts left 8-bit every cycle
50
51 // DDR input output signals
52 wire [7:0] dq_out_ris = dq_sr[63:56];
53 wire [7:0] dq_out_fal = dq_sr[55:48];
54 wire [7:0] dq_in_ris;
55 wire [7:0] dq_in_fal;
56 reg rwds_out_ris, rwds_out_fal, rwds_oen;
57 wire rwds_in_ris, rwds_in_fal;
58 reg additional_latency;
59
60 assign busy = (state != IDLE_ST);
61
62 localparam [3:0] CR_LATENCY = LATENCY == 3 ? 4'b1110 :
63                                LATENCY == 4 ? 4'b1111 :
64                                LATENCY == 5 ? 4'b0 :
65                                LATENCY == 6 ? 4'b0001 : 4'b1110;
66
67 // Main FSM for HyperRAM read/write

```

```

67 always @(posedge clk) begin
68     cycles_sr <= {cycles_sr[22:0], 1'b0};
69     dq_sr <= {dq_sr[47:0], 16'b0};          // shift 16-bits each cycle
70     ck_e_p <= ck_e;
71
72     if (state == INIT_ST && cfg_now) begin
73         cycles_sr <= 24'b1;
74         ram_cs_n <= 0;
75         state <= CONFIG_ST;
76     end
77     if (state == CONFIG_ST) begin
78         if (cycles_sr[0]) begin
79             dq_sr <= {8'h60, 8'h00, 8'h01, 8'h00, 8'h00, 8'h00, 8'h8f, CR_LATENCY, 4'h7}; //
80                 last byte, 'e' (3 cycle latency max 83Mhz), '7' (variable 1x/2x latency)
81             dq_oen <= 0;
82             ck_e <= 1;          // this needs to be earlier 1 cycle to allow for phase shifted clk_p
83         end
84         if (cycles_sr[4]) begin
85             state <= IDLE_ST;
86             ck_e <= 0;
87             cycles_sr <= 24'b1;
88             dq_oen <= 1;
89             ram_cs_n <= 1;
90         end
91     end
92     if (state == IDLE_ST) begin
93         rwds_oen <= 1;
94         ck_e <= 0;
95         ram_cs_n <= 1;
96         if (read || write) begin
97             // start read/write operation
98             dq_sr <= {~write, 13'b010_0000_0000_00, addr[21:4], 13'b0, addr[3:1], 16'
100                 b0000_0100_1101_0100};
101                 //          14-bit          18-bit   13-bit   3-bit          total
102                 48-bit CA
103             ram_cs_n <= 0;
104             ck_e <= 1;
105             dq_oen <= 0;
106             wait_for_rd_data <= 0;
107             w_din <= din;
108             cycles_sr <= 32'b10;    // start from cycle 1
109             state <= write ? WRITE_ST : READ_ST;
110         end
111     end
112 end
113
114 if (state == READ_ST) begin
115     if (cycles_sr[3]) begin
116         // command sent, now wait for result
117         dq_oen <= 1;
118     end
119     if (cycles_sr[9])
120         wait_for_rd_data <= 1;
121     if (wait_for_rd_data && (rwds_in_ris ^ rwds_in_fal)) begin    // sample rwds falling
122         edge to get a word / \_
123         dout <= {dq_in_ris, dq_in_fal};
124         ram_cs_n <= 1;
125         ck_e <= 0;
126         state <= IDLE_ST;
127     end
128 end
129
130 if (state == WRITE_ST) begin
131     if (cycles_sr[5])
132         additional_latency <= rwds_in_fal; // sample RWDS to see if we need additional
133             latency
134     // Write timing is trickier - we sample RWDS at cycle 5 to determine whether we need to
135         wait another tACC.
136     // If it is low, data starts at 2+LATENCY. If high, then data starts at 2+LATENCY*2.
137     if (cycles_sr[2+LATENCY] && (LATENCY == 3 ? ~rwds_in_fal : ~additional_latency)
138         || cycles_sr[2+LATENCY*2])

```

```

131     begin
132         rwds_oen <= 0;
133         rwds_out_ris <= byte_write ? ~addr[0] : 1'b0;           // RWDS is data mask (1 means not
                           writing)
134         rwds_out_fal <= byte_write ? addr[0] : 1'b0;
135         dq_sr[63:48] <= w_din;
136         state <= IDLE_ST;
137     end
138 end
139
140 if (~resetn) begin
141     state <= INIT_ST;
142     ram_cs_n <= 1;
143     ck_e <= 0;
144 end
145 end
146
147 // 150us initialization delay
148 //
149 // Generate cfg_now pulse after 150us delay
150 //
151 localparam INIT_TIME = FREQ / 1000 * 160 / 1000;
152 reg [$clog2(INIT_TIME+1):0] rst_cnt;
153 reg rst_done, rst_done_p1, cfg_busy;
154
155 always @(posedge clk) begin
156     rst_done_p1 <= rst_done;
157     cfg_now      <= rst_done & ~rst_done_p1; // Rising Edge Detect
158
159     if (rst_cnt != INIT_TIME) begin           // count to 160 us
160         rst_cnt <= rst_cnt[14:0] + 1;
161         rst_done <= 0;
162         cfg_busy <= 1;
163     end else begin
164         rst_done <= 1;
165         cfg_busy <= 0;
166     end
167 end
168
169 if (~resetn) begin
170     rst_cnt <= 15'd0;
171     rst_done <= 0;
172     cfg_busy <= 1;
173 end
174 end
175
176 // Tristate DDR output
177 wire dq_out_tbuf[7:0];
178 wire dq_oen_tbuf[7:0];
179 ODDR oddr_cs_n(
180     .CLK(clk), .D0(ram_cs_n), .D1(ram_cs_n), .Q0(cs_n_tbuf)
181 );
182 assign O_psram_cs_n[0] = cs_n_tbuf;
183 ODDR oddr_rwds(
184     .CLK(clk), .D0(rwds_out_ris), .D1(rwds_out_fal), .TX(rwds_oen), .Q0(rwds_tbuf), .Q1(
        rwds_oen_tbuf)
185 );
186 assign IO_psram_rwds[0] = rwds_oen_tbuf ? 1'bz : rwds_tbuf;
187
188 genvar i1;
189 generate
190     for (i1=0; i1<=7; i1=i1+1) begin: gen_i1
191         ODDR oddr_dq_i1(
192             .CLK(clk), .D0(dq_out_ris[i1]), .D1(dq_out_fal[i1]), .TX(dq_oen), .Q0(dq_out_tbuf[i1
                ], .Q1(dq_oen_tbuf[i1])
193         );
194         assign IO_psram_dq[i1] = dq_oen_tbuf[i1] ? 1'bz : dq_out_tbuf[i1];
195     end
196 endgenerate
197 // Note: ck uses phase-shifted clock clk_p

```

```

198 ODDR oddr_ck(
199     .CLK(clk_p), .D0(ck_e_p), .D1(1'b0), .Q0(ck_tbuf)
200 );
201 assign O_psram_ck[0] = ck_tbuf;
202
203
204 // Tristate DDR input
205 IDDR iddr_rwds(
206     .CLK(clk), .D(IO_psram_rwds[0]), .Q0(rwds_in_ris), .Q1(rwds_in_fal)
207 );
208 genvar i2;
209 generate
210     for (i2=0; i2<=7; i2=i2+1) begin: gen_i2
211         IDDR iddr_dq_i2(
212             .CLK(clk), .D(IO_psram_dq[i2]), .Q0(dq_in_ris[i2]), .Q1(dq_in_fal[i2])
213         );
214     end
215 endgenerate
216
217 endmodule

```

```

1
2
3 module memory_test (
4     input sys_clk, // 27 Mhz, crystal clock from board
5     input sys_resestn,
6     input button, // 0 when pressed
7
8     output [5:0] led,
9     output uart_txp,
10
11     output [1:0] O_psram_ck, // Magic ports for PSRAM to be inferred
12     output [1:0] O_psram_ck_n,
13     inout [1:0] IO_psram_rwds,
14     inout [15:0] IO_psram_dq,
15     output [1:0] O_psram_reset_n,
16     output [1:0] O_psram_cs_n
17 );
18
19 // Customization of the test
20 localparam [21:0] BYTES = 1024*1024; // Test write/read this many bytes
21
22 // Change PLL and here to choose another speed.
23 localparam FREQ = 81_000_000;
24 localparam LATENCY = 3;
25 //localparam FREQ = 102_600_000;
26 //localparam LATENCY = 4;
27
28 // Remove UART print module for timing closure (check LED5 for error)
29 //`define NO_UART_PRINT
30
31 // For GAO debug
32 //localparam [21:0] BYTES = 2;
33 //localparam NO_PAUSE = 1;
34 localparam NO_PAUSE = 0; // Pause between states to allow UART printing
35
36 // End of customization
37
38 assign O_psram_reset_n = sys_resestn;
39
40 Gowin_rPLL pll(
41     .clkout(clk), // MHZ main clock
42     .clkoutp(clk_p), // MHZ phase shifted (90 degrees)
43     .clkin(sys_clk) // 27Mhz system clock
44 );
45
46 // Memory Controller under test -----
47 reg read, readd, write, byte_write;
48 reg [21:0] address;
49 reg [15:0] din;
50 wire [15:0] dout;
51 wire [7:0] dout_byte = address[0] ? dout[15:8] : dout[7:0];
52
53 PsramController #(
54     .LATENCY(LATENCY)
55 ) mem_ctrl(
56     .clk(clk), .clk_p(clk_p), .resestn(sys_resestn), .read(read), .write(write), .byte_write(
57         byte_write),
58     .addr(address), .din(din), .dout(dout), .busy(busy),
59     .O_psram_ck(O_psram_ck), .IO_psram_rwds(IO_psram_rwds), .IO_psram_dq(IO_psram_dq),
60     .O_psram_cs_n(O_psram_cs_n)
61 );
62 defparam mem_clk_freq=FREQ;
63
64 // The test -----
65 localparam [3:0] TEST_ZERO = 4'd0;
66 localparam [3:0] TEST_INIT = 4'd1;

```

```

67 localparam [3:0] TEST_WRITE = 4'd2;
68 localparam [3:0] TEST_READ = 4'd3;
69 localparam [3:0] TEST_DONE = 4'd4;
70 localparam [3:0] TEST_FAIL_INIT_TIMEOUT = 4'd5;
71 localparam [3:0] TEST_FAIL_WRITE_TIMEOUT = 4'd6;
72 localparam [3:0] TEST_FAIL_READ_TIMEOUT = 4'd7;
73 localparam [3:0] TEST_FAIL_READ_WRONG = 4'd8;
74 localparam [3:0] PAUSE = 4'd9;
75
76 // pass in address to get hash value
77 `define hash(a) (a[7:0] ^ a[15:8] ^ a[21:16] ^ 8'hc3)
78
79 reg [3:0] state, new_state;
80 reg [4:0] cycle = 0; // max 16
81 reg [23:0] write_1x, write_2x, read_1x, read_2x; // counter for 1x or 2x latencies
82 reg tick; // pulse once per 0.1 second
83 reg [3:0] ticks = 0; // counter for 0.1 second delays
84 reg error;
85 assign dout_parity = dout[7] & dout[6] & dout[5] & dout[4] & dout[3] & dout[2] & dout[1] & dout
    [0];
86 assign led = ~(error, dout_parity, dout, state);
87
88 // pipeline addr+1 to meet timing constraint
89 reg [8:0] new_addr_0;
90 reg [8:0] new_addr_1;
91 reg [21:0] new_addr; // available after 3 cycles
92 always @(posedge clk) begin
93     // stage 0
94     new_addr_0 = address[7:0] + 1;
95     // stage 1
96     new_addr_1 = address[15:8] + new_addr_0[8];
97     // stage 2, add higher 6 bits
98     new_addr = {address[21:16] + new_addr_1[8], new_addr_1[7:0], new_addr_0[7:0]};
99 end
100
101 always @(posedge clk) begin
102     read <= 0; write <= 0; byte_write <= 1; // default values
103     ticks <= tick && (state == TEST_INIT || state == PAUSE) ? ticks + 1 : ticks;
104     if (~sys_resetn || state == TEST_ZERO) begin
105         cycle <= 0;
106         ticks <= 0;
107         new_state <= TEST_INIT;
108         state <= PAUSE;
109         error <= 0;
110
111     end else if (state == TEST_INIT) begin
112         // wait for memory to become ready
113         if (!busy) begin
114             new_state <= TEST_WRITE;
115             state <= PAUSE;
116         end else if (ticks == 5) begin // 0.5 second timeout
117             new_state <= TEST_FAIL_INIT_TIMEOUT;
118             error <= 1'b1;
119             state <= PAUSE;
120         end
121
122     end if (state == TEST_WRITE) begin
123         // write some bytes
124         cycle <= cycle + 1;
125         if (cycle == 0) begin
126             // issue write command
127             write <= 1;
128             din <= {'hash(address), 'hash(address)};
129         end else if (!write && !busy) begin
130             // write finished
131             cycle <= 0;
132             if (cycle > 4+LATENCY)
133                 write_2x <= write_2x + 1;
134             else
135                 write_1x <= write_1x + 1;

```



```

136         if (address == BYTES - 1) begin
137             new_state <= TEST_READ;
138             state <= PAUSE;
139         end else
140             address <= new_addr;
141     end else if (cycle == 5+LATENCY*2) begin
142         new_state <= TEST_FAIL_WRITE_TIMEOUT;
143         error <= 1'b1;
144         state <= PAUSE;
145     end
146
147 end if (state == TEST_READ) begin
148     // read and verify some bytes
149     cycle <= cycle + 1;
150     if (cycle == 0) begin
151         // issue read command
152         read <= 1;
153     end else if (!read && !busy) begin
154         // read finished
155         cycle <= 0;
156         if (cycle > 10+LATENCY) // read_is on cycle 1, so cycle==13 means latency is 12
157             read_2x <= read_2x + 1;
158         else
159             read_1x <= read_1x + 1;
160         if (dout_byte != 'hash(address)) begin
161             new_state <= TEST_FAIL_READ_WRONG;
162             error <= 1'b1;
163             state <= PAUSE;
164         end else if (address == BYTES - 1) begin
165             new_state <= TEST_DONE;
166             state <= PAUSE;
167         end else
168             address <= new_addr;
169     end else if (cycle == 10+LATENCY*2) begin
170         new_state <= TEST_FAIL_READ_TIMEOUT;
171         error <= 1'b1;
172         state <= PAUSE;
173     end
174
175 end else if (state == PAUSE) begin
176     address <= 0;
177     // pause for 0.1 seconds for print to finish, then enter new_state
178     if (ticks == 2 || NO_PAUSE) begin // pause for 0.1 second
179         ticks <= 0;
180         state <= new_state;
181     end
182 end
183 end
184
185
186 reg [23:0] tick_counter; // max 16M
187 always @(posedge clk) begin
188     if (~sys_resetn) begin
189         tick_counter <= FREQ/10;
190     end
191     tick_counter <= tick_counter == 0 ? FREQ/10 : tick_counter - 1;
192     tick <= tick_counter == 0;
193 end
194
195
196 //Print Controll -----
197
198 `ifndef NO_UART_PRINT
199 `include "print.v"
200 defparam tx.uart_freq=115200;
201 defparam tx.clk_freq=FREQ;
202 assign print_clk = clk;
203 assign txp = uart_txp;
204
205 reg [3:0] state_p;

```

```

206 reg [3:0] print_counters = 0;          // 1. "write_1x=", 2. write_1x, 3. ", write_2x=", 4. write_2x
    , 5. ", read_1x=", 6. "read_1x", 7. ", read_2x=", 8. read_2x., 9. "\n"
207 reg [3:0] print_counters_p;
208
209 always @(posedge clk) begin
210     state_p <= state;
211     print_counters_p <= print_counters;
212     if (state != state_p) begin
213         if (state == TEST_INIT) 'print("Initializing HyperRAM test...\n", STR);
214         if (state == TEST_WRITE) 'print("Writing...\n", STR);
215         if (state == TEST_READ) 'print("Reading...\n", STR);
216         if (state == TEST_DONE) 'print("All done successfully.\n", STR);
217         if (state == TEST_FAIL_INIT_TIMEOUT) 'print("FAIL. Initialization timeout.\n", STR);
218         if (state == TEST_FAIL_WRITE_TIMEOUT) 'print("FAIL. Write time out.\n", STR);
219         if (state == TEST_FAIL_READ_TIMEOUT) 'print("FAIL. Read time out.\n", STR);
220         if (state == TEST_FAIL_READ_WRONG) 'print("FAIL. Read wrong data.\n", STR);
221
222         if (state == TEST_DONE || state == TEST_FAIL_INIT_TIMEOUT || state ==
            TEST_FAIL_READ_TIMEOUT || state == TEST_FAIL_READ_WRONG || state ==
            TEST_FAIL_WRITE_TIMEOUT)
            print_counters <= 1;
223
224     end
225
226     if (print_counters > 0 && print_counters == print_counters_p && print_state ==
        PRINT_IDLE_STATE) begin
227         case (print_counters)
228             1: 'print("Latency counters: write_1x=", STR);
229             2: 'print(write_1x, 3);
230             3: 'print(", write_2x=", STR);
231             4: 'print(write_2x, 3);
232             5: 'print(", read_1x=", STR);
233             6: 'print(read_1x, 3);
234             7: 'print(", read_2x=", STR);
235             8: 'print(read_2x, 3);
236             9: 'print("\n", STR);
237         endcase
238         print_counters <= print_counters == 9 ? 0 : print_counters + 1;
239     end
240
241 end
242 'endif
243
244
245 endmodule

```

```

1 module uart_tx_V2(
2     input wire clk,
3     input wire [7:0] din,
4     input wire wr_en,
5     output wire tx_busy,
6
7     output reg tx_p
8 );
9
10 initial begin
11     tx_p = 1'b1;
12 end
13
14 parameter clk_freq = 27000000;
15 parameter uart_freq = 115200;
16
17 localparam STATE_IDLE    = 2'b00;
18 localparam STATE_START  = 2'b01;
19 localparam STATE_DATA    = 2'b10;
20 localparam STATE_STOP    = 2'b11;
21
22 reg[7:0] localdin;
23 reg localwr_en;
24
25 //always@(posedge clk)begin
26 always@(*)begin
27     localdin<=din;
28     localwr_en<=wr_en;
29 end
30
31 reg [7:0] data= 8'h00;
32 reg [2:0] bitpos= 3'h0;
33 reg [1:0] state= STATE_IDLE;
34
35 wire tx_clk;
36
37 localparam TX_CLK_MAX = (clk_freq / uart_freq)-1;
38
39 reg[$clog2(TX_CLK_MAX+1)+1:0] tx_clkcnt;
40
41 assign tx_clk = (tx_clkcnt == 0);
42
43 initial tx_clkcnt=0;
44
45 always @(posedge clk) begin
46     if (tx_clkcnt >= TX_CLK_MAX)
47         tx_clkcnt <= 0;
48     else
49         tx_clkcnt <= tx_clkcnt + 1;
50 end
51
52
53 always @(posedge clk) begin
54     case (state)
55     STATE_IDLE: begin
56         if (localwr_en) begin
57             state <= STATE_START;
58             data <= localdin;
59             bitpos <= 3'h0;
60         end
61     end
62     STATE_START: begin
63         if (tx_clk) begin
64             tx_p <= 1'b0;
65             state <= STATE_DATA;
66         end
67     end

```

```

68     STATE_DATA: begin
69         if (tx_clk) begin
70             if (bitpos == 3'h7)
71                 state <= STATE_STOP;
72             else
73                 bitpos <= bitpos + 3'h1;
74             tx_p <= data[bitpos];
75         end
76     end
77     STATE_STOP: begin
78         if (tx_clk) begin
79             tx_p <= 1'b1;
80             state <= STATE_IDLE;
81         end
82     end
83     default: begin
84         tx_p <= 1'b1;
85         state <= STATE_IDLE;
86     end
87 endcase
88 end
89
90 assign tx_busy = (state != STATE_IDLE);
91
92 endmodule

```

fpga/src/rtl/psram_controller.v

```

1 //
2 // PSRAM/HyperRAM controller for Tang Nano 9K / Gowin GW1NR-LV9QN88PC6/15.
3 // Feng Zhou, 2022.8
4 //
5 // This is a word or byte based, non-bursting controller for accessing the on-chip HyperRAM.
6 // - 1:1 clock design. Memory and main logic work under the same clock.
7 // - Low latency. Under default settings, write latency is 7 cycles (1x) or 10 cycles (2x).
8 //   Read latency is 12 cycles (1x) or 15 cycles(2x). In my test, 2x latency happens about
9 //   0.05% of time.
10
11 module PsramController #(
12     parameter FREQ=81_000_000, // Actual clk frequency, to time 150us initialization delay
13     parameter LATENCY=3        // tACC (Initial Latency) in W955D8MBYA datasheet:
14                                 // 3 (max 83Mhz), 4 (max 104Mhz), 5 (max 133Mhz) or 6 (max 166Mhz)
15 ) (
16     input clk,
17     input clk_p,                // phase-shifted clock for driving O_psram_ck
18     input resetn,
19     input read,                 // Set to 1 to read from RAM
20     input write,                // Set to 1 to write to RAM
21     input [21:0] addr,          // Byte address to read / write
22     input [15:0] din,           // Data word to write
23     input byte_write,           // When writing, only write one byte instead of the whole word.
24                                 // addr[0]==1 means we write the upper half of din. lower half
25                                 // otherwise.
26     output reg [15:0] dout,     // Last read data. Read is always word-based.
27     output busy,                // 1 while an operation is in progress
28
29     // HyperRAM physical interface. Gowin interface is for 2 dies.
30     // We currently only use the first die (4MB).
31     output [1:0] O_psram_ck,
32     inout [1:0] IO_psram_rwds,
33     inout [15:0] IO_psram_dq,
34     output [1:0] O_psram_cs_n
35 );
36
37 reg [2:0] state;
38 localparam [2:0] INIT_ST = 3'd0;
39 localparam [2:0] CONFIG_ST= 3'd1;
40 localparam [2:0] IDLE_ST = 3'd2;
41 localparam [2:0] READ_ST = 3'd3;
42 localparam [2:0] WRITE_ST = 3'd4;
43
44 reg cfg_now, dq_oen, ram_cs_n, ck_e, ck_e_p;
45 reg wait_for_rd_data;
46 reg ub;                        // 1 for upper byte
47 reg [15:0] w_din;
48 reg [23:0] cycles_sr;          // shift register counting cycles
49 reg [63:0] dq_sr;              // shifts left 8-bit every cycle
50
51 // DDR input output signals
52 wire [7:0] dq_out_ris = dq_sr[63:56];
53 wire [7:0] dq_out_fal = dq_sr[55:48];
54 wire [7:0] dq_in_ris;
55 wire [7:0] dq_in_fal;
56 reg rwds_out_ris, rwds_out_fal, rwds_oen;
57 wire rwds_in_ris, rwds_in_fal;
58 reg additional_latency;
59
60 assign busy = (state != IDLE_ST);
61
62 localparam [3:0] CR_LATENCY = LATENCY == 3 ? 4'b1110 :
63                               LATENCY == 4 ? 4'b1111 :
64                               LATENCY == 5 ? 4'b0 :
65                               LATENCY == 6 ? 4'b0001 : 4'b1110;
66
67 // Main FSM for HyperRAM read/write

```

```

67 always @(posedge clk) begin
68     cycles_sr <= {cycles_sr[22:0], 1'b0};
69     dq_sr <= {dq_sr[47:0], 16'b0};           // shift 16-bits each cycle
70     ck_e_p <= ck_e;
71
72     if (state == INIT_ST && cfg_now) begin
73         cycles_sr <= 24'b1;
74         ram_cs_n <= 0;
75         state <= CONFIG_ST;
76     end
77     if (state == CONFIG_ST) begin
78         if (cycles_sr[0]) begin
79             dq_sr <= {8'h60, 8'h00, 8'h01, 8'h00, 8'h00, 8'h00, 8'h8f, CR_LATENCY, 4'h7}; //
80                 last byte, 'e' (3 cycle latency max 83Mhz), '7' (variable 1x/2x latency)
81             dq_oen <= 0;
82             ck_e <= 1;           // this needs to be earlier 1 cycle to allow for phase shifted clk_p
83         end
84         if (cycles_sr[4]) begin
85             state <= IDLE_ST;
86             ck_e <= 0;
87             cycles_sr <= 24'b1;
88             dq_oen <= 1;
89             ram_cs_n <= 1;
90         end
91     end
92     if (state == IDLE_ST) begin
93         rwds_oen <= 1;
94         ck_e <= 0;
95         ram_cs_n <= 1;
96         if (read || write) begin
97             // start read/write operation
98             dq_sr <= {~write, 13'b010_0000_0000_00, addr[21:4], 13'b0, addr[3:1], 16'
100                 b0000_0100_1101_0100};
101                 //          14-bit          18-bit   13-bit   3-bit          total
102                 48-bit CA
103             ram_cs_n <= 0;
104             ck_e <= 1;
105             dq_oen <= 0;
106             wait_for_rd_data <= 0;
107             w_din <= din;
108             cycles_sr <= 32'b10; // start from cycle 1
109             state <= write ? WRITE_ST : READ_ST;
110         end
111     end
112 end
113
114 if (state == READ_ST) begin
115     if (cycles_sr[3]) begin
116         // command sent, now wait for result
117         dq_oen <= 1;
118     end
119     if (cycles_sr[9])
120         wait_for_rd_data <= 1;
121     if (wait_for_rd_data && (rwds_in_ris ^ rwds_in_fal)) begin // sample rwds falling
122         edge to get a word / \_
123         dout <= {dq_in_ris, dq_in_fal};
124         ram_cs_n <= 1;
125         ck_e <= 0;
126         state <= IDLE_ST;
127     end
128 end
129
130 if (state == WRITE_ST) begin
131     if (cycles_sr[5])
132         additional_latency <= rwds_in_fal; // sample RWDS to see if we need additional
133             latency
134     // Write timing is trickier - we sample RWDS at cycle 5 to determine whether we need to
135         wait another tACC.
136     // If it is low, data starts at 2+LATENCY. If high, then data starts at 2+LATENCY*2.
137     if (cycles_sr[2+LATENCY] && (LATENCY == 3 ? ~rwds_in_fal : ~additional_latency)
138         || cycles_sr[2+LATENCY*2])

```

```

131     begin
132         rwds_oen <= 0;
133         rwds_out_ris <= byte_write ? ~addr[0] : 1'b0;           // RWDS is data mask (1 means not
                           writing)
134         rwds_out_fal <= byte_write ? addr[0] : 1'b0;
135         dq_sr[63:48] <= w_din;
136         state <= IDLE_ST;
137     end
138 end
139
140 if (~resetn) begin
141     state <= INIT_ST;
142     ram_cs_n <= 1;
143     ck_e <= 0;
144 end
145 end
146
147
148 // 150us initialization delay
149 //
150 // Generate cfg_now pulse after 150us delay
151 //
152 localparam INIT_TIME = FREQ / 1000 * 160 / 1000;
153 reg [$clog2(INIT_TIME+1):0] rst_cnt;
154 reg rst_done, rst_done_p1, cfg_busy;
155
156 always @(posedge clk) begin
157     rst_done_p1 <= rst_done;
158     cfg_now      <= rst_done & ~rst_done_p1; // Rising Edge Detect
159
160     if (rst_cnt != INIT_TIME) begin           // count to 160 us
161         rst_cnt <= rst_cnt[14:0] + 1;
162         rst_done <= 0;
163         cfg_busy <= 1;
164     end else begin
165         rst_done <= 1;
166         cfg_busy <= 0;
167     end
168
169     if (~resetn) begin
170         rst_cnt <= 15'd0;
171         rst_done <= 0;
172         cfg_busy <= 1;
173     end
174 end
175
176 // Tristate DDR output
177 wire dq_out_tbuf[7:0];
178 wire dq_oen_tbuf[7:0];
179 ODDR oddr_cs_n(
180     .CLK(clk), .D0(ram_cs_n), .D1(ram_cs_n), .Q0(cs_n_tbuf)
181 );
182 assign O_psram_cs_n[0] = cs_n_tbuf;
183 ODDR oddr_rwds(
184     .CLK(clk), .D0(rwds_out_ris), .D1(rwds_out_fal), .TX(rwds_oen), .Q0(rwds_tbuf), .Q1(
        rwds_oen_tbuf)
185 );
186 assign IO_psram_rwds[0] = rwds_oen_tbuf ? 1'bz : rwds_tbuf;
187
188 genvar i1;
189 generate
190     for (i1=0; i1<=7; i1=i1+1) begin: gen_i1
191         ODDR oddr_dq_i1(
192             .CLK(clk), .D0(dq_out_ris[i1]), .D1(dq_out_fal[i1]), .TX(dq_oen), .Q0(dq_out_tbuf[i1
                ], .Q1(dq_oen_tbuf[i1])
193         );
194         assign IO_psram_dq[i1] = dq_oen_tbuf[i1] ? 1'bz : dq_out_tbuf[i1];
195     end
196 endgenerate
197 // Note: ck uses phase-shifted clock clk_p

```

```

198 ODDR oddr_ck(
199     .CLK(clk_p), .D0(ck_e_p), .D1(1'b0), .Q0(ck_tbuf)
200 );
201 assign O_psram_ck[0] = ck_tbuf;
202
203
204 // Tristate DDR input
205 IDDR iddr_rwds(
206     .CLK(clk), .D(IO_psram_rwds[0]), .Q0(rwds_in_ris), .Q1(rwds_in_fal)
207 );
208 genvar i2;
209 generate
210     for (i2=0; i2<=7; i2=i2+1) begin: gen_i2
211         IDDR iddr_dq_i2(
212             .CLK(clk), .D(IO_psram_dq[i2]), .Q0(dq_in_ris[i2]), .Q1(dq_in_fal[i2])
213         );
214     end
215 endgenerate
216
217 endmodule

```


fpga/src/rtl/seq_msk.v

```
1  /*
2  * seq_msk.v -- zero coeffs for r+c>T
3  *
4  * T is a parameter; yosys evaluates r+c<=T at elaboration time.
5  * No comparators in hardware.
6  *
7  * Ports flattened for yosys: element [r][c] = flat[(r*8+c)*W +: W]
8  */
9
10 module seq_msk
11     #(parameter W = 16, T = 6)
12     (input  wire signed [W*64-1:0] c_i,
13      output wire signed [W*64-1:0] c_o);
14
15     genvar r, c;
16     generate
17         for (r = 0; r < 8; r = r + 1) begin : row
18             for (c = 0; c < 8; c = c + 1) begin : col
19                 assign c_o[(r*8+c)*W +: W] =
20                     (r + c <= T) ? c_i[(r*8+c)*W +: W]
21                     : {W{1'b0}};
22             end
23         end
24     endgenerate
25 endmodule
```

fpga/src/rtl/top.v

```
1  /*
2  * top.v
3  *
4  * Trail Camera IV – FPGA Top Module
5  * SPI Slave → BSRAM → LCD Controller
6  * I2C GPIO Expander → stepper coils + joystick buttons
7  * Branchless datapath: if only for async reset
8  */
9
10 module top
11     (input wire sys_clk,
12      // Pin 52 (27MHz)
13      input wire btn,
14      // Pin 4 (S2 button)
15
16      // SPI from ESP32 DevKit
17      input wire esp_mosi,
18      // Pin 77
19      output wire esp_miso,
20      // Pin 76
21      input wire esp_sclk,
22      // Pin 48
23      input wire esp_cs_n,
24      // Pin 49
25
26      // LCD RGB interface
27      output wire lcd_clk,
28      output wire lcd_de,
29      output wire lcd_hsync,
30      output wire lcd_vsync,
31      output wire [4 : 0] lcd_r,
32      output wire [5 : 0] lcd_g,
33      output wire [4 : 0] lcd_b,
34
35      // I2C GPIO Expander (open-drain, external 4.7k pull-ups to 3.3V)
36      inout wire i2c_sda,
37      // Pin 82 (IOT11A) LVCMOS18
38      input wire i2c_scl,
39      // Pin 81 (IOT11B) LVCMOS18
40
41      // Physical GPIO – driven by i2c_gpio_expander registers
42      // Stepper coils: output (DIR_REG bits [3:0] = 0)
43      output wire step_1,
44      // Pin 25
45      output wire step_2,
46      // Pin 26
47      output wire step_3,
48      // Pin 27
49      output wire step_4,
50      // Pin 28
51
52      // Joystick buttons: input (DIR_REG bits [2:0] = 1 on reset)
53      input wire button_L,
54      // Pin 84
55      input wire button_R,
56      // Pin 83
57      input wire button_S,
58      // Pin 85
59
60      // PSRAM (internal to GW1NR package -- no pin constraints needed)
61      output wire [1:0] o_psram_ck,
62      output wire [1:0] o_psram_ck_n,
63      inout wire [1:0] io_psram_rwds,
64      inout wire [15:0] io_psram_dq,
65      output wire [1:0] o_psram_reset_n,
66      output wire [1:0] o_psram_cs_n,
67
```

```

68 // Debug LEDs
69 output wire [5 : 0] led);
70
71 // =====
72 // Power-on reset
73 // =====
74 reg [7 : 0] reset_counter = 8'd0;
75 reg sys_rst_n = 1'b0;
76 wire rst_done = (reset_counter == 8'd255);
77
78 always @(posedge sys_clk) begin
79     reset_counter <= reset_counter + {7'd0, ~rst_done};
80     sys_rst_n <= rst_done;
81 end
82
83 // =====
84 // PLL: 27 MHz -> 81 MHz + 81 MHz phase-shifted
85 // =====
86 wire clk_81m;
87 wire clk_81m_p;
88
89 Gowin_rPLL pll(
90     .clkout(clk_81m),
91     .clkoutp(clk_81m_p),
92     .clkkin(sys_clk)
93 );
94
95 // =====
96 // PSRAM Controller (4 MB HyperRAM, internal to GW1NR)
97 // =====
98 localparam PSRAM_FREQ = 81_000_000;
99 localparam PSRAM_LATENCY = 3;
100
101 // PSRAM signals - muxed between SPI write and scheduler read
102 wire sched_psram_read;
103 wire [21:0] sched_psram_addr;
104 reg spi_psram_write;
105 reg [21:0] spi_psram_addr;
106 reg [15:0] spi_psram_din;
107
108 wire psram_read_mux = sched_psram_read & ~spi_psram_write;
109 wire psram_write_mux = spi_psram_write;
110 wire [21:0] psram_addr_mux = spi_psram_write ? spi_psram_addr
111                                     : sched_psram_addr;
112
113 wire [15:0] psram_dout;
114 wire psram_busy;
115
116 assign O_psram_reset_n = {sys_rst_n, sys_rst_n};
117 assign O_psram_ck_n = 2'b00;
118
119 PsramController #(
120     .FREQ(PSRAM_FREQ),
121     .LATENCY(PSRAM_LATENCY)
122 ) psram_ctrl(
123     .clk(clk_81m),
124     .clk_p(clk_81m_p),
125     .resetn(sys_rst_n),
126     .read(psram_read_mux),
127     .write(psram_write_mux),
128     .addr(psram_addr_mux),
129     .din(spi_psram_din),
130     .byte_write(1'b0),
131     .dout(psram_dout),
132     .busy(psram_busy),
133     .O_psram_ck(O_psram_ck),
134     .IO_psram_rwds(IO_psram_rwds),
135     .IO_psram_dq(IO_psram_dq),
136     .O_psram_cs_n(O_psram_cs_n)
137 );

```

```

138 // =====
139 // Parameters
140 // =====
141 localparam BRAM_SIZE = 32768;
142 localparam BRAM_ADDR_WIDTH = 15;
143
144 // =====
145 // Internal signals
146 // =====
147 wire [7 : 0] esp_rx_data;
148 wire esp_rx_valid;
149 wire esp_rx_ready = 1'b1;
150 wire esp_cs_active; // 3-stage synchronized CS from esp_interface
151
152 reg [BRAM_ADDR_WIDTH - 1 : 0] bram_write_addr;
153 reg [7 : 0] bram_write_data;
154 reg bram_write_en;
155
156 wire [14 : 0] lcd_bram_addr;
157 wire [7 : 0] bram_read_data;
158
159 reg frame_ready;
160 reg receiving;
161
162 // =====
163 // I2C SDA tristate
164 // sda_oe=1 -> drive low (open-drain pull-down)
165 // sda_oe=0 -> release (external pull-up handles high)
166 // =====
167 wire sda_oe;
168 assign i2c_sda = sda_oe ? 1'b0 : 1'bz;
169
170 // =====
171 // GPIO expander register outputs
172 // =====
173 wire [7 : 0] i2c_gpio_out;
174 wire [7 : 0] i2c_gpio_dir;
175 wire [7 : 0] i2c_gpio_invert;
176
177 // Button inputs registered for metastability before feeding into
178 // expander Layout: gpio_in[2]=button_S, [1]=button_R, [0]=button_L
179 reg [2 : 0] btn_sync_0, btn_sync_1;
180 always @(posedge sys_clk or negedge sys_rst_n) begin
181     if (!sys_rst_n) begin
182         btn_sync_0 <= 3'b111;
183         btn_sync_1 <= 3'b111;
184     end else begin
185         btn_sync_0 <= {button_S, button_R, button_L};
186         btn_sync_1 <= btn_sync_0;
187     end
188 end
189
190 wire [7 : 0] i2c_gpio_in = {5'b00000,
191                             btn_sync_1}; // upper 5 bits unused
192
193 // =====
194 // I2C GPIO Expander
195 //
196 // Register convention (ESP32 sets on boot):
197 //   gpio_out[3:0] -> step_1..4   (DIR bits[3:0] = 0 = output)
198 //   gpio_in[2:0]  -> L, R, S     (DIR bits[2:0] = 1 = input)
199 //   gpio_in[7:3]  -> unused
200 //   ESP32 writes DIR_REG = 0xF0 on boot:
201 //       bits[7:4] = 1 -> unused inputs
202 //       bits[3:0] = 0 -> stepper outputs (step gates: step_N = out[N] &
203 //       ~dir[N])
204 //   Buttons are on gpio_in port (separate wires), DIR does not gate
205 //   them i.e. 0b11110000 = 0xF0
206 // =====
207 i2c_gpio_expander #(.DEVICE_ADDR(7'h27))

```

```

208         gpio_expander(.clk(sys_clk),
209                       .rst_n(sys_rst_n),
210                       .scl_i(i2c_scl),
211                       .sda_i(i2c_sda),
212                       // reads before tristate assignment
213                       .sda_oe(sda_oe),
214                       .gpio_out(i2c_gpio_out),
215                       .gpio_dir(i2c_gpio_dir),
216                       .gpio_invert(i2c_gpio_invert),
217                       .gpio_in(i2c_gpio_in));
218
219 // Stepper outputs gated by direction register – only drive when
220 // configured as output gpio_dir[n]=0 means output; gpio_dir[n]=1 means
221 // input (high-Z effectively)
222 assign step_1 = i2c_gpio_out[0] & ~i2c_gpio_dir[0];
223 assign step_2 = i2c_gpio_out[1] & ~i2c_gpio_dir[1];
224 assign step_3 = i2c_gpio_out[2] & ~i2c_gpio_dir[2];
225 assign step_4 = i2c_gpio_out[3] & ~i2c_gpio_dir[3];
226
227 // =====
228 // TX output buffer (compressed data from scheduler)
229 //   Scheduler writes -> buffer -> ESP reads on MISO
230 //   Max compressed frame ~200 bytes, 512 depth is plenty
231 // =====
232 reg [7:0] tx_buf [0:511];
233 reg [7:0] tx_rd_data;
234 reg [8:0] tx_wr_ptr;
235 reg [8:0] tx_rd_ptr;
236 reg [8:0] tx_len;
237 wire [7:0] tx_cur_byte = (tx_rd_ptr < tx_len) ? tx_rd_data : 8'd0;
238
239 always @(posedge sys_clk) begin
240     if (sched_tx_valid)
241         tx_buf[tx_wr_ptr] <= sched_tx_data;
242     tx_rd_data <= tx_buf[tx_rd_ptr];
243 end
244 wire esp_tx_next;
245
246 // =====
247 // ESP SPI Interface
248 // =====
249 esp_interface esp_slave(.clk(sys_clk),
250                        .rst_n(sys_rst_n),
251                        .esp_mosi(esp_mosi),
252                        .esp_miso(esp_miso),
253                        .esp_sclk(esp_sclk),
254                        .esp_cs_n(esp_cs_n),
255                        .rx_data(esp_rx_data),
256                        .rx_valid(esp_rx_valid),
257                        .rx_ready(esp_rx_ready),
258                        .tx_data(tx_cur_byte),
259                        .tx_next(esp_tx_next),
260                        .cs_active_sync(esp_cs_active));
261
262 // =====
263 // BSRAM Image Buffer
264 // =====
265 bram_image_buffer image_buf(.clk_wr(sys_clk),
266                             .we(bram_write_en),
267                             .addr_wr(bram_write_addr),
268                             .data_wr(bram_write_data),
269                             .clk_rd(sys_clk),
270                             .addr_rd(lcd_bram_addr),
271                             .data_rd(bram_read_data));
272
273 // =====
274 // SPI Receive Logic – branchless
275 // =====
276 reg esp_cs_active_prev;
277

```

```

278 // cs_start: CS just asserted (rising edge of cs_active = falling edge
279 // of cs_n) cs_end: CS just deasserted (falling edge of cs_active =
280 // rising edge of cs_n)
281 wire cs_start = ~esp_cs_active_prev & esp_cs_active;
282 wire cs_end = esp_cs_active_prev & ~esp_cs_active;
283
284 always @(posedge sys_clk or negedge sys_rst_n) begin
285     if (!sys_rst_n)
286         esp_cs_active_prev <= 1'b0;
287     else
288         esp_cs_active_prev <= esp_cs_active;
289 end
290
291 wire rx_write = esp_rx_valid & esp_cs_active;
292 wire addr_last = (bram_write_addr == BRAM_SIZE - 1);
293 wire [14 : 0] addr_inc = (bram_write_addr + 15'd1) & {15{~addr_last}};
294
295 wire [14 : 0] addr_on_write = (addr_inc & {15{rx_write & ~cs_start}});
296 wire [14 : 0] addr_on_hold =
297     (bram_write_addr & {15{~rx_write & ~cs_start}});
298 wire [14 : 0] addr_next = addr_on_write | addr_on_hold;
299
300 wire next_receiving = receiving ^ ((receiving ^ 1'b1) & cs_start) ^
301     ((receiving ^ 1'b0) & (cs_end & receiving));
302
303 wire next_frame_ready = frame_ready ^
304     ((frame_ready ^ 1'b0) & cs_start) ^
305     ((frame_ready ^ 1'b1) & (cs_end & receiving));
306
307 always @(posedge sys_clk or negedge sys_rst_n) begin
308     if (!sys_rst_n) begin
309         bram_write_addr <= 15'd0;
310         bram_write_data <= 8'd0;
311         bram_write_en <= 1'b0;
312         receiving <= 1'b0;
313         frame_ready <= 1'b0;
314     end else begin
315         bram_write_addr <= addr_next;
316         bram_write_data <= esp_rx_data;
317         bram_write_en <= rx_write;
318         receiving <= next_receiving;
319         frame_ready <= next_frame_ready;
320     end
321 end
322
323 // =====
324 // SPI -> PSRAM write path
325 // Buffer byte pairs, issue word writes.
326 // Runs during SPI receive (receiving == 1).
327 // =====
328 reg [7:0] byte_hold;
329 reg byte_phase;
330
331 always @(posedge sys_clk or negedge sys_rst_n) begin
332     if (!sys_rst_n) begin
333         spi_psram_write <= 1'b0;
334         spi_psram_addr <= 22'd0;
335         spi_psram_din <= 16'd0;
336         byte_hold <= 8'd0;
337         byte_phase <= 1'b0;
338     end else begin
339         spi_psram_write <= 1'b0;
340
341         if (cs_start) begin
342             spi_psram_addr <= cur_base;
343             byte_phase <= 1'b0;
344         end
345
346         if (esp_rx_valid & receiving & ~psram_busy) begin
347             if (~byte_phase) begin

```

```

348         byte_hold <= esp_rx_data;
349         byte_phase <= 1'b1;
350     end else begin
351         spi_psram_din <= {esp_rx_data, byte_hold};
352         spi_psram_write <= 1'b1;
353         spi_psram_addr <= spi_psram_addr + 22'd2;
354         byte_phase <= 1'b0;
355     end
356 end
357
358 end
359
360 // =====
361 // TX buffer write (scheduler -> buffer) and read (MISO out)
362 // =====
363 always @(posedge sys_clk or negedge sys_rst_n) begin
364     if (!sys_rst_n) begin
365         tx_wr_ptr <= 9'd0;
366         tx_rd_ptr <= 9'd0;
367         tx_len <= 9'd0;
368     end else begin
369         // Scheduler start: reset write pointer
370         if (cs_end & receiving) begin
371             tx_wr_ptr <= 9'd0;
372         end
373
374         // Scheduler writes compressed bytes
375         if (sched_tx_valid) begin
376             tx_wr_ptr <= tx_wr_ptr + 9'd1;
377         end
378
379         // Latch length when scheduler finishes
380         if (sched_frame_done)
381             tx_len <= tx_wr_ptr;
382
383         // Reset read pointer on new SPI transaction
384         if (cs_start)
385             tx_rd_ptr <= 9'd0;
386
387         // Advance read pointer on each MISO byte sent
388         if (esp_tx_next & esp_cs_active)
389             tx_rd_ptr <= tx_rd_ptr + 9'd1;
390     end
391 end
392
393 // =====
394 // PSRAM ping-pong frame regions
395 //   Region A: 0x000000 (76800 bytes = 320*240)
396 //   Region B: 0x012C00
397 //   SPI writes to cur_base, scheduler diffs cur vs prev.
398 //   Swap after each frame.
399 // =====
400 localparam [21:0] REGION_A = 22'h000000;
401 localparam [21:0] REGION_B = 22'h012C00;
402
403 reg frame_sel;
404 wire [21:0] cur_base = frame_sel ? REGION_B : REGION_A;
405 wire [21:0] prev_base = frame_sel ? REGION_A : REGION_B;
406
407 always @(posedge sys_clk or negedge sys_rst_n) begin
408     if (!sys_rst_n)
409         frame_sel <= 1'b0;
410     else if (sched_frame_done)
411         frame_sel <= ~frame_sel;
412 end
413
414 // =====
415 // Block Scheduler (WHT compress pipeline)
416 // =====
417 wire sched_frame_done;

```

```

418     wire sched_tx_valid;
419     wire [7:0] sched_tx_data;
420
421     block_sched #(.W(16), .TOP_K(5)) sched(
422         .clk(sys_clk),
423         .rst_n(sys_rst_n),
424         .start(cs_end & receiving),
425         .frame_done(sched_frame_done),
426         .psram_read(sched_psram_read),
427         .psram_addr(sched_psram_addr),
428         .psram_rdata(psram_dout),
429         .psram_busy(psram_busy),
430         .tx_valid(sched_tx_valid),
431         .tx_data(sched_tx_data),
432         .tx_ready(1'b1),
433         .cur_base(cur_base),
434         .prev_base(prev_base)
435     );
436
437     // =====
438     // LCD Controller
439     // =====
440     lcd_controller lcd_ctrl(.clk(sys_clk),
441                             .rst_n(sys_rst_n),
442                             .btn(btn),
443                             .bram_addr(lcd_bram_addr),
444                             .bram_data(bram_read_data),
445                             .lcd_clk(lcd_clk),
446                             .lcd_hsync(lcd_hsync),
447                             .lcd_vsync(lcd_vsync),
448                             .lcd_de(lcd_de),
449                             .lcd_r(lcd_r),
450                             .lcd_g(lcd_g),
451                             .lcd_b(lcd_b));
452
453     // =====
454     // Debug LEDs
455     // =====
456     assign led[0] = receiving;
457     assign led[1] = frame_ready;
458     assign led[2] = esp_cs_active;
459     assign led[3] = esp_rx_valid;
460     assign led[4] = |i2c_gpio_out[3 : 0]; // any stepper active
461     assign led[5] = |btn_sync_1[2 : 0]; // any button pressed
462 endmodule

```


fpga/src/rtl/wht2d.v

```
1  /*
2  * wht2d.v -- Single wht8 wrapper
3  *
4  * Caller feeds one row or column (8 elements) per cycle,
5  * latches results back. 16 passes = full 2D WHT on 8x8 block.
6  */
7
8  module wht2d
9      #(parameter W = 16)
10     (input wire signed [W*8-1:0] x,
11      output wire signed [W*8-1:0] y);
12
13     wht8 #(.W(W)) u(
14         .x0(x[0*W +: W]),
15         .x1(x[1*W +: W]),
16         .x2(x[2*W +: W]),
17         .x3(x[3*W +: W]),
18         .x4(x[4*W +: W]),
19         .x5(x[5*W +: W]),
20         .x6(x[6*W +: W]),
21         .x7(x[7*W +: W]),
22         .y0(y[0*W +: W]),
23         .y1(y[1*W +: W]),
24         .y2(y[2*W +: W]),
25         .y3(y[3*W +: W]),
26         .y4(y[4*W +: W]),
27         .y5(y[5*W +: W]),
28         .y6(y[6*W +: W]),
29         .y7(y[7*W +: W])
30     );
31 endmodule
```

fpga/src/rtl/wht8.v

```
1  /*
2  * wht8.v -- As the name implies, 8-point combinational WHT
3  * Author: Cole Zenk
4  *
5  * Charecteristics:
6  * - 1 dimentional
7  * - Combinational butterfly
8  * - Three stages: Stride 1, 2, 3
9  * - Purely addition or subtraction
10 *
11 */
12
13 module wht8
14     #(parameter W=16)
15     (
16         input  wire signed [W-1 : 0] x0, x1, x2, x3,
17         input  wire signed [W-1 : 0] x4, x5, x6, x7,
18         output wire signed [W-1 : 0] y0, y1, y2, y3,
19         output wire signed [W-1 : 0] y4, y5, y6, y7
20     );
21
22     // stride=1: Butterfly adjacent pairs
23     wire signed [W-1 : 0] s1_0 = x0 + x1;
24     wire signed [W-1 : 0] s1_1 = x0 - x1;
25     wire signed [W-1 : 0] s1_2 = x2 + x3;
26     wire signed [W-1 : 0] s1_3 = x2 - x3;
27     wire signed [W-1 : 0] s1_4 = x4 + x5;
28     wire signed [W-1 : 0] s1_5 = x4 - x5;
29     wire signed [W-1 : 0] s1_6 = x6 + x7;
30     wire signed [W-1 : 0] s1_7 = x6 - x7;
31
32     // stride=2: Butterfly pairs at distance 2
33     wire signed [W-1 : 0] s2_0 = s1_0 + s1_2;
34     wire signed [W-1 : 0] s2_1 = s1_0 - s1_2;
35     wire signed [W-1 : 0] s2_2 = s1_1 + s1_3;
36     wire signed [W-1 : 0] s2_3 = s1_1 - s1_3;
37     wire signed [W-1 : 0] s2_4 = s1_4 + s1_6;
38     wire signed [W-1 : 0] s2_5 = s1_4 - s1_6;
39     wire signed [W-1 : 0] s2_6 = s1_5 + s1_7;
40     wire signed [W-1 : 0] s2_7 = s1_5 - s1_7;
41
42     // stride=4: Butterfly pairs at distance 4
43     assign y0 = s2_0 + s2_4; assign y4 = s2_0 - s2_4;
44     assign y1 = s2_1 + s2_5; assign y5 = s2_1 - s2_5;
45     assign y2 = s2_2 + s2_6; assign y6 = s2_2 - s2_6;
46     assign y3 = s2_3 + s2_7; assign y7 = s2_3 - s2_7;
47
48 endmodule
```